Руководство Биджа по сетевому программированию

Брайан «Бидж Йоргенсен» Холл

Работа с интернет-сокетами

Русский перевод версии 3.3.3 · 15 октября 2026 г.

Оригинал: beej.us/guide/bgnet

Содержание

Глава 1. Введение

- 1.1. Аудитория
- 1.2. Платформа и компилятор
- 1.3. Официальная страница и печатные книги
- 1.4. Примечание для программистов Solaris/SunOS/illumos
- 1.5. Примечание для программистов Windows
- 1.6. Правила переписки
- 1.7. Зеркала
- 1.8. Примечание для переводчиков
- 1.9. Авторское право, распространение и правовые положения
- 1.10. Посвящение
- 1.11. Информация об издании

Глава 2. Что такое сокет?

- 2.1. Два типа интернет-сокетов
- 2.2. Низкоуровневый вздор и теория сетей

Глава 3. IP-адреса, структуры и преобразование данных

- 3.1. IP-адреса версий 4 и 6
- 3.2. Порядок байтов
- 3.3. Структуры
- 3.4. IP-адреса, часть вторая

Глава 4. Переход с IPv4 на IPv6

5. Системные вызовы или ничего

- 5.1. getaddrinfo() — готовимся к запуску!
- 5.2. socket() — получаем дескриптор файла!
- 5.3. bind() — какой у меня порт?
- 5.4. connect() — эй, ты!
- 5.5. listen() — кто-нибудь мне позвонит?
- 5.6. accept() — «Спасибо, что позвонили на порт 3490»
- 5.7. send() и recv() — поговори со мной!
- 5.8. sendto() и recvfrom() — разговор на языке DGRAM
- 5.9. close() и shutdown() — убирайся с глаз моих!
- 5.10. getpeername() — кто ты?
- 5.11. gethostname() — кто я?

6. Основы клиент-серверной модели

- 6.1. Простой потоковый сервер
- 6.2. Простой потоковый клиент
- 6.3. Дейтаграммные сокеты

7. Чуть более сложные приёмы

7.1. Блокирование

7.2. poll() — синхронное мультиплексирование ввода-вывода

7.3. select() — синхронное мультиплексирование ввода-вывода по старинке

7.4. Обработка частичной отправки send()

7.5. Сериализация — как упаковать данные

7.6. Возвращение инкапсуляции данных

7.7. Широковещательные пакеты — привет, мир!

Глава 8. Частые вопросы

Где взять эти заголовочные файлы?

Что делать, когда bind() сообщает «Address already in use»?

Как получить список открытых сокетов?

Как посмотреть таблицу маршрутизации?

Как запустить клиент и сервер, если у меня только один компьютер? Разве для сетевых программ не нужна сеть?

Как узнать, что другая сторона закрыла соединение?

Как написать утилиту ping? Что такое ICMP? Где почитать про сырые сокеты и SOCK_RAW?

Как изменить или сократить время ожидания connect()?

Как собрать программу под Windows?

Как собрать программу под Solaris/SunOS? При компиляции постоянно появляются ошибки компоновщика!

Почему select() постоянно прерывается сигналом?

Как задать время ожидания для recv()?

Как зашифровать или сжать данные перед отправкой через сокет?

Что за PF_INET мне всё время встречается? Оно связано с AF_INET?

Как написать сервер, принимающий от клиента команды оболочки и выполняющий их?

Я отправляю много данных, но recv() получает за раз только 536 или 1460 байт. На локальной машине данные приходят целиком. Почему?

В Windows нет системного вызова fork() и структуры sigaction. Что делать?

Я за межсетевым экраном. Как сообщить внешним пользователям свой IP-адрес, чтобы они могли подключиться ко мне?

Как написать анализатор пакетов? Как включить неразборчивый режим сетевого интерфейса Ethernet?

Как задать собственное время ожидания для TCP- или UDP-сокета?

Как узнать, какие порты свободны? Есть ли список официальных номеров портов?

Глава 9. Страницы руководства

9.1 accept()

9.2 bind()

9.3 connect()

9.4 close()

9.5 getaddrinfo(), freeaddrinfo(), gai_strerror()

9.6 gethostname()

9.7 gethostbyname(), gethostbyaddr()

9.8 getnameinfo()

9.9 getpeername()

9.10 errno

9.11 fcntl()

9.12 htons(), htonl(), ntohs(), ntohl()
9.13 inet_ntoa(), inet_aton(), inet_addr()
9.14 inet_ntop(), inet_pton()
9.15 listen()
9.16 perror(), strerror()
9.17 poll()
9.18 recv(), recvfrom()
9.19 select()
9.20 setsockopt(), getsockopt()
9.21 send(), sendto()
9.22 shutdown()
9.23 socket()
9.24 struct sockaddr и её друзья

Глава 10. Дополнительные источники

10.1 Книги
10.2 Интернет-источники
10.3 RFC

Глава 1. Введение

Привет! Программирование сокетов вызывает уныние? Страницы man слишком запутанны? Вы хотите создавать крутые интернет-программы, но у вас нет времени продираться через кучу структур, выясняя, нужно ли вызвать `bind()` перед `connect()`, и так далее?

Что ж, угадайте! Я уже проделал эту неприятную работу и очень хочу поделиться результатами со всеми! Вы пришли по адресу. Этот документ должен помочь обычному толковому программисту на C разобраться во всей этой сетевой кутерьме.

И посмотрите: я наконец догнал будущее (как раз вовремя!) и обновил руководство для IPv6! Приятного чтения!

1.1. Аудитория

Этот документ написан как учебное пособие, а не полный справочник. Вероятно, он принесёт больше всего пользы тем, кто только начинает программировать сокеты и ищет, за что зацепиться. Ни в коем случае нельзя назвать его исчерпывающим руководством по программированию сокетов.

Но, надеюсь, его будет достаточно, чтобы страницы man начали обретать смысл… :-)

1.2. Платформа и компилятор

Приведённый здесь код компилировался на Linux-ПК компилятором GNU `gcc`. Тем не менее он должен собираться почти на любой платформе, где есть `gcc`. Разумеется, это не относится к программированию для Windows — см. соответствующий раздел ниже.

1.3. Официальная страница и печатные книги

Официальный адрес этого документа: https://beej.us/guide/bgnet/. Там же вы найдёте примеры кода и переводы руководства на различные языки.

Купить красиво переплетённые бумажные экземпляры (некоторые называют их «книгами») можно по адресу https://beej.us/guide/url/bgbuy. Я буду благодарен за покупку: она помогает мне и дальше писать документацию!

1.4. Примечание для программистов Solaris/SunOS/illumos

При компиляции под разновидность Solaris или SunOS необходимо указать дополнительные параметры командной строки, чтобы подключить нужные библиотеки. Просто добавьте `-lnsl -lsocket -lresolv` в конец команды компиляции:

```
cc -o server server.c -lnsl -lsocket -lresolv
```

Если ошибки остались, попробуйте дополнительно добавить `-lxnet` в конец команды. Я точно не знаю, что это делает, но некоторым, похоже, это необходимо.

Ещё одно возможное затруднение связано с вызовом `setsockopt()`. Его прототип отличается от прототипа в моей системе Linux, поэтому вместо

```
int yes=1;
```

напишите:

```
char yes='1';
```

У меня нет компьютера Sun, поэтому я не проверял приведённые выше сведения: их прислали мне по электронной почте.

1.5. Примечание для программистов Windows

Раньше в этом месте руководства я немного ругал Windows просто потому, что она мне не очень нравилась. Но потом Windows и Microsoft как компания стали значительно лучше. Windows 10 вместе с WSL (см. ниже) действительно образует неплохую операционную систему. Жаловаться особенно не на что.

Ну, почти: например, я пишу это в 2025 году на ноутбуке 2015 года, на котором когда-то работала Windows 10. Со временем он стал слишком медленным, я установил на него Linux и с тех пор им пользуюсь.

Но теперь есть Windows 11, которой, похоже, требуется более мощное оборудование, чем Windows 10. Мне это не нравится. Операционная система должна быть как можно менее заметной и не заставлять тратить больше денег. Дополнительная мощность процессора должна доставаться приложениям, а не ОС! Кроме того, Microsoft знает, чего вы хотите, а хотите вы больше рекламы! Правда? В вашей операционной системе! Разве вам её не хватало? Теперь она есть в Windows 11.

Так что… я по-прежнему советую попробовать вместо Windows Linux, BSD, illumos или другую разновидность Unix. Откуда тут взялась эта трибуна?

Впрочем, людям нравится то, что нравится, и пользователи Windows будут рады узнать, что эти сведения в основном применимы и к Windows с несколькими небольшими изменениями.

Вам стоит серьёзно рассмотреть Windows Subsystem for Linux. Она позволяет установить в Windows 10 нечто вроде виртуальной машины Linux. Это точно поможет вам приступить к делу: вы сможете собирать и запускать эти программы как есть.

Ещё можно установить Cygwin — набор инструментов Unix для Windows. Я слышал, что с ним все эти программы компилируются без изменений, но сам не пробовал.

Некоторые захотят сделать всё чисто по-виндовому. Это очень смело, и вот что вам нужно сделать: немедленно бегите за Unix! Нет-нет, я шучу. В наши дни мне положено быть более дружелюбным к Windows…

Ладно, ладно, перейду к делу.

Прежде всего игнорируйте почти все системные заголовочные файлы, которые я упоминаю. Вместо них подключите:

```
#include <winsock2.h>
#include <ws2tcpip.h>
```

`winsock2` — «новая» (примерно с 1994 года) версия библиотеки сокетов Windows. К сожалению, если подключить `windows.h`, он автоматически подключит старый заголовок `winsock.h` (версия 1), конфликтующий с `winsock2.h`! Весёлые времена.

Поэтому, если нужно подключить `windows.h`, определите макрос, запрещающий подключение старого заголовка:

```
#define WIN32_LEAN_AND_MEAN // Сначала это...
#include <windows.h>        // Теперь можно подключить это.
#include <winsock2.h>       // И это.
```

Стоп! Кроме того, прежде чем делать что-либо с библиотекой сокетов, нужно вызвать `WSAStartup()`. Ей передаётся желаемая версия Winsock (например, 2.2); затем можно проверить результат и убедиться, что она доступна. Код выглядит примерно так:

```
#include <winsock2.h>

{
    WSADATA wsaData;

    if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
        fprintf(stderr, "WSAStartup failed.\n");
        exit(1);
    }

    if (LOBYTE(wsaData.wVersion) != 2 ||
        HIBYTE(wsaData.wVersion) != 2)
    {
        fprintf(stderr, "Version 2.2 of Winsock not available.\n");
        WSACleanup();
        exit(2);
    }
```

Обратите внимание на вызов `WSACleanup()`: его следует вызвать, когда вы закончите работу с библиотекой Winsock.

Также нужно указать компилятору подключить библиотеку Winsock: для Winsock 2 она называется `ws2_32.lib`. В VC++ это можно сделать через меню Project, пункт Settings…. Откройте вкладку Link и найдите поле «Object/library modules». Добавьте в список `ws2_32.lib` (или другую нужную библиотеку). По крайней мере, так мне говорили.

После этого остальные примеры пособия в целом должны работать, за некоторыми исключениями. Например, закрывать сокет с помощью `close()` нельзя — вместо этого используйте `closesocket()`. Кроме того, `select()` работает только с дескрипторами сокетов, но не файлов (например, 0 для `stdin`).

Также можно использовать класс сокетов `CSocket`. Подробнее см. справку компилятора. За дополнительной информацией о Winsock обратитесь к официальной странице Microsoft.

Наконец, я слышал, что в Windows нет системного вызова `fork()`, который, к сожалению, используется в некоторых примерах. Возможно, придётся подключить библиотеку POSIX или что-то подобное, чтобы он заработал, либо использовать `CreateProcess()`. `fork()` не принимает аргументов, а `CreateProcess()` принимает около 48 миллиардов. Если вы к этому не готовы, `CreateThread()` немного проще для понимания… К сожалению, обсуждение многопоточности выходит за рамки этого документа. Я не могу рассказать обо всём на свете!

И совсем напоследок: Стивен Митчелл перенёс ряд примеров на Winsock. Посмотрите.

1.6. Правила переписки

Как правило, я готов помочь с вопросами по электронной почте, так что смело пишите, но ответ гарантировать не могу. Я очень занят, и иногда просто не могу ответить на ваш вопрос. В таких случаях я обычно удаляю письмо. Ничего личного: просто у меня никогда не найдётся времени на подробный ответ, который вам нужен.

Как правило, чем сложнее вопрос, тем меньше вероятность ответа. Если перед отправкой вы сузите круг вопроса и укажете все необходимые сведения (например, платформу, компилятор, сообщения об ошибках и всё остальное, что, по вашему мнению, поможет разобраться), шансы получить ответ заметно возрастут. Дополнительные советы см. в документе Эрика Реймонда «Как задавать вопросы правильно».

Если ответа нет, поработайте над проблемой ещё, попытайтесь найти решение и, если оно всё ещё ускользает, напишите мне снова с найденной информацией. Надеюсь, её хватит, чтобы я смог помочь.

После всех этих наставлений о том, как мне писать и как не писать, хочу сказать, что искренне ценю все похвалы, которых руководство удостоилось за эти годы. Они здорово поднимают настроение, и мне радостно слышать, что оно приносит пользу! :-) Спасибо!

1.7. Зеркала

Вы можете свободно создавать публичные и частные зеркала этого сайта. Если вы создали публичное зеркало и хотите, чтобы я дал на него ссылку с главной страницы, напишите мне: beej@beej.us.

1.8. Примечание для переводчиков

Если хотите перевести руководство на другой язык, напишите мне на beej@beej.us, и я дам ссылку на ваш перевод с главной страницы. Не стесняйтесь добавить в перевод своё имя и контактные данные.

Исходный документ Markdown использует кодировку UTF-8.

Обратите внимание на ограничения лицензии в разделе «Авторское право, распространение и правовые положения» ниже.

Если хотите, чтобы я разместил перевод у себя, просто попросите. Если вы предпочитаете разместить его самостоятельно, я также дам ссылку; мне подходят оба варианта.

1.9. Авторское право, распространение и правовые положения

«Beej’s Guide to Network Programming», Copyright © 2019 Brian «Beej Jorgensen» Hall.

За отдельными исключениями для исходного кода и переводов, описанными ниже, это произведение распространяется по лицензии Creative Commons Attribution-Noncommercial-No Derivative Works 3.0. С текстом лицензии можно ознакомиться по адресу https://creativecommons.org/licenses/by-nc-nd/3.0/ либо запросить его письмом: Creative Commons, 171 Second Street, Suite 300, San Francisco, California, 94105, USA.

Одно специальное исключение из запрета производных произведений: руководство можно свободно переводить на любой язык при условии точности перевода и воспроизведения руководства целиком. На перевод распространяются те же лицензионные ограничения, что и на оригинал. Перевод также может включать имя и контактные данные переводчика.

Исходный код на C, представленный в этом документе, передаётся в общественное достояние и полностью свободен от лицензионных ограничений.

Преподавателям разрешается и рекомендуется рекомендовать руководство студентам или предоставлять им его копии.

Если стороны письменно не договорились об ином, автор предоставляет произведение «как есть» и не даёт никаких заверений или гарантий любого рода, явных, подразумеваемых, установленных законом или иных, в отношении произведения, включая, помимо прочего, гарантии права собственности, товарной пригодности, пригодности для определённой цели, отсутствия нарушения прав, скрытых или иных дефектов, точности, наличия или отсутствия ошибок, независимо от того, обнаружимы ли они.

За исключением случаев, предусмотренных применимым законодательством, автор ни при каких обстоятельствах не несёт перед вами ответственности на каком-либо правовом основании за особый, случайный, косвенный, штрафной или примерный ущерб, возникший вследствие использования произведения, даже если автор был уведомлён о возможности такого ущерба.

За дополнительными сведениями обращайтесь по адресу beej@beej.us.

1.10. Посвящение

Спасибо всем, кто помогал мне писать это руководство в прошлом и поможет в будущем. Спасибо также всем создателям свободного программного обеспечения и пакетов, которые я использую при работе над ним: GNU, Linux, Slackware, vim, Python, Inkscape, pandoc и многим другим. И наконец, огромное спасибо буквально тысячам читателей, приславших предложения по улучшению и слова поддержки.

Я посвящаю это руководство некоторым из моих главных героев и вдохновителей в мире компьютеров: Дональду Кнуту, Брюсу Шнайеру, У. Ричарду Стивенсу и Возу, своим читателям и всему сообществу свободного и открытого программного обеспечения.

1.11. Информация об издании

Эта книга написана на Markdown в редакторе vim на компьютере с Arch Linux и инструментами GNU. «Иллюстрация» обложки и диаграммы созданы в Inkscape. Markdown преобразуется в HTML и LaTeX/PDF с помощью Python, Pandoc и XeLaTeX с использованием шрифтов Liberation. Весь набор инструментов состоит из свободного программного обеспечения с открытым исходным кодом.

Глава 2. Что такое сокет?

Вы постоянно слышите разговоры о «сокетах» и, возможно, хотите узнать, что же это такое. Вот ответ: это способ взаимодействовать с другими программами с помощью стандартных файловых дескрипторов Unix.

Что?

Хорошо. Возможно, вы слышали, как какой-нибудь Unix-хакер восклицал: «Да в Unix всё — файл!» Вероятно, речь шла о том, что любую операцию ввода-вывода программы Unix выполняют, читая файловый дескриптор или записывая в него данные. Файловый дескриптор — просто целое число, связанное с открытым файлом. Но вот в чём загвоздка: этим файлом может быть сетевое соединение, FIFO, канал, терминал, обычный файл на диске или почти что угодно ещё. В Unix всё — файл! Поэтому, когда вы захотите общаться с другой программой через интернет, поверьте, делать это придётся через файловый дескриптор.

«И где же взять этот файловый дескриптор для сетевого общения, умник?» — возможно, это сейчас последний вопрос, который вас занимает, но я всё равно отвечу. Вы вызываете системную функцию `socket()`. Она возвращает дескриптор сокета, а общаться через него можно с помощью специальных вызовов `send()` и `recv()` (см. `man send` и `man recv`).

«Но постойте! — можете воскликнуть вы. — Если это файловый дескриптор, почему, именем Нептуна, нельзя просто использовать обычные вызовы `read()` и `write()` для работы с сокетом?» Короткий ответ: «Можно!» Более длинный: «Можно, но `send()` и `recv()` дают значительно больше возможностей управлять передачей данных».

Что дальше? Например, вот что: сокеты бывают разных видов. Есть адреса DARPA Internet (интернет-сокеты), имена путей на локальном узле (сокеты Unix), адреса CCITT X.25 (сокеты X.25, которые можно спокойно игнорировать) и, вероятно, многие другие — в зависимости от используемой разновидности Unix. В этом документе рассматривается только первый вид: интернет-сокеты.

2.1. Два типа интернет-сокетов

Что такое? Интернет-сокеты бывают двух типов? Да. Ну, нет. Я солгал. Их больше, но я не хотел вас пугать. Здесь я расскажу только о двух типах. За исключением этого предложения: «сырые сокеты» (*raw sockets*) тоже очень мощны, и вам стоит о них почитать.

Ладно, что же это за два типа? Первый — потоковые сокеты (*stream sockets*), второй — дейтаграммные сокеты (*datagram sockets*). Далее они могут называться соответственно `SOCK_STREAM` и `SOCK_DGRAM`. Дейтаграммные сокеты иногда называют сокетами без установления соединения. Хотя при желании для них можно вызвать `connect()` — см. раздел о `connect()` ниже.

Потоковые сокеты предоставляют надёжный двусторонний поток связи с установленным соединением. Если отправить в сокет два элемента в порядке «1, 2», на другом конце они придут в порядке «1, 2». И без ошибок. Я настолько уверен в отсутствии ошибок, что просто заткну уши пальцами и запою «ла-ла-ла-ла», если кто-нибудь попробует утверждать обратное.

Где используются потоковые сокеты? Вы наверняка слышали о приложениях `telnet` и `ssh`. Они используют потоковые сокеты: все введённые символы должны приходить в том же порядке, в каком вы их набрали, верно? Кроме того, веб-браузеры используют протокол HTTP, который применяет потоковые

сокеты для получения страниц. Если подключиться через `telnet` к веб-сайту на порт 80, ввести `GET / HTTP/1.0` и дважды нажать Enter, в ответ вы получите HTML.

Если `telnet` не установлен и вы не хотите его устанавливать, либо он капризничает при подключении, к руководству прилагается похожая программа telnot. Её должно хватить для примеров руководства. Заметьте, что telnet на самом деле является отдельным сетевым протоколом; `telnot` его совсем не реализует.

Как потоковые сокеты достигают такого высокого качества передачи? Они используют протокол управления передачей — Transmission Control Protocol, или TCP. Подробнейшие сведения о TCP см. в RFC 793. TCP обеспечивает последовательную доставку данных без ошибок. Возможно, вы уже слышали «TCP» как первую половину «TCP/IP», где IP — Internet Protocol (RFC 791). IP прежде всего занимается маршрутизацией в интернете и обычно не отвечает за целостность данных.

Здорово. А что насчёт дейтаграммных сокетов? Почему они называются сокетами без соединения? Что вообще происходит и почему они ненадёжны? Вот факты: отправленная дейтаграмма может дойти, а может и не дойти. Дейтаграммы могут прийти не по порядку. Если пакет доходит, данные внутри него будут без ошибок.

Дейтаграммные сокеты тоже используют IP для маршрутизации, но вместо TCP применяют User Datagram Protocol, или UDP (RFC 768).

Почему они считаются работающими без соединения? В основном потому, что вам не нужно поддерживать открытое соединение, как при работе с потоковым сокетом. Вы просто создаёте пакет, добавляете IP-заголовок с данными адресата и отправляете его. Соединение не требуется. Обычно такие сокеты применяют, когда стек TCP недоступен либо потеря нескольких пакетов сама по себе не означает конец Вселенной. Примеры: `tftp` — упрощённый протокол передачи файлов, младший брат FTP; `dhcpcd` — клиент DHCP; многопользовательские игры, потоковое аудио, видеоконференции и так далее.

«Минутку! `tftp` и `dhcpcd` используются для передачи двоичных приложений с одного компьютера на другой! Нельзя терять данные, если приложение должно работать после получения! Что это за тёмная магия?»

Что ж, мой друг-человек, у `tftp` и подобных программ есть собственный протокол поверх UDP. Например, протокол `tftp` требует, чтобы на каждый отправленный пакет получатель отвечал пакетом «Я получил!» — подтверждением, или ACK. Если отправитель исходного пакета не получает ответа, скажем, за пять секунд, он повторяет отправку, пока наконец не получит ACK. Эта процедура подтверждения очень важна при создании надёжных приложений на `SOCK_DGRAM`.

Для приложений, допускающих потери, например игр, аудио или видео, потерянные пакеты можно просто игнорировать или попробовать хитро компенсировать. Игроки в Quake знают проявление этого эффекта под техническим названием «проклятый лаг». Слово «проклятый» здесь заменяет любое крайне непристойное выражение.

Зачем использовать ненадёжный базовый протокол? По двум причинам: скорость и скорость. Отправить и забыть гораздо быстрее, чем следить за успешной доставкой, правильным порядком и всем прочим. Если вы отправляете сообщения чата, TCP отлично подойдёт. Но если вы 40 раз в секунду передаёте обновления положения игроков в мире, потеря одного-двух обновлений может быть не так важна, и UDP окажется хорошим выбором.

2.2. Низкоуровневый вздор и теория сетей

Раз уж я упомянул слои протоколов, пора поговорить о том, как на самом деле работают сети, и показать несколько примеров построения пакетов `SOCK_DGRAM`. На практике вы, вероятно, можете пропустить этот раздел. Однако он даст полезные базовые сведения.



Рисунок 2.1. Инкапсуляция данных.

Эй, ребята, пора узнать об *инкапсуляции данных*! Это очень-очень важно. Настолько важно, что вы, возможно, узнаете об этом на курсе компьютерных сетей в университете Чико ;-) Суть такова: рождается пакет; первый протокол, скажем TFTP, оборачивает — инкапсулирует — его заголовком и, реже, концевиком. Затем весь результат вместе с заголовком TFTP снова инкапсулируется следующим протоколом, например UDP, потом ещё раз — IP, а затем последним протоколом аппаратного, или физического, уровня, например Ethernet.

Когда другой компьютер получает пакет, оборудование снимает заголовок Ethernet, ядро снимает заголовки IP и UDP, программа TFTP снимает заголовок TFTP и наконец получает данные.

Теперь я наконец могу рассказать о знаменитой многоуровневой сетевой модели, также известной как ISO/OSI. Она описывает систему сетевых функций, имеющую немало преимуществ перед другими моделями. Например, вы можете писать одни и те же программы с сокетами, не заботясь о физической передаче данных — последовательное соединение, тонкий Ethernet, AUI или что-то другое, — потому что этим занимаются нижние уровни. Физическое сетевое оборудование и топология прозрачны для программиста, работающего с сокетами.

Без дальнейших церемоний перечислю уровни полной модели. Запомните их для экзамена по сетям:

1. Прикладной.
2. Представления.
3. Сеансовый.
4. Транспортный.
5. Сетевой.
6. Канальный.
7. Физический.

Физический уровень — это оборудование: последовательные соединения, Ethernet и так далее. Прикладной уровень находится настолько далеко от физического, насколько можно представить: здесь пользователи взаимодействуют с сетью.

Модель настолько общая, что при желании её, наверное, можно использовать как руководство по ремонту автомобилей. Многоуровневая модель, более согласующаяся с Unix, могла бы выглядеть так:

1. Прикладной уровень — `telnet`, `ftp` и тому подобное.

2. Транспортный уровень между узлами — TCP, UDP.
3. Межсетевой уровень — IP и маршрутизация.
4. Уровень доступа к сети — Ethernet, Wi-Fi и другие технологии.

Теперь вы, вероятно, видите соответствие между этими уровнями и инкапсуляцией исходных данных.

Видите, сколько работы требуется для создания простого пакета? Ух ты! И вам приходится вручную набирать заголовки через `cat`! Шучу. Для потокового сокета достаточно отправить данные вызовом `send()`. Для дейтаграммного — инкапсулировать данные выбранным способом и отправить их через `sendto()`. Ядро само создаёт транспортный и межсетевой уровни, а оборудование отвечает за уровень доступа к сети. Ах, современные технологии.

На этом наше краткое путешествие в теорию сетей заканчивается. Ах да, я забыл рассказать всё, что хотел, о маршрутизации: ничего! Верно, я вообще не буду об этом говорить. Маршрутизатор снимает с пакета всё до IP-заголовка, обращается к таблице маршрутизации, бла-бла-бла. Если вам действительно очень интересно, обратитесь к RFC IP. Если вы никогда этого не узнаете, ничего страшного — жить будете.

Глава 3. IP-адреса, структуры и преобразование данных

Вот и та часть игры, где мы для разнообразия поговорим о коде.

Но сначала обсудим ещё немного не-кода! Ура! Сперва я хочу чуть-чуть рассказать об IP-адресах и портах, чтобы прояснить их. Затем поговорим о том, как API сокетов хранит и обрабатывает IP-адреса и другие данные.

3.1. IP-адреса версий 4 и 6

В старые добрые времена, когда Бена Кеноби ещё звали Оби-Ван Кеноби, существовала замечательная система сетевой маршрутизации под названием Internet Protocol Version 4, или IPv4. Её адреса состояли из четырёх байтов (то есть четырёх «октетов») и обычно записывались числами через точки, например `192.0.2.111`.

Наверняка вы их видели. На момент написания этого текста практически каждый интернет-сайт использует IPv4.

Все, включая Оби-Вана, были довольны. Всё было прекрасно, пока какой-то скептик по имени Винт Серф не предупредил, что адреса IPv4 скоро закончатся!

(Помимо предупреждения о грядущем мрачном и ужасном апокалипсисе IPv4, Винт Серф известен как отец интернета. Так что мне трудно оспаривать его суждения.)

Адреса закончатся? Как такое возможно? Ведь в 32-битном адресе IPv4 миллиарды вариантов. Неужели у нас действительно миллиарды компьютеров?

Да.

К тому же в начале, когда компьютеров было мало и всем казалось, что миллиард — невообразимо большое число, некоторым крупным организациям щедро выделили миллионы IP-адресов для собственного использования. Среди них Xerox, MIT, Ford, HP, IBM, GE, AT&T и небольшая компания под названием Apple, и это лишь некоторые.

На самом деле без нескольких временных мер адреса закончились бы давным-давно.

Но теперь мы живём в эпоху, когда об IP-адресе говорят применительно к каждому человеку, компьютеру, калькулятору, телефону, парковочному счётчику и, почему бы нет, каждому щенку. Так родился IPv6. Поскольку Винт Серф, вероятно, бессмертен (даже если его физическая оболочка, не дай бог, исчезнет, он, наверное, уже существует в глубинах Internet2 как сверхразумная программа вроде ELIZA), никому не хочется снова услышать от него «а я говорил», если в новой версии интернет-протокола не хватит адресов.

О чём это говорит? О том, что адресов нам нужно намного больше. Не вдвое, не в миллиард и не в тысячу триллионов раз больше, а в 79 МИЛЛИОНОВ МИЛЛИАРДОВ ТРИЛЛИОНОВ раз больше возможных адресов! Вот мы им покажем!

Вы спросите: «Бидж, это правда? У меня есть все основания не верить большим числам». Разница между 32 и 128 битами может показаться небольшой: всего на 96 бит больше, верно? Но помните, речь идёт о

степенях: 32 бита представляют примерно 4 миллиарда чисел (2^{32}), а 128 бит — примерно 340 триллионов триллионов триллионов чисел (действительно, 2^{128}). Это как миллион интернетов IPv4 на каждую отдельную звезду во Вселенной.

Забудьте и о привычных точках и числах IPv4. Теперь используется шестнадцатеричная запись, где каждая двухбайтовая группа отделена двоеточием:

```
2001:0db8:c9d2:aee5:73e3:934a:a5ae:9551
```

И это ещё не всё! Часто адрес содержит множество нулей; их последовательность можно сократить до двух двоеточий. Ведущие нули каждой пары байтов тоже можно опустить. Например, адреса в каждой паре эквивалентны:

```
2001:0db8:c9d2:0012:0000:0000:0000:0051
2001:db8:c9d2:12::51

2001:0db8:ab00:0000:0000:0000:0000:0000
2001:db8:ab00::

0000:0000:0000:0000:0000:0000:0000:0001
::1
```

Адрес `::1` — адрес обратной петли (loopback). Он всегда означает «компьютер, на котором я сейчас работаю». В IPv4 адрес обратной петли — `127.0.0.1`.

Наконец, вам может встретиться режим совместимости адресов IPv6 с IPv4. Например, чтобы представить IPv4-адрес `192.0.2.33` как IPv6-адрес, используют запись `::ffff:192.0.2.33`.

Вот это серьёзное развлечение.

Настолько серьёзное, что создатели IPv6 довольно беззаботно отрезали триллионы триллионов адресов для специальных нужд. Но их так много, что, откровенно говоря, кто их теперь считает? Оставшихся хватит с избытком на каждого мужчину, женщину, ребёнка, щенка и парковочный счётчик на каждой планете галактики. И поверьте, на каждой планете галактики есть парковочные счётчики. Вы же знаете, что это правда.

3.1.1. Подсети

По организационным причинам иногда удобно объявить: «Первая часть IP-адреса до этого бита — сетевая часть, а остальное — часть узла».

Например, в адресе IPv4 `192.0.2.12` можно считать первые три байта сетью, а последний — узлом. Другими словами, это узел 12 в сети `192.0.2.0` (обратите внимание, что байт узла мы обнулили).

А теперь ещё немного устаревших сведений! Готовы? В древние времена существовали «классы» подсетей: один, два или три первых байта адреса составляли сетевую часть. Если вам посчастливилось получить один байт для сети и три для узла, в сети могло быть 24 бита адресов узлов (около 16 миллионов). Это сеть «класса A». На другом конце спектра был «класс C»: три байта сети и один байт узла (256 узлов, за вычетом нескольких зарезервированных).

Как видите, сетей класса A было немного, класса C — огромное количество, а между ними находились сети класса B.

Сетевая часть IP-адреса описывается так называемой маской сети: применив к ней и IP-адресу побитовое И, получаем номер сети. Обычно маска выглядит примерно как `255.255.255.0`. Например, при такой маске и IP-адресе `192.0.2.12` сеть определяется как `192.0.2.12 AND 255.255.255.0`, что даёт `192.0.2.0`.

К сожалению, такая разбивка оказалась недостаточно гибкой для потребностей интернета: сети класса C быстро заканчивались, а сетей класса A точно уже не осталось, даже не спрашивайте. Чтобы это исправить, ответственные лица разрешили использовать в маске произвольное число битов, а не только 8, 16 или 24. Например, маска `255.255.255.252` задаёт 30 битов сети и 2 бита узла, позволяя получить четыре адреса узлов в сети. (Заметьте: маска ВСЕГДА состоит из последовательности единичных битов, за которыми следуют нулевые.)

Но использовать в качестве маски длинную цепочку чисел вроде `255.192.0.0` неудобно. Во-первых, не сразу понятно, сколько это битов; во-вторых, запись некомпактна. Поэтому появился новый, гораздо более удобный способ: после IP-адреса ставят косую черту и десятичное число битов сети. Например, `192.0.2.12/30`.

Для IPv6 это может выглядеть так: `2001:db8::/32` или `2001:db8:5413:4028::9db9/64`.

3.1.2. Номера портов

Вспомните, пожалуйста, многоуровневую сетевую модель, которую я представил ранее: в ней интернет-уровень (IP) отделён от транспортного уровня между узлами (TCP и UDP). Разберитесь с этим перед следующим абзацем.

Оказывается, помимо IP-адреса (используемого уровнем IP), существует другой адрес, который используют TCP (потоковые сокеты) и UDP (дейтаграммные сокеты). Это номер порта — 16-битное число, служащее чем-то вроде локального адреса соединения.

Представьте IP-адрес как адрес гостиницы, а номер порта — как номер комнаты. Неплохая аналогия; возможно, позже я придумаю ещё одну, связанную с автомобилями.

Допустим, вы хотите, чтобы компьютер обрабатывал входящую почту И предоставлял веб-службы. Как отличить их друг от друга на компьютере с одним IP-адресом?

У разных интернет-служб есть свои общеизвестные номера портов. Их можно посмотреть в большом списке портов IANA, а в Unix — в файле `/etc/services`. HTTP (веб) использует порт 80, telnet — 23, SMTP — 25, игра DOOM использовала 666 и так далее. Порты меньше 1024 часто считаются особыми, и для их использования обычно нужны специальные права операционной системы.

Вот и всё!

3.2. Порядок байтов

Указом королевства! Да будет два порядка байтов, впредь именуемых Паршивым и Великолепным!

Шучу, но один из них действительно лучше другого. :-)

Не знаю, как сказать это помягче, так что скажу прямо: возможно, ваш компьютер за вашей спиной хранил байты в обратном порядке. Да! Никто не хотел вам об этом говорить.

Дело в том, что в мире интернета принято считать: если нужно представить двухбайтовое шестнадцатеричное число, скажем `b34f`, его хранят в двух последовательных байтах — сначала `b3`, затем

`4f`. Логично и, как сказал бы Уилфорд Бримли, правильно. Порядок хранения, при котором старший байт идёт первым, называется Big-Endian.

К сожалению, некоторые разбросанные по миру компьютеры, а именно любые с процессорами Intel или совместимыми с Intel, хранят байты наоборот: `b34f` хранится как последовательность `4f`, затем `b3`. Этот способ называется Little-Endian.

Но подождите, с терминологией я ещё не закончил! Более разумный Big-Endian также называется сетевым порядком байтов (Network Byte Order), потому что нам, сетевикам, нравится именно такой порядок.

Ваш компьютер хранит числа в порядке байтов узла (Host Byte Order). Для Intel 80x86 это Little-Endian, для Motorola 68k — Big-Endian, а для PowerPC… смотря как!

При построении пакетов или заполнении структур данных часто нужно убедиться, что двух- и четырёхбайтовые числа записаны в сетевом порядке. Но как это сделать, если вы не знаете порядок байтов узла?

Хорошая новость! Просто исходите из того, что порядок узла неправильный, и всегда пропускайте значение через функцию преобразования в сетевой порядок. При необходимости функция волшебным образом выполнит преобразование, а код останется переносимым между машинами с разным порядком байтов.

Итак. Можно преобразовывать два типа чисел: `short` (два байта) и `long` (четыре байта). Эти функции работают и с их беззнаковыми разновидностями. Допустим, вы хотите преобразовать `short` из порядка узла в сетевой. Начните с `h` — host, добавьте `to`, затем `n` — network и `s` — short: h-to-n-s, или `htons()` («Host to Network Short»).

Почти слишком просто…

Можно использовать любые комбинации `n`, `h`, `s` и `l`, кроме совсем уж глупых. Например, функции `stolh()` («Short to Long Host») НЕ существует — во всяком случае, не на этой вечеринке. Зато есть:

Функция	Назначение
`htons()`	short: из порядка узла в сетевой
`htonl()`	long: из порядка узла в сетевой
`ntohs()`	short: из сетевого порядка в порядок узла
`ntohl()`	long: из сетевого порядка в порядок узла

В сущности, перед передачей по сети числа следует преобразовать в сетевой порядок байтов, а после приёма — в порядок узла.

В API сокетов нет стандартных 64-битных вариантов, но другие возможности обсуждаются на справочной странице `htons()`. Если нужны числа с плавающей точкой, см. раздел о сериализации ниже.

Считайте, что числа в этом документе записаны в порядке байтов узла, если не указано иное.

3.3. Структуры

Ну вот мы наконец и добрались. Пора поговорить о программировании. В этом разделе я рассмотрю различные типы данных интерфейса сокетов: некоторые из них действительно непросто понять.

Сначала простое: дескриптор сокета имеет тип

```
int
```

Обычный `int`.

Дальше всё становится страннее, так что просто читайте и потерпите.

Моя первая структура™ — `struct addrinfo`. Она появилась сравнительно недавно и используется для подготовки структур адресов сокетов к дальнейшей работе. Её также используют при поиске имён узлов и служб. Когда мы перейдём к практике, станет понятнее; пока достаточно знать, что это одна из первых вещей, с которыми вы столкнётесь при создании соединения.

```
struct addrinfo {
    int ai_flags;             // AI_PASSIVE, AI_CANONNAME и т. д.
    int ai_family;            // AF_INET, AF_INET6, AF_UNSPEC
    int ai_socktype;          // SOCK_STREAM, SOCK_DGRAM
    int ai_protocol;          // 0 означает «любой»
    size_t ai_addrlen;        // размер ai_addr в байтах
    struct sockaddr *ai_addr; // struct sockaddr_in или _in6
    char *ai_canonname;       // полное каноническое имя узла
    struct addrinfo *ai_next; // следующий узел связанного списка
};
```

Вы немного заполните эту структуру, а затем вызовете `getaddrinfo()`. Функция вернёт указатель на новый связанный список таких структур, заполненных всеми необходимыми данными.

Поле `ai_family` позволяет принудительно выбрать IPv4 или IPv6 либо оставить `AF_UNSPEC`, чтобы использовать любой вариант. Это удобно: код может не зависеть от версии IP.

Обратите внимание: это связанный список, и `ai_next` указывает на следующий элемент. Результатов может быть несколько, и вам предстоит выбрать. Я бы использовал первый работающий вариант, но у вас могут быть другие задачи; я ведь не всё знаю!

Поле `ai_addr` структуры `addrinfo` — указатель на `struct sockaddr`. Здесь мы начинаем разбираться в деталях содержимого структуры IP-адреса.

Обычно вам не нужно самостоятельно записывать данные в эти структуры: часто достаточно вызвать `getaddrinfo()`, чтобы заполнить `struct addrinfo`. Но вам придётся заглянуть внутрь структур, чтобы извлечь значения, поэтому я их здесь покажу.

(Кроме того, во всём коде, написанном до изобретения `struct addrinfo`, данные заполнялись вручную. Поэтому вы встретите немало кода IPv4, который делает именно так — например, в старых версиях этого руководства.)

Некоторые структуры относятся к IPv4, некоторые к IPv6, а некоторые подходят для обоих. Я отмечу, какая к чему относится.

Итак, `struct sockaddr` содержит информацию об адресе для многих типов сокетов:

```
struct sockaddr {
    unsigned short sa_family; // семейство адресов, AF_xxx
    char sa_data[14];         // 14 байтов адреса протокола
};
```

`sa_family` может принимать разные значения, но во всём, что мы делаем в этом документе, это будет `AF_INET` (IPv4) или `AF_INET6` (IPv6). `sa_data` содержит адрес назначения и номер порта сокета. Это не очень удобно: вам вряд ли захочется утомительно упаковывать адрес в `sa_data` вручную.

Для работы с `struct sockaddr` программисты создали параллельную структуру `struct sockaddr_in` («in» означает Internet), предназначенную для IPv4.

И вот что важно: указатель на `struct sockaddr_in` можно привести к указателю на `struct sockaddr`, и наоборот. Поэтому, хотя `connect()` ожидает `struct sockaddr*`, можно использовать `struct sockaddr_in` и привести указатель непосредственно перед вызовом!

```
// Только IPv4; для IPv6 см. struct sockaddr_in6
struct sockaddr_in {
    short int sin_family;        // семейство адресов, AF_INET
    unsigned short int sin_port; // номер порта
    struct in_addr sin_addr;     // интернет-адрес
    unsigned char sin_zero[8];   // размер как у struct sockaddr
};
```

Эта структура позволяет легко обращаться к элементам адреса сокета. Заметьте: `sin_zero` добавлен для доведения размера структуры до размера `struct sockaddr` и должен быть полностью заполнен нулями с помощью `memset()`. Кроме того, `sin_family` соответствует `sa_family` в `struct sockaddr` и должен иметь значение `AF_INET`. Наконец, `sin_port` должен быть записан в сетевом порядке байтов (используйте `htons()`!).

Копнём глубже! Поле `sin_addr` имеет тип `struct in_addr`. Что это такое? Не желая чрезмерно драматизировать, скажу: это одно из самых страшных объединений всех времён:

```
// Только IPv4; для IPv6 см. struct in6_addr
// Интернет-адрес (структура по историческим причинам)
struct in_addr {
    uint32_t s_addr; // 32-битное целое (4 байта)
};
```

Ух ты! Раньше это действительно было объединение, но те времена, похоже, прошли. И хорошо. Если переменная `ina` объявлена как `struct sockaddr_in`, то `ina.sin_addr.s_addr` обозначает четырёхбайтовый IP-адрес в сетевом порядке байтов. Даже если в вашей системе для `struct in_addr` всё ещё используется это ужасное объединение, к четырёхбайтовому IP-адресу можно обращаться точно так же, как показано выше (благодаря `#define`).

А что насчёт IPv6? Для него существуют похожие структуры:

```
// Только IPv6; для IPv4 см. struct sockaddr_in и struct in_addr
struct sockaddr_in6 {
    u_int16_t sin6_family;   // семейство адресов, AF_INET6
    u_int16_t sin6_port;     // порт, сетевой порядок байтов
    u_int32_t sin6_flowinfo; // информация о потоке IPv6
    struct in6_addr sin6_addr; // адрес IPv6
    u_int32_t sin6_scope_id; // идентификатор области действия
};

struct in6_addr {
    unsigned char s6_addr[16]; // адрес IPv6
};
```

Обратите внимание, что IPv6 имеет адрес IPv6 и номер порта — точно так же, как IPv4 имеет адрес IPv4 и номер порта.

Также заметьте, что пока я не буду рассказывать о полях информации о потоке IPv6 и идентификатора области действия… Это ведь руководство для начинающих. :-)

И последнее, но не менее важное: ещё одна простая структура, `struct sockaddr_storage`, достаточно велика, чтобы вместить структуры и IPv4, и IPv6. Дело в том, что при некоторых вызовах вы заранее не знаете, заполнит ли функция вашу `struct sockaddr` адресом IPv4 или IPv6. Поэтому передайте эту параллельную структуру, очень похожую на `struct sockaddr`, но большую по размеру, а затем приведите её к нужному типу:

```
struct sockaddr_storage {
    sa_family_t ss_family; // семейство адресов
    // остальное – заполнение, зависит от реализации; игнорируйте:
    char __ss_pad1[_SS_PAD1SIZE];
    int64_t __ss_align;
    char __ss_pad2[_SS_PAD2SIZE];
};
```

Важно, что семейство адресов доступно в поле `ss_family`: проверьте, равно ли оно `AF_INET` или `AF_INET6` (для IPv4 или IPv6). После этого при желании можно привести структуру к `struct sockaddr_in` или `struct sockaddr_in6`.

3.4. IP-адреса, часть вторая

К счастью, существует множество функций для работы с IP-адресами. Не нужно разбирать их вручную и запихивать в `long` с помощью оператора `<<`.

Допустим, у вас есть `struct sockaddr_in ina` и IP-адрес `10.12.110.57` или `2001:db8:63b3:1::3490`, который вы хотите сохранить. Функция `inet_pton()` преобразует IP-адрес из числовой записи с точками в `struct in_addr` или `struct in6_addr` в зависимости от того, указали ли вы `AF_INET` или `AF_INET6`. (`pton` означает presentation to network — «из представления в сетевой формат»; если легче запомнить, можете считать это printable to network — «из печатного вида в сетевой».) Для IPv4 и IPv6 преобразование выполняется так:

```
struct sockaddr_in sa;   // IPv4
struct sockaddr_in6 sa6; // IPv6
inet_pton(AF_INET, "10.12.110.57", &(sa.sin_addr));
inet_pton(AF_INET6, "2001:db8:63b3:1::3490", &(sa6.sin6_addr));
```

Краткое замечание: раньше для этого использовали функции `inet_addr()` или `inet_aton()`. Теперь они устарели и не работают с IPv6.

Приведённый фрагмент не очень надёжен: в нём нет проверки ошибок. `inet_pton()` возвращает -1 при ошибке или 0, если адрес некорректен. Поэтому перед использованием результата убедитесь, что возвращённое значение больше 0!

Итак, теперь вы умеете преобразовывать строковые IP-адреса в двоичное представление. А как быть с обратным преобразованием? Допустим, у вас есть `struct in_addr`, и вы хотите вывести её в виде чисел и точек. Или `struct in6_addr`, которую нужно вывести в виде, хм, «шестнадцатеричных чисел и

двоеточий». В этом случае используйте `inet_ntop()` (`ntop` означает network to presentation — «из сетевого формата в представление»; если проще, запомните как network to printable):

```
// IPv4:
char ip4[INET_ADDRSTRLEN]; // место для строки IPv4
struct sockaddr_in sa;    // предположим, она чем-то заполнена
inet_ntop(AF_INET, &(sa.sin_addr), ip4, INET_ADDRSTRLEN);
printf("The IPv4 address is: %s\n", ip4);

// IPv6:
char ip6[INET6_ADDRSTRLEN]; // место для строки IPv6
struct sockaddr_in6 sa6;    // предположим, она чем-то заполнена
inet_ntop(AF_INET6, &(sa6.sin6_addr), ip6, INET6_ADDRSTRLEN);
printf("The address is: %s\n", ip6);
```

При вызове передаются тип адреса (IPv4 или IPv6), адрес, указатель на строку для результата и максимальная длина этой строки. (Два удобных макроса задают размер строки, достаточный для самого длинного адреса IPv4 или IPv6: `INET_ADDRSTRLEN` и `INET6_ADDRSTRLEN`.)

Ещё одно краткое замечание о старом способе: исторически для этого преобразования использовали `inet_ntoa()`. Она тоже устарела и не работает с IPv6.

Наконец, эти функции работают только с числовыми IP-адресами: они не выполняют DNS-поиск имени узла, например `www.example.com`. Для этого, как мы увидим позже, используется `getaddrinfo()`.

3.4.1. Частные или изолированные сети

Во многих местах есть межсетевой экран, скрывающий сеть от остального мира для её защиты. Часто экран преобразует «внутренние» IP-адреса во «внешние» (известные всему остальному миру) с помощью механизма Network Address Translation, или NAT.

Уже нервничаете? «К чему он ведёт со всеми этими странностями?»

Расслабьтесь и купите себе безалкогольный (или алкогольный) напиток: новичку о NAT даже не нужно беспокоиться, поскольку преобразование происходит прозрачно. Но я хотел рассказать о сети за межсетевым экраном на случай, если увиденные номера сетей вас запутают.

Например, у меня дома есть межсетевой экран. Компания DSL выделила мне два статических адреса IPv4, но в моей сети семь компьютеров. Как это возможно? Два компьютера не могут иметь один IP-адрес, иначе данные не будут знать, куда идти!

Ответ: у них разные IP-адреса. Они находятся в частной сети, которой выделено 24 миллиона IP-адресов. Все они только мои. Ну, насколько это касается всех остальных.

Вот что происходит. Когда я вхожу на удалённый компьютер, он сообщает, что вход выполнен с `192.0.2.33` — публичного IP-адреса, предоставленного провайдером. Но если спросить мой локальный компьютер о его IP-адресе, он ответит `10.0.0.5`. Кто же преобразует один IP-адрес в другой? Правильно, межсетевой экран! Он выполняет NAT!

`10.x.x.x` — одна из нескольких зарезервированных сетей, которые разрешено использовать только в полностью изолированных сетях либо за межсетевыми экранами. Подробности о доступных номерах частных сетей приведены в RFC 1918, но часто встречаются `10.x.x.x` и `192.168.x.x`, где `x` обычно лежит в диапазоне 0–255. Реже встречается `172.y.x.x`, где `y` — от 16 до 31.

Сети за межсетевым экраном, выполняющим NAT, не обязаны использовать один из этих зарезервированных диапазонов, но обычно используют.

Забавный факт! Мой внешний IP-адрес на самом деле не `192.0.2.33`. Сеть `192.0.2.x` зарезервирована для вымышленных «настоящих» IP-адресов в документации — например, в этом руководстве! Вот так!

В некотором смысле частные сети есть и в IPv6. Их адреса начинаются с `fdXX:` (или, возможно, в будущем с `fcXX:`), согласно RFC 4193. Впрочем, NAT и IPv6 обычно не сочетаются (если не считать шлюз между IPv6 и IPv4, который выходит за рамки документа): теоретически в вашем распоряжении будет столько адресов, что NAT больше не понадобится. Но если вы хотите выделить себе адреса в сети, трафик которой не маршрутизируется наружу, делается это именно так.

Глава 4. Переход с IPv4 на IPv6

Но я просто хочу знать, что изменить в коде, чтобы он заработал с IPv6! Скажите сейчас же!

Хорошо, хорошо!

Почти всё это мы уже разобрали выше, но вот краткая версия для нетерпеливых. (Разумеется, это ещё не всё, но для нашего руководства достаточно.)

1. Прежде всего постарайтесь получать все данные `struct sockaddr` с помощью `getaddrinfo()`, а не заполнять структуры вручную. Так код не будет зависеть от версии IP, и многие из последующих шагов не понадобятся.
2. Везде, где версия IP жёстко задана в коде, постарайтесь перенести соответствующие действия во вспомогательную функцию.
3. Замените `AF_INET` на `AF_INET6`.
4. Замените `PF_INET` на `PF_INET6`.
5. Замените присваивания `INADDR_ANY` на присваивания `in6addr_any`, которые немного отличаются:

```
struct sockaddr_in sa;
struct sockaddr_in6 sa6;
sa.sin_addr.s_addr = INADDR_ANY; // использовать мой адрес IPv4
sa6.sin6_addr = in6addr_any;    // использовать мой адрес IPv6
```

Также при объявлении структуры `in6_addr` можно использовать инициализатор `IN6ADDR_ANY_INIT`:

```
struct in6_addr ia6 = IN6ADDR_ANY_INIT;
```

6. Вместо `struct sockaddr_in` используйте `struct sockaddr_in6`, не забывая добавлять «6» к соответствующим полям (см. раздел о структурах выше). Поля `sin6_zero` не существует.
7. Вместо `struct in_addr` используйте `struct in6_addr`, не забывая добавлять «6» к соответствующим полям (см. раздел о структурах выше).
8. Вместо `inet_aton()` или `inet_addr()` используйте `inet_pton()`.
9. Вместо `inet_ntoa()` используйте `inet_ntop()`.
10. Вместо `gethostbyname()` используйте более совершенную `getaddrinfo()`.
11. Вместо `gethostbyaddr()` используйте более совершенную `getnameinfo()` (хотя `gethostbyaddr()` всё ещё может работать с IPv6).
12. `INADDR_BROADCAST` больше не работает. Вместо него используйте многоадресную рассылку IPv6.

Et voilà!

5. Системные вызовы или ничего

В этом разделе мы рассмотрим системные и другие библиотечные вызовы, которые открывают доступ к сетевым возможностям Unix и вообще любой системы с API сокетов: BSD, Windows, Linux, Mac и других. При вызове одной из этих функций управление принимает ядро и волшебным образом выполняет за вас всю работу.

Чаще всего здесь возникает вопрос: в каком порядке вызывать эти функции? Как вы, наверное, уже заметили, справочные страницы man в этом не помогают. Чтобы исправить положение, в следующих разделах я расположил вызовы примерно в том порядке, в каком они понадобятся вашим программам.

Добавьте к этому несколько примеров кода, молоко с печеньем (их, боюсь, придётся добыть самостоятельно), немного мужества — и вы будете пересылать данные через Интернет, как настоящий наследник Джона Постела!

Обратите внимание: ради краткости многие приведённые ниже фрагменты не проверяют ошибки. Также они часто предполагают, что вызов `getaddrinfo()` прошёл успешно и вернул корректный элемент связного списка. В самостоятельных примерах программ обе эти ситуации обрабатываются должным образом, поэтому берите за образец именно их.

5.1. `getaddrinfo()` — готовимся к запуску!

Эта функция — настоящая рабочая лошадка со множеством параметров, хотя пользоваться ею довольно просто. Она помогает подготовить структуры, которые понадобятся позже.

Немного истории: раньше для DNS-запросов применяли функцию `gethostbyname()`. Затем полученную информацию вручную помещали в `struct sockaddr_in` и использовали эту структуру в вызовах.

К счастью, теперь это необязательно. Более того, если вы хотите написать код, работающий и с IPv4, и с IPv6, так делать нежелательно! Сегодня у нас есть функция `getaddrinfo()`: она выполняет множество полезных действий, в том числе ищет имена хостов через DNS и разрешает имена служб, а заодно заполняет необходимые структуры.

Посмотрим на неё:

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>

int getaddrinfo(const char *node,    // например, "www.example.com" или IP
                const char *service, // например, "http" или номер порта
                const struct addrinfo *hints,
                struct addrinfo **res);
```

Вы передаёте функции три входных параметра, а она возвращает через `res` указатель на связный список результатов. Параметр `node` — имя хоста, к которому нужно подключиться, или IP-адрес.

Параметр `service` может содержать номер порта, например `"80"`, или название конкретной службы — например, `"http"`, `"ftp"`, `"telnet"` или `"smtp"`. Их можно найти в списке портов IANA или в файле `/etc/services` вашей Unix-системы.

Наконец, `hints` указывает на структуру `addrinfo`, которую вы уже заполнили подходящими значениями.

Вот пример вызова для сервера, который собирается слушать порт 3490 на IP-адресе своей машины. Обратите внимание: этот код ещё не начинает прослушивание и не настраивает сеть, а лишь подготавливает структуры для дальнейшей работы:

```
int status;
struct addrinfo hints;
struct addrinfo *servinfo; // будет указывать на результаты

memset(&hints, 0, sizeof hints); // убедимся, что структура пуста
hints.ai_family = AF_UNSPEC;     // IPv4 или IPv6 — без разницы
hints.ai_socktype = SOCK_STREAM; // потоковые TCP-сокеты
hints.ai_flags = AI_PASSIVE;     // подставить мой IP-адрес

if ((status = getaddrinfo(NULL, "3490", &hints, &servinfo)) != 0) {
    fprintf(stderr, "gai error: %s\n", gai_strerror(status));
    exit(1);
}

// servinfo теперь указывает на связный список из одной или более
// структур addrinfo

// ... выполняем действия, пока servinfo больше не понадобится ...

freeaddrinfo(servinfo); // освобождаем связный список
```

Обратите внимание: я установил `ai_family` в `AF_UNSPEC`, показав тем самым, что мне всё равно, будет использоваться IPv4 или IPv6. Если вам нужен строго один из вариантов, можно указать `AF_INET` или `AF_INET6`.

Здесь также присутствует флаг `AI_PASSIVE`: он предписывает `getaddrinfo()` назначить структурам сокетов адрес локального узла. Это удобно, поскольку вам не приходится прописывать адрес в коде. (Можно также передать конкретный адрес первым аргументом `getaddrinfo()` вместо указанного выше `NULL`.)

Затем мы вызываем функцию. Если произошла ошибка (`getaddrinfo()` вернула ненулевое значение), её можно вывести с помощью `gai_strerror()`, как показано в примере. При успехе `servinfo` будет указывать на связный список структур `addrinfo`, каждая из которых содержит структуру `sockaddr` определённого вида, которую мы сможем использовать позже. Удобно!

Когда мы закончим работу со списком, любезно выделенным для нас функцией `getaddrinfo()`, его можно и нужно освободить вызовом `freeaddrinfo()`.

Вот пример вызова для клиента, который хочет подключиться к конкретному серверу, скажем `www.example.net`, на порт 3490. И снова: подключение пока не выполняется, мы только готовим структуры для дальнейшей работы:

```
int status;
struct addrinfo hints;
struct addrinfo *servinfo; // будет указывать на результаты

memset(&hints, 0, sizeof hints); // убедимся, что структура пуста
hints.ai_family = AF_UNSPEC;     // IPv4 или IPv6 — без разницы
hints.ai_socktype = SOCK_STREAM; // потоковые TCP-сокеты

// готовимся к подключению
status = getaddrinfo("www.example.net", "3490", &hints, &servinfo);
```

```
// servinfo теперь указывает на связный список из одной или более
// структур addrinfo

// и так далее
```

Я всё повторяю, что `servinfo` — это связный список со всевозможными сведениями об адресах. Напишем небольшую демонстрационную программу, чтобы увидеть эти сведения. Эта короткая программа выводит IP-адреса хоста, заданного в командной строке:

```
/*
** showip.c
**
** показать IP-адреса хоста, заданного в командной строке
*/

#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
#include <arpa/inet.h>
#include <netinet/in.h>

int main(int argc, char *argv[])
{
    struct addrinfo hints, *res, *p;
    int status;
    char ipstr[INET6_ADDRSTRLEN];

    if (argc != 2) {
        fprintf(stderr,"usage: showip hostname\n");
        return 1;
    }

    memset(&hints, 0, sizeof hints);
    hints.ai_family = AF_UNSPEC; // IPv4 или IPv6
    hints.ai_socktype = SOCK_STREAM;

    if ((status = getaddrinfo(argv[1], NULL, &hints, &res)) != 0) {
        fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(status));
        return 2;
    }

    printf("IP addresses for %s:\n\n", argv[1]);

    for (p = res; p != NULL; p = p->ai_next) {
        void *addr;
        char *ipver;
        struct sockaddr_in *ipv4;
        struct sockaddr_in6 *ipv6;

        // получаем указатель на сам адрес;
        // поля IPv4 и IPv6 различаются:
        if (p->ai_family == AF_INET) { // IPv4
            ipv4 = (struct sockaddr_in *)p->ai_addr;
            addr = &(ipv4->sin_addr);
            ipver = "IPv4";
        } else { // IPv6
            ipv6 = (struct sockaddr_in6 *)p->ai_addr;
            addr = &(ipv6->sin6_addr);
            ipver = "IPv6";
        }
```

```
        // преобразуем IP в строку и выводим:
        inet_ntop(p->ai_family, addr, ipstr, sizeof ipstr);
        printf("  %s: %s\n", ipver, ipstr);
    }

    freeaddrinfo(res); // освобождаем связный список

    return 0;
}
```

Как видите, код вызывает `getaddrinfo()` с тем, что вы передали в командной строке. Эта функция заполняет связный список, на который указывает `res`; затем мы можем пройти по списку и вывести данные или сделать с ними что-нибудь ещё.

(Здесь есть небольшая некрасивость: приходится обращаться к разным видам структур `sockaddr` в зависимости от версии IP. Простите! Я не знаю, как лучше это обойти.)

Пример запуска! Все любят снимки экрана:

```
$ showip www.example.net
IP addresses for www.example.net:
  IPv4: 192.0.2.88
$ showip ipv6.example.com
IP addresses for ipv6.example.com:
  IPv4: 192.0.2.101
  IPv6: 2001:db8:8c00:22::171
```

Теперь, когда мы с этим разобрались, будем передавать результаты `getaddrinfo()` другим функциям сокетов и наконец установим сетевое соединение! Читайте дальше!

5.2. `socket()` — получаем дескриптор файла!

Похоже, больше откладывать нельзя: пора поговорить о системном вызове `socket()`. Вот его описание:

```
#include <sys/types.h>
#include <sys/socket.h>

int socket(int domain, int type, int protocol);
```

Что означают аргументы? Они позволяют указать, какой сокет вам нужен: IPv4 или IPv6, потоковый или дейтаграммный, TCP или UDP.

Раньше эти значения обычно прописывали в коде, и вы по-прежнему можете так сделать: `domain` принимает `PF_INET` или `PF_INET6`, `type` — `SOCK_STREAM` или `SOCK_DGRAM`, а `protocol` можно установить в 0, чтобы выбрать протокол, подходящий для указанного типа. Или же можно найти нужный протокол, `"tcp"` или `"udp"`, вызвав `getprotobyname()`.

`PF_INET` — близкий родственник `AF_INET`, применяемого при инициализации поля `sin_family` структуры `sockaddr_in`. Они настолько близки, что их значения одинаковы, поэтому многие программисты передают в `socket()` первым аргументом `AF_INET` вместо `PF_INET`. А теперь возьмите молока с печеньем: пришло время истории. Давным-давно считалось, что семейство адресов (address family, отсюда AF) может поддерживать несколько протоколов, относящихся к семейству протоколов (protocol family, отсюда PF). Этого не произошло. И жили они долго и счастливо. Конец. Поэтому формально правильнее использовать `AF_INET` в `sockaddr_in`, а `PF_INET` — при вызове `socket()`.

Хватит об этом. На практике лучше передавать в `socket()` значения, полученные от `getaddrinfo()`, прямо так:

```
int s;
struct addrinfo hints, *res;

// ищем адрес
// [представим, что мы уже заполнили структуру hints]
getaddrinfo("www.example.com", "http", &hints, &res);

// Ещё раз: следует проверять ошибки getaddrinfo() и перебирать
// связный список res в поисках подходящих элементов, а не предполагать,
// что первый элемент годится (как делают многие примеры здесь).
// Настоящие примеры см. в разделе о клиенте и сервере.

s = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
```

`socket()` возвращает дескриптор сокета, который можно использовать в последующих системных вызовах, или `-1` при ошибке. Глобальной переменной `errno` присваивается значение ошибки; подробности, в том числе краткое замечание об использовании `errno` в многопоточных программах, см. на соответствующей странице man.

Хорошо, хорошо, но какой толк от этого сокета? Сам по себе он практически бесполезен. Чтобы он обрёл смысл, читайте дальше и вызывайте другие системные функции.

5.3. `bind()` — какой у меня порт?

Получив сокет, вы, возможно, захотите связать его с портом локальной машины. Обычно это делают, когда собираются с помощью `listen()` ожидать входящие соединения на определённом порту. Так поступают сетевые игры, когда предлагают «подключиться к 192.168.5.10, порт 3490». Номер порта позволяет ядру сопоставить входящий пакет с дескриптором сокета определённого процесса. Если вы собираетесь только вызывать `connect()` — то есть выступаете клиентом, а не сервером, — это, вероятно, не нужно. Но всё равно прочитайте, просто ради интереса.

Прототип системного вызова `bind()`:

```
#include <sys/types.h>
#include <sys/socket.h>

int bind(int sockfd, struct sockaddr *my_addr, int addrlen);
```

`sockfd` — дескриптор сокета, возвращённый `socket()`. `my_addr` указывает на `struct sockaddr`, содержащую сведения о вашем адресе, а именно IP-адрес и порт. `addrlen` — длина этого адреса в байтах.

Уф! Немало информации за раз. Посмотрим пример, который привязывает сокет к узлу, на котором запущена программа, и порту 3490:

```
struct addrinfo hints, *res;
int sockfd;

// сначала заполняем структуры адреса с помощью getaddrinfo():
memset(&hints, 0, sizeof hints);
hints.ai_family = AF_UNSPEC;     // IPv4 или IPv6, без разницы
hints.ai_socktype = SOCK_STREAM;
hints.ai_flags = AI_PASSIVE;     // подставить мой IP-адрес
```

```
getaddrinfo(NULL, "3490", &hints, &res);

// создаём сокет:
sockfd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);

// привязываем его к порту, переданному getaddrinfo():
bind(sockfd, res->ai_addr, res->ai_addrlen);
```

Флагом `AI_PASSIVE` я указываю привязаться к IP-адресу узла, на котором выполняется программа. Если нужен конкретный локальный IP-адрес, уберите `AI_PASSIVE` и передайте адрес первым аргументом `getaddrinfo()`.

При ошибке `bind()` также возвращает `-1` и записывает код ошибки в `errno`.

Во множестве старых программ структура `sockaddr_in` заполняется вручную перед вызовом `bind()`. Очевидно, это относится к IPv4, но ничто не мешает сделать то же самое для IPv6, хотя `getaddrinfo()` обычно удобнее. Старый код выглядит примерно так:

```
// !!! ЭТО СТАРЫЙ СПОСОБ !!!
int sockfd;
struct sockaddr_in my_addr;

sockfd = socket(PF_INET, SOCK_STREAM, 0);
my_addr.sin_family = AF_INET;
my_addr.sin_port = htons(MYPORT); // short, сетевой порядок байтов
my_addr.sin_addr.s_addr = inet_addr("10.12.110.57");
memset(my_addr.sin_zero, '\0', sizeof my_addr.sin_zero);

bind(sockfd, (struct sockaddr *)&my_addr, sizeof my_addr);
```

В этом коде можно также присвоить полю `s_addr` значение `INADDR_ANY`, чтобы привязаться к локальному IP-адресу, как при использовании `AI_PASSIVE` выше. Аналог `INADDR_ANY` для IPv6 — глобальная переменная `in6addr_any`, которую присваивают полю `sin6_addr` структуры `sockaddr_in6`. (Есть и макрос `IN6ADDR_ANY_INIT` для использования при инициализации переменной.)

Выбирая номер порта для `bind()`, не берите слишком маленький: все порты ниже 1024 зарезервированы, если только вы не суперпользователь! Можно выбрать любой порт выше, вплоть до 65535, при условии что он ещё не занят другой программой.

Иногда при повторном запуске сервера `bind()` завершается ошибкой «Address already in use» — «адрес уже используется». Что это значит? Остатки прежнего соединённого сокета всё ещё находятся в ядре и удерживают порт. Можно подождать, пока они исчезнут (около минуты), или разрешить программе повторно использовать порт:

```
int yes = 1;
// char yes = '1'; // для Solaris

// избавляемся от назойливой ошибки «Address already in use»
setsockopt(listener, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);
```

И ещё одно небольшое замечание о `bind()`: иногда вызывать её необязательно. Если вы подключаетесь к удалённой машине через `connect()` и вам всё равно, какой локальный порт использовать (как в telnet, где важен только удалённый), просто вызовите `connect()`. Она проверит, привязан ли сокет, и при необходимости привяжет его к свободному локальному порту.

5.4. `connect()` — эй, ты!

Представим на минуту, что вы — приложение telnet. Пользователь приказывает вам, как в фильме «Трон», получить дескриптор сокета. Вы подчиняетесь и вызываете `socket()`. Затем пользователь велит подключиться к `10.12.110.57` через порт `23` — стандартный порт telnet. Ого! Что теперь делать?

К счастью для вас, программа, вы как раз читаете раздел о том, как подключиться к удалённому узлу с помощью `connect()`. Так что читайте скорее: терять время нельзя!

Вызов `connect()` выглядит так:

```
#include <sys/types.h>
#include <sys/socket.h>

int connect(int sockfd, struct sockaddr *serv_addr, int addrlen);
```

`sockfd` — наш старый знакомый, дескриптор сокета, возвращённый вызовом `socket()`. `serv_addr` — структура `sockaddr`, содержащая IP-адрес и порт назначения, а `addrlen` — длина структуры адреса сервера в байтах.

Всю эту информацию можно получить из результата вызова `getaddrinfo()`. Здорово!

Начинает проясняться? Я вас отсюда не слышу, так что остаётся надеяться. Рассмотрим пример подключения сокета к `www.example.com`, порт `3490`:

```
struct addrinfo hints, *res;
int sockfd;

// сначала заполняем структуры адреса с помощью getaddrinfo():
memset(&hints, 0, sizeof hints);
hints.ai_family = AF_UNSPEC;
hints.ai_socktype = SOCK_STREAM;
getaddrinfo("www.example.com", "3490", &hints, &res);

// создаём сокет:
sockfd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);

// подключаемся!
connect(sockfd, res->ai_addr, res->ai_addrlen);
```

И снова: старые программы сами заполняли структуры `sockaddr_in` для передачи в `connect()`. При желании вы можете поступить так же. См. аналогичное замечание в разделе `bind()` выше.

Обязательно проверяйте результат `connect()`: при ошибке функция возвращает `-1` и устанавливает `errno`.

Заметьте также, что мы не вызывали `bind()`. По существу, номер локального порта нам безразличен; важно только, куда мы подключаемся, то есть удалённый порт. Ядро само выберет локальный порт, а узел, к которому мы подключимся, автоматически получит эти сведения. Не беспокойтесь.

5.5. `listen()` — кто-нибудь мне позвонит?

Пора сменить направление. Что, если вы не хотите подключаться к удалённому узлу, а, скажем, хотите ждать входящие соединения и как-то их обрабатывать? Этот процесс состоит из двух шагов: сначала вызывается `listen()`, затем `accept()` (см. ниже).

Вызов `listen()` довольно прост, но требует пояснения:

```
int listen(int sockfd, int backlog);
```

`sockfd` — обычный дескриптор сокета, возвращённый системным вызовом `socket()`. `backlog` — число соединений, допустимое в очереди входящих соединений. Что это значит? Входящие соединения ожидают в очереди, пока вы не примете их с помощью `accept()` (см. ниже); этот параметр ограничивает их количество. Большинство систем молча ограничивают число примерно двадцатью; вероятно, вам подойдёт значение 5 или 10.

Как обычно, при ошибке `listen()` возвращает `-1` и устанавливает `errno`.

Как вы, наверное, догадываетесь, перед `listen()` нужно вызвать `bind()`, чтобы сервер работал на конкретном порту. Ведь вы должны иметь возможность сообщить друзьям, к какому порту подключаться! Итак, для ожидания входящих соединений последовательность системных вызовов будет такой:

```
getaddrinfo();
socket();
bind();
listen();
/* здесь будет accept() */
```

Оставлю это вместо примера кода: всё достаточно понятно. В разделе `accept()` ниже пример полнее. Самая сложная часть всей этой затеи — вызов `accept()`.

5.6. `accept()` — «Спасибо, что позвонили на порт 3490»

Приготовьтесь: вызов `accept()` немного странный! Произойдёт вот что: кто-то очень далеко попытается подключиться к вашей машине через `connect()` на порт, который вы слушаете с помощью `listen()`. Его соединение попадёт в очередь и будет ждать вызова `accept()`. Вы вызываете `accept()` и просите её взять ожидающее соединение. Она возвращает совершенно новый дескриптор сокета для этого конкретного соединения! Верно: внезапно у вас два дескриптора сокета по цене одного! Первоначальный продолжает слушать новые соединения, а только что созданный наконец готов к `send()` и `recv()`. Мы у цели!

Вот прототип вызова:

```
#include <sys/types.h>
#include <sys/socket.h>

int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```

`sockfd` — дескриптор прослушивающего сокета. Достаточно просто. `addr` обычно указывает на локальную структуру `sockaddr_storage`. В неё будут помещены сведения о входящем соединении, по которым можно определить, какой узел и с какого порта подключился. `addrlen` — локальная целочисленная переменная, которую перед передачей её адреса в `accept()` следует установить равной `sizeof(struct sockaddr_storage)`. Функция запишет в `addr` не больше указанного числа байтов. Если запишет меньше, она изменит значение `addrlen` соответственно.

Догадайтесь! При ошибке `accept()` возвращает `-1` и устанавливает `errno`. Уверен, вы не ожидали.

Как и прежде, информации много, поэтому вот пример фрагмента кода:

```
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>

#define MYPORT "3490" // порт, к которому будут подключаться пользователи
#define BACKLOG 10    // сколько ожидающих соединений вмещает очередь

int main(void)
{
    struct sockaddr_storage their_addr;
    socklen_t addr_size;
    struct addrinfo hints, *res;
    int sockfd, new_fd;

    // !! не забудьте проверять ошибки этих вызовов !!

    // сначала заполняем структуры адресов с помощью getaddrinfo():
    memset(&hints, 0, sizeof hints);
    hints.ai_family = AF_UNSPEC;     // IPv4 или IPv6
    hints.ai_socktype = SOCK_STREAM;
    hints.ai_flags = AI_PASSIVE;     // подставить мой IP-адрес
    getaddrinfo(NULL, MYPORT, &hints, &res);

    // создаём сокет, привязываем его и начинаем прослушивание:
    sockfd = socket(res->ai_family, res->ai_socktype,
                    res->ai_protocol);
    bind(sockfd, res->ai_addr, res->ai_addrlen);
    listen(sockfd, BACKLOG);

    // принимаем входящее соединение:
    addr_size = sizeof their_addr;
    new_fd = accept(sockfd, (struct sockaddr *)&their_addr,
                    &addr_size);

    // можно общаться через дескриптор new_fd!
    ...
```

Обратите внимание: для всех вызовов `send()` и `recv()` мы будем использовать дескриптор `new_fd`. Если вам нужно только одно соединение, можете при желании закрыть прослушивающий `sockfd` с помощью `close()`, чтобы предотвратить новые входящие соединения на этом порту.

5.7. `send()` и `recv()` — поговори со мной!

Эти две функции служат для обмена данными через потоковые сокеты или подключённые дейтаграммные сокеты. Если вы хотите использовать обычные неподключённые дейтаграммные сокеты, обратитесь к разделу о `sendto()` и `recvfrom()` ниже.

Возможно, это для вас новость: эти вызовы являются блокирующими. То есть `recv()` будет ждать, пока не появятся данные для приёма. «Но что значит “блокировать”?» Это значит, что программа остановится на этом системном вызове, пока кто-нибудь что-нибудь не отправит. (В техническом жаргоне про ОС вместо «остановится» на самом деле говорят «уснёт», поэтому я могу употреблять эти слова как взаимозаменяемые.) `send()` также может блокироваться, если отправляемые данные где-то застряли, но это бывает реже. Мы ещё вернёмся к этому вопросу и поговорим, как при необходимости этого избежать.

Вот вызов `send()`:

```
int send(int sockfd, const void *msg, int len, int flags);
```

`sockfd` — дескриптор сокета, в который нужно отправить данные: полученный либо от `socket()`, либо от `accept()`. `msg` указывает на отправляемые данные, а `len` — их длина в байтах. Просто установите `flags` в 0. Подробнее о флагах см. страницу man для `send()`.

Пример кода:

```
char *msg = "Beej was here!";
int len, bytes_sent;
...
len = strlen(msg);
bytes_sent = send(sockfd, msg, len, 0);
...
```

`send()` возвращает число фактически отправленных байтов. Оно может быть меньше числа байтов, которые вы просили отправить! Иногда вы поручаете функции отправить кучу данных, а она не справляется. Она отправит столько, сколько сможет, и предоставит вам отправить остальное позже. Помните: если возвращённое `send()` значение не совпадает с `len`, вы сами отвечаете за отправку остатка строки. Хорошая новость: если пакет небольшой, меньше примерно 1 КБ, функция, вероятно, сумеет отправить его целиком за один раз. При ошибке снова возвращается `-1`, а `errno` содержит её код.

Вызов `recv()` во многом похож:

```
int recv(int sockfd, void *buf, int len, int flags);
```

`sockfd` — дескриптор сокета, из которого нужно читать, `buf` — буфер для полученных данных, `len` — максимальная длина буфера, а `flags` снова можно установить в 0. Сведения о флагах см. на странице man для `recv()`.

`recv()` возвращает число байтов, фактически записанных в буфер, или `-1` при ошибке; при этом устанавливается `errno`.

Погодите! `recv()` может вернуть 0. Это может означать только одно: удалённая сторона закрыла соединение! Так `recv()` сообщает вам об этом.

Вот и всё, просто, правда? Теперь вы можете обмениваться данными через потоковые сокеты! Ура! Вы — программист сетевых приложений Unix!

5.8. `sendto()` и `recvfrom()` — разговор на языке DGRAM

«Всё это замечательно, — слышу я ваш голос, — но как быть с неподключёнными дейтаграммными сокетами?» Нет проблем, amigo. У нас есть именно то, что нужно.

Поскольку дейтаграммные сокеты не подключены к удалённому узлу, догадаетесь, какую информацию нужно передать перед отправкой пакета? Верно: адрес назначения! Вот как это работает:

```
int sendto(int sockfd, const void *msg, int len, unsigned int flags,
           const struct sockaddr *to, socklen_t tolen);
```

Как видите, вызов практически совпадает с `send()`, но добавляются ещё два параметра. `to` — указатель на `struct sockaddr` (скорее всего, это будет `struct sockaddr_in`, `struct sockaddr_in6` или `struct sockaddr_storage`, приведённая к нужному типу в последний момент), содержащую IP-адрес и порт назначения. `tolen`, по сути являющийся `int`, можно просто установить в `sizeof *to` или `sizeof(struct sockaddr_storage)`.

Структуру адреса назначения вы, вероятно, получите от `getaddrinfo()` или `recvfrom()`, о которой речь ниже, либо заполните вручную.

Как и `send()`, функция `sendto()` возвращает число фактически отправленных байтов (которое опять-таки может быть меньше запрошенного) или `-1` при ошибке.

Столь же похожи `recv()` и `recvfrom()`. Прототип `recvfrom()`:

```
int recvfrom(int sockfd, void *buf, int len, unsigned int flags,
             struct sockaddr *from, int *fromlen);
```

И снова это почти `recv()`, но с парой дополнительных полей. `from` указывает на локальную структуру `sockaddr_storage`, в которую будут записаны IP-адрес и порт отправителя. `fromlen` указывает на локальный `int`, который следует инициализировать значением `sizeof *from` или `sizeof(struct sockaddr_storage)`. После возврата функции `fromlen` будет содержать длину адреса, фактически записанного в `from`.

`recvfrom()` возвращает число принятых байтов или `-1` при ошибке, соответственно устанавливая `errno`.

Возникает вопрос: почему мы используем тип `struct sockaddr_storage`, а не `struct sockaddr_in`? Потому что не хотим привязываться ни к IPv4, ни к IPv6. Поэтому используем универсальную структуру `sockaddr_storage`, которой заведомо достаточно для обоих вариантов.

А вот ещё вопрос: почему сама структура `sockaddr` недостаточно велика для любого адреса? Мы даже приводим универсальную `sockaddr_storage` к универсальной `sockaddr`! Кажется лишним и избыточным, правда? Ответ в том, что она просто недостаточно велика, а менять её теперь, полагаю, было бы проблематично. Поэтому создали новую.

Помните: если вызвать `connect()` для дейтаграммного сокета, затем можно просто использовать `send()` и `recv()` для всех операций. Сам сокет по-прежнему остаётся дейтаграммным, пакеты по-прежнему используют UDP, но интерфейс сокетов автоматически добавляет сведения об отправителе и получателе.

5.9. `close()` и `shutdown()` — убирайся с глаз моих!

Уф! Вы целый день отправляли и принимали данные и сыты этим по горло. Пора закрыть соединение на дескрипторе сокета. Это просто: можно воспользоваться обычной Unix-функцией `close()` для файловых дескрипторов:

```
close(sockfd);
```

После этого больше нельзя будет читать из сокета и писать в него. Любая попытка чтения или записи сокета на удалённой стороне приведёт к ошибке.

Если вам нужно немного больше контроля над закрытием сокета, воспользуйтесь функцией `shutdown()`. Она позволяет прекратить обмен в одном направлении или в обоих, как `close()`. Прототип:

```
int shutdown(int sockfd, int how);
```

`sockfd` — дескриптор сокета, работу которого нужно прекратить, а `how` принимает одно из значений:

how	Действие
0	Запретить дальнейший приём
1	Запретить дальнейшую отправку
2	Запретить и отправку, и приём, как `close()`

При успехе `shutdown()` возвращает 0, а при ошибке — `-1`, соответственно устанавливая `errno`.

Если вы соблаговолите вызвать `shutdown()` на неподключённом дейтаграммном сокете, он просто станет недоступен для последующих вызовов `send()` и `recv()` (помните: их можно использовать, если подключить дейтаграммный сокет через `connect()`).

Важно отметить, что `shutdown()` на самом деле не закрывает файловый дескриптор, а только меняет возможности его использования. Для освобождения дескриптора сокета нужно вызвать `close()`.

Ничего сложного.

Только помните: в Windows при использовании Winsock вместо `close()` следует вызывать `closesocket()`.

5.10. `getpeername()` — кто ты?

Эта функция настолько проста, что я почти не стал посвящать ей отдельный раздел. Но вот он.

`getpeername()` сообщает, кто находится на другом конце подключённого потокового сокета. Прототип:

```
#include <sys/socket.h>

int getpeername(int sockfd, struct sockaddr *addr, int *addrlen);
```

`sockfd` — дескриптор подключённого потокового сокета; `addr` — указатель на `struct sockaddr` (или `struct sockaddr_in`), которая получит сведения о другой стороне соединения; `addrlen` — указатель на `int`, который следует инициализировать значением `sizeof *addr` или `sizeof(struct sockaddr)`.

При ошибке функция возвращает `-1` и соответственно устанавливает `errno`.

Получив адрес, можно использовать `inet_ntop()`, `getnameinfo()` или `gethostbyaddr()`, чтобы вывести его или получить дополнительные сведения. Нет, имя пользователя так получить нельзя. Ладно, ладно: если на другом компьютере работает демон ident, это возможно, но выходит за рамки документа. Подробнее см. RFC 1413.

5.11. `gethostname()` — кто я?

Ещё проще, чем `getpeername()`, функция `gethostname()`. Она возвращает имя компьютера, на котором работает ваша программа. Это имя можно передать описанной выше `getaddrinfo()`, чтобы определить IP-адрес локальной машины.

Что может быть интереснее? Я могу придумать кое-что, но к программированию сокетов это не относится. Как бы то ни было, вот описание:

```
#include <unistd.h>

int gethostname(char *hostname, size_t size);
```

Аргументы просты: `hostname` указывает на массив символов, в котором после возврата функции окажется имя узла, а `size` — длина массива `hostname` в байтах.

При успешном завершении функция возвращает 0, а при ошибке — `-1` и, как обычно, устанавливает `errno`.

6. Основы клиент-серверной модели

Мы живём в мире клиентов и серверов. Почти всё в сети связано с обменом данными между клиентскими и серверными процессами. Возьмём, например, `telnet`. Когда вы с его помощью — в роли клиента — подключаетесь к удалённому узлу на порт 23, на этом узле начинает работать программа `telnetd` — сервер. Она обрабатывает входящее telnet-соединение, показывает приглашение для входа и так далее.

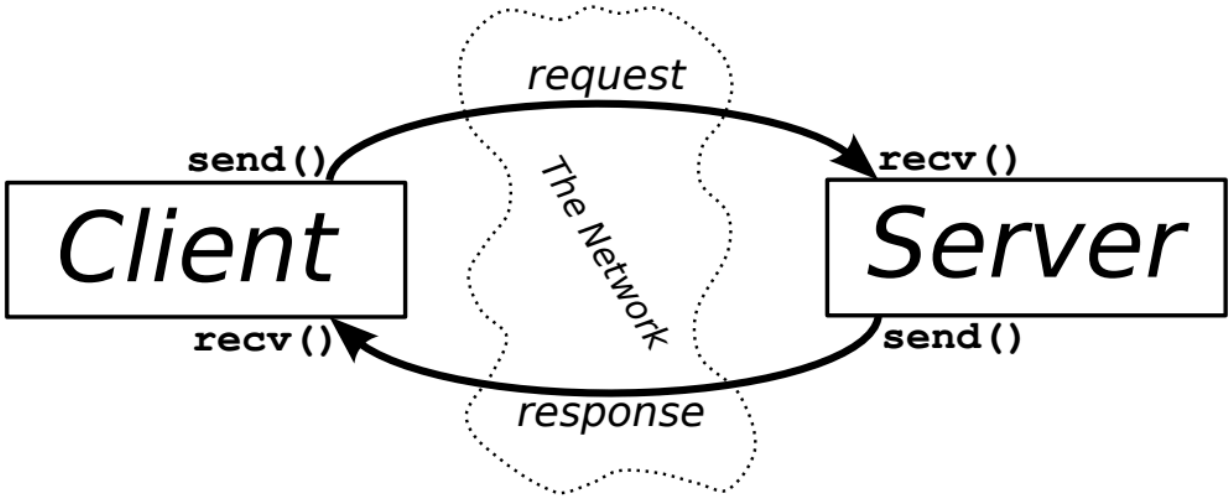


Рис. 6.1. Взаимодействие клиента и сервера. Client — клиент, Server — сервер, The Network — сеть, request — запрос, response — ответ. Схема показывает обмен информацией между ними.

Заметьте: клиент и сервер могут общаться с помощью `SOCK_STREAM`, `SOCK_DGRAM` или чего угодно ещё, если используют одно и то же. Хорошие примеры клиент-серверных пар — `telnet` / `telnetd`, `ftp` / `ftpd` и Firefox / Apache. Каждый раз, когда вы используете `ftp`, удалённая программа `ftpd` обслуживает ваш запрос.

Часто на машине работает только один сервер, который обрабатывает множество клиентов с помощью `fork()`. Основной порядок действий таков: сервер ожидает соединения, принимает его через `accept()` и создаёт дочерний процесс через `fork()` для его обработки. Так работает наш пример сервера в следующем разделе.

6.1. Простой потоковый сервер

Этот сервер делает только одно: отправляет строку `Hello, world!` через потоковое соединение. Чтобы проверить его, запустите сервер в одном окне, а из другого подключитесь к нему через telnet:

```
$ telnet remotehostname 3490
```

Здесь `remotehostname` — имя машины, на которой запущен сервер.

Исходный код сервера:

```
/*
** server.c -- a stream socket server demo
*/
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>
#include <arpa/inet.h>
#include <sys/wait.h>
#include <signal.h>

#define PORT "3490" // the port users will be connecting to

#define BACKLOG 10 // how many pending connections queue will hold

void sigchld_handler(int s)
{
  (void)s; // quiet unused variable warning

  // waitpid() might overwrite errno, so we save and restore it:
  int saved_errno = errno;

  while(waitpid(-1, NULL, WNOHANG) > 0);

  errno = saved_errno;
}


// get sockaddr, IPv4 or IPv6:
void *get_in_addr(struct sockaddr *sa)
{
  if (sa->sa_family == AF_INET) {
    return &(((struct sockaddr_in*)sa)->sin_addr);
  }

  return &(((struct sockaddr_in6*)sa)->sin6_addr);
}

int main(void)
{
  // listen on sock_fd, new connection on new_fd
  int sockfd, new_fd;
  struct addrinfo hints, *servinfo, *p;
  struct sockaddr_storage their_addr; // connector's address info
  socklen_t sin_size;
  struct sigaction sa;
  int yes=1;
  char s[INET6_ADDRSTRLEN];
  int rv;

  memset(&hints, 0, sizeof hints);
  hints.ai_family = AF_INET;
  hints.ai_socktype = SOCK_STREAM;
  hints.ai_flags = AI_PASSIVE; // use my IP

  if ((rv = getaddrinfo(NULL, PORT, &hints, &servinfo)) != 0) {
    fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(rv));
    return 1;
  }

  // loop through all the results and bind to the first we can
  for(p = servinfo; p != NULL; p = p->ai_next) {
    if ((sockfd = socket(p->ai_family, p->ai_socktype,
```

```
         p->ai_protocol)) == -1) {
      perror("server: socket");
      continue;
   }

   if (setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &yes,
         sizeof(int)) == -1) {
      perror("setsockopt");
      exit(1);
   }

   if (bind(sockfd, p->ai_addr, p->ai_addrlen) == -1) {
      close(sockfd);
      perror("server: bind");
      continue;
   }

   break;
 }

 freeaddrinfo(servinfo); // all done with this structure

 if (p == NULL) {
   fprintf(stderr, "server: failed to bind\n");
   exit(1);
 }

 if (listen(sockfd, BACKLOG) == -1) {
   perror("listen");
   exit(1);
}

sa.sa_handler = sigchld_handler; // reap all dead processes
sigemptyset(&sa.sa_mask);
sa.sa_flags = SA_RESTART;
if (sigaction(SIGCHLD, &sa, NULL) == -1) {
  perror("sigaction");
  exit(1);
}

printf("server: waiting for connections...\n");

while(1) { // main accept() loop
  sin_size = sizeof their_addr;
  new_fd = accept(sockfd, (struct sockaddr *)&their_addr,
     &sin_size);
  if (new_fd == -1) {
     perror("accept");
     continue;
  }

  inet_ntop(their_addr.ss_family,
     get_in_addr((struct sockaddr *)&their_addr),
     s, sizeof s);
  printf("server: got connection from %s\n", s);

  if (!fork()) { // this is the child process
     close(sockfd); // child doesn't need the listener
     if (send(new_fd, "Hello, world!", 13, 0) == -1)
        perror("send");
     close(new_fd);
     exit(0);
  }
  close(new_fd); // parent doesn't need this
}
```

```
 return 0;
}
```

Если вам интересно, весь код помещён в одну большую функцию `main()` ради, как мне кажется, синтаксической ясности. Не стесняйтесь разбить его на более мелкие функции, если вам так удобнее.

Возможно, вся эта история с `sigaction()` для вас нова — ничего страшного. Этот код собирает процессы-зомби, которые появляются после завершения дочерних процессов, созданных через `fork()`. Если создавать много зомби и не собирать их, ваш системный администратор забеспокоится.

Получить данные от этого сервера можно с помощью клиента, приведённого в следующем разделе.

6.2. Простой потоковый клиент

Этот парень ещё проще сервера. Клиент только подключается к узлу, указанному в командной строке, на порт 3490 и получает строку, отправленную сервером.

Исходный код клиента:

```
/*
** client.c -- a stream socket client demo
*/

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>
#include <netdb.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <sys/socket.h>

#include <arpa/inet.h>

#define PORT "3490" // the port client will be connecting to

#define MAXDATASIZE 100 // max number of bytes we can get at once

// get sockaddr, IPv4 or IPv6:
void *get_in_addr(struct sockaddr *sa)
{
  if (sa->sa_family == AF_INET) {
    return &(((struct sockaddr_in*)sa)->sin_addr);
  }

  return &(((struct sockaddr_in6*)sa)->sin6_addr);
}

int main(int argc, char *argv[])
{
  int sockfd, numbytes;
  char buf[MAXDATASIZE];
  struct addrinfo hints, *servinfo, *p;
  int rv;
  char s[INET6_ADDRSTRLEN];

  if (argc != 2) {
    fprintf(stderr,"usage: client hostname\n");
    exit(1);
```

```
  }

  memset(&hints, 0, sizeof hints);
  hints.ai_family = AF_UNSPEC;
  hints.ai_socktype = SOCK_STREAM;

  if ((rv = getaddrinfo(argv[1], PORT, &hints, &servinfo)) != 0) {
    fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(rv));
    return 1;
  }

  // loop through all the results and connect to the first we can
  for(p = servinfo; p != NULL; p = p->ai_next) {
    if ((sockfd = socket(p->ai_family, p->ai_socktype,
          p->ai_protocol)) == -1) {
       perror("client: socket");
       continue;
    }

    inet_ntop(p->ai_family,
       get_in_addr((struct sockaddr *)p->ai_addr),
       s, sizeof s);
    printf("client: attempting connection to %s\n", s);

    if (connect(sockfd, p->ai_addr, p->ai_addrlen) == -1) {
       perror("client: connect");
       close(sockfd);
       continue;
    }

    break;
  }

  if (p == NULL) {
    fprintf(stderr, "client: failed to connect\n");
    return 2;
  }

  inet_ntop(p->ai_family,
       get_in_addr((struct sockaddr *)p->ai_addr),
       s, sizeof s);
  printf("client: connected to %s\n", s);

  freeaddrinfo(servinfo); // all done with this structure

  if ((numbytes = recv(sockfd, buf, MAXDATASIZE-1, 0)) == -1) {
    perror("recv");
    exit(1);
  }

  buf[numbytes] = '\0';

  printf("client: received '%s'\n",buf);

  close(sockfd);

  return 0;
}
```

Обратите внимание: если запустить клиент до сервера, `connect()` вернёт ошибку «Connection refused» — «соединение отклонено». Очень полезно.

6.3. Дейтаграммные сокеты

Основы дейтаграммных UDP-сокетов мы уже рассмотрели, обсуждая выше `sendto()` и `recvfrom()`, поэтому я просто приведу пару примеров: `talker.c` и `listener.c`.

`listener` ожидает входящий пакет на машине с портом 4950. `talker` отправляет на этот порт указанной машины пакет, содержащий то, что пользователь ввёл в командной строке.

Поскольку дейтаграммные сокеты работают без соединения и беззаботно отправляют пакеты в эфир, не заботясь об успехе, мы явно укажем клиенту и серверу использовать IPv6. Так мы избежим ситуации, когда сервер слушает IPv6, а клиент отправляет через IPv4: данные просто не будут получены. (В мире подключённых потоковых TCP-сокетов такое несоответствие тоже возможно, но ошибка `connect()` для одного семейства адресов заставит нас попробовать другое.)

Исходный код listener.c:

```
/*
** listener.c -- a datagram sockets "server" demo
*/

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>

#define MYPORT "4950" // the port users will be connecting to

#define MAXBUFLEN 100

// get sockaddr, IPv4 or IPv6:
void *get_in_addr(struct sockaddr *sa)
{
  if (sa->sa_family == AF_INET) {
    return &(((struct sockaddr_in*)sa)->sin_addr);
  }

  return &(((struct sockaddr_in6*)sa)->sin6_addr);
}

int main(void)
{
  int sockfd;
  struct addrinfo hints, *servinfo, *p;
  int rv;
  int numbytes;
  struct sockaddr_storage their_addr;
  char buf[MAXBUFLEN];
  socklen_t addr_len;
  char s[INET6_ADDRSTRLEN];

  memset(&hints, 0, sizeof hints);
  hints.ai_family = AF_INET6; // or set to AF_INET to use IPv4
  hints.ai_socktype = SOCK_DGRAM;
  hints.ai_flags = AI_PASSIVE; // use my IP
```

```
  if ((rv = getaddrinfo(NULL, MYPORT, &hints, &servinfo)) != 0) {
    fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(rv));
    return 1;
  }

  // loop through all the results and bind to the first we can
  for(p = servinfo; p != NULL; p = p->ai_next) {
    if ((sockfd = socket(p->ai_family, p->ai_socktype,
          p->ai_protocol)) == -1) {
       perror("listener: socket");
       continue;
    }

    if (bind(sockfd, p->ai_addr, p->ai_addrlen) == -1) {
       close(sockfd);
       perror("listener: bind");
       continue;
    }

    break;
  }

  if (p == NULL) {
    fprintf(stderr, "listener: failed to bind socket\n");
    return 2;
  }

  freeaddrinfo(servinfo);

  printf("listener: waiting to recvfrom...\n");

  addr_len = sizeof their_addr;
  if ((numbytes = recvfrom(sockfd, buf, MAXBUFLEN-1 , 0,
    (struct sockaddr *)&their_addr, &addr_len)) == -1) {
    perror("recvfrom");
    exit(1);
  }

  printf("listener: got packet from %s\n",
    inet_ntop(their_addr.ss_family,
       get_in_addr((struct sockaddr *)&their_addr),
       s, sizeof s));
  printf("listener: packet is %d bytes long\n", numbytes);
  buf[numbytes] = '\0';
  printf("listener: packet contains \"%s\"\n", buf);

  close(sockfd);

  return 0;
}
```

Обратите внимание, что в вызове `getaddrinfo()` мы наконец используем `SOCK_DGRAM`. Также заметьте: вызывать `listen()` или `accept()` не нужно. Это одно из преимуществ неподключённых дейтаграммных сокетов!

Далее идёт исходный код talker.c:

```
/*
** talker.c -- a datagram "client" demo
*/

#include <stdio.h>
#include <stdlib.h>
```

```
#include <unistd.h>
#include <errno.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>

#define SERVERPORT "4950" // the port users will be connecting to

int main(int argc, char *argv[])
{
  int sockfd;
  struct addrinfo hints, *servinfo, *p;
  int rv;
  int numbytes;

  if (argc != 3) {
    fprintf(stderr,"usage: talker hostname message\n");
    exit(1);
  }

  memset(&hints, 0, sizeof hints);
  hints.ai_family = AF_INET6; // set to AF_INET to use IPv4
  hints.ai_socktype = SOCK_DGRAM;

  rv = getaddrinfo(argv[1], SERVERPORT, &hints, &servinfo);
  if (rv != 0) {
    fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(rv));
    return 1;
  }

  // loop through all the results and make a socket
  for(p = servinfo; p != NULL; p = p->ai_next) {
    if ((sockfd = socket(p->ai_family, p->ai_socktype,
          p->ai_protocol)) == -1) {
       perror("talker: socket");
       continue;
    }

    break;
  }

  if (p == NULL) {
    fprintf(stderr, "talker: failed to create socket\n");
    return 2;
  }

  if ((numbytes = sendto(sockfd, argv[2], strlen(argv[2]), 0,
        p->ai_addr, p->ai_addrlen)) == -1) {
    perror("talker: sendto");
    exit(1);
  }

  freeaddrinfo(servinfo);

  printf("talker: sent %d bytes to %s\n", numbytes, argv[1]);
  close(sockfd);

  return 0;
}
```

Вот и всё! Запустите `listener` на одной машине, затем `talker` на другой. Смотрите, как они общаются! Развлечение для всей семьи без возрастных ограничений!

На этот раз даже не обязательно запускать сервер! Можно запустить `talker` отдельно, и он будет с радостью отправлять пакеты в эфир, где они исчезнут, если на другой стороне никто не готов принять их через `recvfrom()`. Помните: доставка данных, отправленных через дейтаграммные UDP-сокеты, не гарантируется!

За исключением ещё одной маленькой подробности, о которой я много раз упоминал: подключённые дейтаграммные сокеты. Нужно рассказать о них здесь, раз уж мы обсуждаем дейтаграммы. Допустим, `talker` вызывает `connect()` и указывает адрес `listener`. С этого момента `talker` может отправлять данные и получать их только по адресу, указанному в `connect()`. Поэтому пользоваться `sendto()` и `recvfrom()` необязательно: можно просто вызвать `send()` и `recv()`.

7. Чуть более сложные приёмы

На самом деле эти приёмы не такие уж сложные, но они выходят за пределы рассмотренных нами основ. Вообще, если вы дочитали до этого места, считайте, что вы вполне освоили основы сетевого программирования в Unix! Поздравляю!

Теперь отправимся в дивный новый мир более необычных вещей, которые вы, возможно, захотите узнать о сокетах. Вперёд!

7.1. Блокирование

Блокирование. Вы о нём слышали, но что же это такое? В двух словах, «блокироваться» на техническом жаргоне значит «засыпать». Вы, наверное, заметили, что приведённая выше программа `listener` просто ждёт, пока не придёт пакет. Произошло вот что: она вызвала `recvfrom()`, данных не оказалось, и поэтому говорят, что `recvfrom()` блокируется, то есть засыпает, до поступления данных.

Множество функций блокируются. Блокируется `accept()`. Блокируются все функции семейства `recv()`. Они могут это делать потому, что им разрешено. Когда вы создаёте дескриптор сокета через `socket()`, ядро по умолчанию делает его блокирующим. Если вы хотите, чтобы сокет был неблокирующим, нужно вызвать `fcntl()`:

```
#include <unistd.h>
#include <fcntl.h>
...
sockfd = socket(PF_INET, SOCK_STREAM, 0);
fcntl(sockfd, F_SETFL, O_NONBLOCK);
...
```

Установив сокет в неблокирующий режим, можно фактически «опрашивать» его на наличие информации. Если попытаться читать из неблокирующего сокета, в котором нет данных, ему нельзя блокироваться: он вернёт `-1`, а `errno` будет установлен в `EAGAIN` или `EWOULDBLOCK`.

Погодите — может вернуться и `EAGAIN`, и `EWOULDBLOCK`? Что проверять? Спецификация не определяет, какой из кодов вернёт ваша система, поэтому для переносимости проверяйте оба.

Впрочем, обычно такой опрос — плохая идея. Если программа будет в цикле активного ожидания искать данные в сокете, она станет безудержно расходовать процессорное время. Более элегантный способ проверить, ожидают ли данные чтения, рассмотрен в следующем разделе о `poll()`.

7.2. `poll()` — синхронное мультиплексирование ввода-вывода

На самом деле вам хотелось бы одновременно следить за множеством сокетов и обрабатывать те из них, для которых готовы данные. Тогда не придётся постоянно опрашивать все сокеты, выясняя, какие готовы к чтению.

Предупреждение: при огромном числе соединений `poll()` ужасно медленна. В таких условиях лучшую производительность даст библиотека обработки событий, например libevent, которая старается выбрать самый быстрый метод из доступных в вашей системе.

Как же избежать опроса? По иронии судьбы — вызвав `poll()`. Если коротко, мы попросим операционную систему сделать за нас всю грязную работу и сообщить, на каких сокетах готовы данные для чтения. Тем временем наш процесс может спать, экономя системные ресурсы.

Общий план таков: мы храним массив структур `pollfd` со сведениями о том, за какими дескрипторами сокетов и какими событиями хотим следить. ОС блокирует вызов `poll()`, пока не произойдёт одно из событий — например, «сокет готов к чтению!» — или не истечёт заданное время ожидания.

Удобно, что прослушивающий сокет сообщает о готовности к чтению, когда новое входящее соединение готово к принятию через `accept()`.

Хватит разговоров. Как этим пользоваться?

```
#include <poll.h>
int poll(struct pollfd fds[], nfds_t nfds, int timeout);
```

`fds` — массив сведений о том, какие события каких сокетов отслеживать; `nfds` — число элементов массива; `timeout` — время ожидания в миллисекундах. Функция возвращает число элементов массива, для которых произошло событие.

Посмотрим на структуру:

```
struct pollfd {
    int fd;        // дескриптор сокета
    short events;  // битовая маска интересующих нас событий
    short revents; // после возврата — маска произошедших событий
};
```

Мы создадим массив таких структур и запишем в поле `fd` каждого элемента дескриптор сокета, за которым хотим следить. Затем в поле `events` укажем типы интересующих нас событий.

`events` содержит побитовое ИЛИ следующих значений:

Макрос	Описание
`POLLIN`	Сообщить, когда данные на сокете готовы к приёму через `recv()`.
`POLLOUT`	Сообщить, когда данные можно отправить в сокет через `send()` без блокировки.
`POLLHUP`	Сообщить, когда удалённая сторона закрыла соединение.

Подготовив массив `pollfd`, передайте его в `poll()` вместе с размером массива и временем ожидания в миллисекундах. Отрицательное время ожидания означает бесконечное ожидание.

После возврата `poll()` проверьте поле `revents`: установленный бит `POLLIN` или `POLLOUT` укажет, что соответствующее событие произошло.

С `poll()` можно сделать и больше; подробности см. на странице man для `poll()` ниже.

В следующем примере мы подождём 2,5 секунды готовности данных для чтения со стандартного ввода, то есть вашего нажатия Enter:

```
#include <stdio.h>
#include <poll.h>
```

```
int main(void)
{
    struct pollfd pfds[1]; // больше элементов, если нужно следить за большим числом
    pfds[0].fd = 0;        // стандартный ввод
    pfds[0].events = POLLIN; // сообщить о готовности к чтению

    // Если нужно следить и за другими объектами:
    // pfds[1].fd = some_socket; // некоторый дескриптор сокета
    // pfds[1].events = POLLIN;  // сообщить о готовности к чтению

    printf("Hit RETURN or wait 2.5 seconds for timeout\n");
    int num_events = poll(pfds, 1, 2500); // ожидание 2,5 секунды

    if (num_events == 0) {
        printf("Poll timed out!\n");
    } else {
        int pollin_happened = pfds[0].revents & POLLIN;
        if (pollin_happened) {
            printf("File descriptor %d is ready to read\n", pfds[0].fd);
        } else {
            printf("Unexpected event occurred: %d\n", pfds[0].revents);
        }
    }
    return 0;
}
```

Ещё раз обратите внимание: `poll()` возвращает число элементов массива `pfds`, для которых произошли события. Она не сообщает, какие именно это элементы — их по-прежнему нужно искать перебором, — но говорит, у скольких записей поле `revents` ненулевое. Поэтому, найдя нужное количество, поиск можно прекратить.

Здесь могут возникнуть вопросы. Как добавить новые файловые дескрипторы в набор, передаваемый `poll()`? Просто убедитесь, что в массиве достаточно места, или расширяйте его по мере необходимости с помощью `realloc()`.

Как удалить элемент из набора? Можно скопировать последний элемент массива на место удаляемого и передать в `poll()` уменьшенное на единицу количество. Другой вариант — установить поле `fd` в отрицательное число, и `poll()` проигнорирует его.

Как собрать всё вместе и получить сервер чата, к которому можно подключиться через telnet?

Мы создадим прослушивающий сокет и добавим его к набору дескрипторов файлов для `poll()`: он покажет готовность к чтению при входящем соединении. Затем будем добавлять новые соединения в массив `pollfd` и динамически расширять его, если место закончится. Когда соединение закрывается, мы удалим его из массива. А когда соединение готово к чтению, прочитаем данные и отправим их всем остальным соединениям, чтобы другие пользователи увидели набранный текст.

Попробуйте этот сервер на poll. Запустите его в одном окне, а затем выполните `telnet localhost 9034` в нескольких других окнах терминала. То, что вы наберёте в одном окне, должно появиться в остальных после нажатия Enter.

Более того, если нажать Ctrl-] и ввести `quit`, чтобы выйти из telnet, сервер должен обнаружить разрыв соединения и удалить вас из массива файловых дескрипторов.

```
/*
** pollserver.c -- a cheezy multiperson chat server
*/
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>
#include <poll.h>

#define PORT "9034" // Port we're listening on

/*
* Convert socket to IP address string.
* addr: struct sockaddr_in or struct sockaddr_in6
*/
const char *inet_ntop2(void *addr, char *buf, size_t size)
{
  struct sockaddr_storage *sas = addr;
  struct sockaddr_in *sa4;
  struct sockaddr_in6 *sa6;
  void *src;

  switch (sas->ss_family) {
    case AF_INET:
       sa4 = addr;
       src = &(sa4->sin_addr);
       break;
    case AF_INET6:
       sa6 = addr;
       src = &(sa6->sin6_addr);
       break;
    default:
       return NULL;
  }

  return inet_ntop(sas->ss_family, src, buf, size);
}

/*
* Return a listening socket.
*/
int get_listener_socket(void)
{
  int listener; // Listening socket descriptor
  int yes=1; // For setsockopt() SO_REUSEADDR, below
  int rv;

  struct addrinfo hints, *ai, *p;

  // Get us a socket and bind it
  memset(&hints, 0, sizeof hints);
  hints.ai_family = AF_INET;
  hints.ai_socktype = SOCK_STREAM;
  hints.ai_flags = AI_PASSIVE;
  if ((rv = getaddrinfo(NULL, PORT, &hints, &ai)) != 0) {
    fprintf(stderr, "pollserver: %s\n", gai_strerror(rv));
    exit(1);
  }

  for(p = ai; p != NULL; p = p->ai_next) {
    listener = socket(p->ai_family, p->ai_socktype,
          p->ai_protocol);
    if (listener < 0) {
       continue;
```

```
    }

    // Lose the pesky "address already in use" error message
    setsockopt(listener, SOL_SOCKET, SO_REUSEADDR, &yes,
          sizeof(int));

    if (bind(listener, p->ai_addr, p->ai_addrlen) < 0) {
       close(listener);
       continue;
    }

    break;
  }

  // If we got here, it means we didn't get bound
  if (p == NULL) {
    return -1;
  }

  freeaddrinfo(ai); // All done with this

  // Listen
  if (listen(listener, 10) == -1) {
    return -1;
  }

  return listener;
}

/*
* Add a new file descriptor to the set.
*/
void add_to_pfds(struct pollfd **pfds, int newfd, int *fd_count,
   int *fd_size)
{
 // If we don't have room, add more space in the pfds array
 if (*fd_count == *fd_size) {
   *fd_size *= 2; // Double it
   *pfds = realloc(*pfds, sizeof(**pfds) * (*fd_size));
 }

 (*pfds)[*fd_count].fd = newfd;
 (*pfds)[*fd_count].events = POLLIN; // Check ready-to-read
 (*pfds)[*fd_count].revents = 0;

 (*fd_count)++;
}

/*
* Remove a file descriptor at a given index from the set.
*/
void del_from_pfds(struct pollfd pfds[], int i, int *fd_count)
{
 // Copy the one from the end over this one
 pfds[i] = pfds[*fd_count-1];

 (*fd_count)--;
}

/*
* Handle incoming connections.
*/
void handle_new_connection(int listener, int *fd_count,
   int *fd_size, struct pollfd **pfds)
{
 struct sockaddr_storage remoteaddr; // Client address
```

```
 socklen_t addrlen;
 int newfd; // Newly accept()ed socket descriptor
 char remoteIP[INET6_ADDRSTRLEN];

 addrlen = sizeof remoteaddr;
 newfd = accept(listener, (struct sockaddr *)&remoteaddr,
      &addrlen);

 if (newfd == -1) {
   perror("accept");
 } else {
   add_to_pfds(pfds, newfd, fd_count, fd_size);

   printf("pollserver: new connection from %s on socket %d\n",
         inet_ntop2(&remoteaddr, remoteIP, sizeof remoteIP),
         newfd);
 }
}

/*
* Handle regular client data or client hangups.
*/
void handle_client_data(int listener, int *fd_count,
   struct pollfd *pfds, int *pfd_i)
{
 char buf[256]; // Buffer for client data

 int nbytes = recv(pfds[*pfd_i].fd, buf, sizeof buf, 0);

 int sender_fd = pfds[*pfd_i].fd;

 if (nbytes <= 0) { // Got error or connection closed by client
   if (nbytes == 0) {
      // Connection closed
      printf("pollserver: socket %d hung up\n", sender_fd);
   } else {
      perror("recv");
   }

   close(pfds[*pfd_i].fd); // Bye!

   del_from_pfds(pfds, *pfd_i, fd_count);

   // reexamine the slot we just deleted
   (*pfd_i)--;

 } else { // We got some good data from a client
   printf("pollserver: recv from fd %d: %.*s", sender_fd,
         nbytes, buf);
   // Send to everyone!
   for(int j = 0; j < *fd_count; j++) {
      int dest_fd = pfds[j].fd;

      // Except the listener and ourselves
      if (dest_fd != listener && dest_fd != sender_fd) {
         if (send(dest_fd, buf, nbytes, 0) == -1) {
           perror("send");
         }
      }
   }
 }
}

/*
* Process all existing connections.
*/
```

```
void process_connections(int listener, int *fd_count, int *fd_size,
   struct pollfd **pfds)
{
 for(int i = 0; i < *fd_count; i++) {

   // Check if someone's ready to read
   if ((*pfds)[i].revents & (POLLIN | POLLHUP)) {
      // We got one!!

      if ((*pfds)[i].fd == listener) {
         // If we're the listener, it's a new connection
         handle_new_connection(listener, fd_count, fd_size,
              pfds);
      } else {
         // Otherwise we're just a regular client
         handle_client_data(listener, fd_count, *pfds, &i);
      }
   }
 }
}

/*
* Main: create a listener and connection set, loop forever
* processing connections.
*/
int main(void)
{
 int listener; // Listening socket descriptor

 // Start off with room for 5 connections
 // (We'll realloc as necessary)
 int fd_size = 5;
 int fd_count = 0;
 struct pollfd *pfds = malloc(sizeof *pfds * fd_size);

 // Set up and get a listening socket
 listener = get_listener_socket();

 if (listener == -1) {
   fprintf(stderr, "error getting listening socket\n");
   exit(1);
 }

 // Add the listener to set;
 // Report ready to read on incoming connection
 pfds[0].fd = listener;
 pfds[0].events = POLLIN;

 fd_count = 1; // For the listener

 puts("pollserver: waiting for connections...");

 // Main loop
 for(;;) {
   int poll_count = poll(pfds, fd_count, -1);

   if (poll_count == -1) {
      perror("poll");
      exit(1);
   }

   // Run through connections looking for data to read
   process_connections(listener, &fd_count, &fd_size, &pfds);
 }
```

```
    free(pfds);
}
```

В следующем разделе мы рассмотрим похожую, более старую функцию `select()`. Функции `select()` и `poll()` предоставляют сходные возможности и производительность, различаясь главным образом способом использования. `select()` может быть несколько более переносимой, но несколько менее удобной. Выбирайте ту, которая вам больше нравится, если она поддерживается вашей системой.

7.3. `select()` — синхронное мультиплексирование ввода-вывода по старинке

Эта функция несколько необычна, но очень полезна. Представьте: вы — сервер и хотите одновременно ожидать входящие соединения и читать данные из уже существующих.

«Нет проблем, — скажете вы, — достаточно `accept()` и пары вызовов `recv()`». Не так быстро! Что, если вы заблокированы на вызове `accept()`? Как одновременно получать данные через `recv()`? «Использовать неблокирующие сокеты!» Ни за что: вы ведь не хотите зря расходовать процессорное время. Что же делать?

`select()` позволяет одновременно следить за несколькими сокетами. Она сообщит, какие готовы к чтению, какие готовы к записи и на каких возникли исключительные ситуации, если вам это нужно.

Предупреждение: хотя `select()` очень переносима, при огромном числе соединений она ужасно медленна. В этих условиях лучше воспользоваться библиотекой событий, например libevent, которая старается выбрать самый быстрый метод из доступных в системе.

Без лишних слов приведу прототип `select()`:

```
#include <sys/time.h>
#include <sys/types.h>
#include <unistd.h>

int select(int numfds, fd_set *readfds, fd_set *writefds,
           fd_set *exceptfds, struct timeval *timeout);
```

Функция наблюдает за «наборами» файловых дескрипторов: `readfds`, `writefds` и `exceptfds`. Если вы хотите узнать, можно ли читать со стандартного ввода и из сокета `sockfd`, добавьте файловые дескрипторы 0 и `sockfd` в набор `readfds`. Параметр `numfds` должен равняться наибольшему файловому дескриптору плюс один. В этом примере это `sockfd + 1`, поскольку он наверняка больше дескриптора стандартного ввода (0).

После возврата `select()` набор `readfds` изменится так, чтобы показать, какие из выбранных дескрипторов готовы к чтению. Проверить их можно макросом `FD_ISSET()`, описанным ниже.

Прежде чем двигаться дальше, поговорим о работе с наборами. Каждый набор имеет тип `fd_set`. С ним работают следующие макросы:

Макрос	Действие
`FD_SET(int fd, fd_set *set)`	Добавить `fd` в набор `set`.
`FD_CLR(int fd, fd_set *set)`	Удалить `fd` из набора `set`.
`FD_ISSET(int fd, fd_set *set)`	Вернуть истину, если `fd` входит в `set`.

Макрос	Действие
`FD_ZERO(fd_set *set)`	Удалить все элементы набора `set`.

Наконец, что это за странная `struct timeval`? Иногда не хочется вечно ждать, пока кто-нибудь пришлёт данные. Возможно, вы хотите каждые 96 секунд выводить в терминал «Всё ещё работаю…», даже если ничего не произошло. Эта структура позволяет задать предельное время ожидания. Если оно истечёт и `select()` всё ещё не найдёт готовых дескрипторов, функция вернёт управление и вы сможете продолжить обработку.

Поля `struct timeval` выглядят так:

```
struct timeval {
    int tv_sec;  // секунды
    int tv_usec; // микросекунды
};
```

Установите `tv_sec` в число секунд ожидания, а `tv_usec` — в число микросекунд. Да, именно *микросекунд*, а не миллисекунд: в миллисекунде 1000 микросекунд, в секунде — 1000 миллисекунд, то есть 1 000 000 микросекунд. Почему «usec»? Буква `u` должна напоминать греческую μ (мю), обозначающую «микро». Кроме того, при возврате функции значение `timeout` может быть обновлено, чтобы показать оставшееся время; это зависит от вашей разновидности Unix.

Ура! У нас таймер с разрешением в микросекунду! Впрочем, не рассчитывайте на это: независимо от того, насколько малое значение вы зададите в `timeval`, вам, вероятно, придётся подождать некоторую часть обычного временного кванта Unix.

Ещё несколько деталей: если установить поля `timeval` в 0, время ожидания `select()` истечёт немедленно, и функция фактически опросит все дескрипторы в наборах. Если передать `timeout` как `NULL`, время ожидания никогда не истечёт и функция будет ждать готовности первого дескриптора. Наконец, если вас не интересует ожидание определённого набора, можно передать вместо него `NULL`.

Следующий фрагмент ждёт 2,5 секунды появления чего-либо на стандартном вводе:

```
/* select.c -- демонстрация select() */
#include <stdio.h>
#include <sys/time.h>
#include <sys/types.h>
#include <unistd.h>

#define STDIN 0 // файловый дескриптор стандартного ввода

int main(void)
{
    struct timeval tv;
    fd_set readfds;
    tv.tv_sec = 2;
    tv.tv_usec = 500000;
    FD_ZERO(&readfds);
    FD_SET(STDIN, &readfds);
    // writefds и exceptfds нас не интересуют:
    select(STDIN+1, &readfds, NULL, NULL, &tv);
    if (FD_ISSET(STDIN, &readfds))
        printf("A key was pressed!\n");
    else
        printf("Timed out.\n");
```

```
    return 0;
}
```

Если ваш терминал буферизует ввод построчно, нужно нажать Enter, иначе ожидание всё равно истечёт.

Кто-то может подумать, что это отличный способ ждать данных на дейтаграммном сокете, и, возможно, будет прав. Некоторые системы Unix поддерживают такое применение `select`, другие — нет. Если хотите попробовать, посмотрите, что говорит ваша локальная страница man.

Некоторые системы Unix обновляют время в `timeval`, показывая, сколько осталось до истечения ожидания, а другие — нет. Для переносимости не полагайтесь на это. Если нужно отслеживать прошедшее время, используйте `gettimeofday()`. Обидно, знаю, но так уж устроено.

Что произойдёт, если сокет из набора чтения закроет соединение? В этом случае `select()` вернёт его дескриптор как «готовый к чтению». Когда вы действительно вызовете для него `recv()`, функция вернёт 0. Так вы узнаете, что клиент закрыл соединение.

Ещё одно замечание о `select()`: если сокет находится в состоянии прослушивания, можно проверить наличие нового соединения, включив его файловый дескриптор в набор `readfds`.

И это, друзья, был краткий обзор всемогущей функции `select()`.

Но по многочисленным просьбам приведу подробный пример. К сожалению, разница между приведённым выше предельно простым примером и этим существенна. И всё же посмотрите на код, а затем прочитайте следующее за ним описание.

Эта программа работает как простой многопользовательский сервер чата. Запустите её в одном окне и подключитесь к ней из нескольких других через telnet командой `telnet hostname 9034`. Когда вы напечатаете что-нибудь в одном сеансе telnet, текст должен появиться во всех остальных.

```
/*
** selectserver.c -- a cheezy multiperson chat server
*/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>

#define PORT "9034" // port we're listening on

/*
* Convert socket to IP address string.
* addr: struct sockaddr_in or struct sockaddr_in6
*/
const char *inet_ntop2(void *addr, char *buf, size_t size)
{
  struct sockaddr_storage *sas = addr;
  struct sockaddr_in *sa4;
  struct sockaddr_in6 *sa6;
  void *src;

  switch (sas->ss_family) {
    case AF_INET:
```

```
        sa4 = addr;
        src = &(sa4->sin_addr);
        break;
    case AF_INET6:
        sa6 = addr;
        src = &(sa6->sin6_addr);
        break;
    default:
        return NULL;
  }

  return inet_ntop(sas->ss_family, src, buf, size);
}

/*
* Return a listening socket
*/
int get_listener_socket(void)
{
  struct addrinfo hints, *ai, *p;
  int yes=1; // for setsockopt() SO_REUSEADDR, below
  int rv;
  int listener;

  // get us a socket and bind it
  memset(&hints, 0, sizeof hints);
  hints.ai_family = AF_UNSPEC;
  hints.ai_socktype = SOCK_STREAM;
  hints.ai_flags = AI_PASSIVE;
  if ((rv = getaddrinfo(NULL, PORT, &hints, &ai)) != 0) {
    fprintf(stderr, "selectserver: %s\n", gai_strerror(rv));
    exit(1);
  }

  for(p = ai; p != NULL; p = p->ai_next) {
    listener = socket(p->ai_family, p->ai_socktype,
          p->ai_protocol);
    if (listener < 0) {
       continue;
    }

    // lose the pesky "address already in use" error message
    setsockopt(listener, SOL_SOCKET, SO_REUSEADDR, &yes,
          sizeof(int));

    if (bind(listener, p->ai_addr, p->ai_addrlen) < 0) {
       close(listener);
       continue;
    }

    break;
  }

  // if we got here, it means we didn't get bound
  if (p == NULL) {
    fprintf(stderr, "selectserver: failed to bind\n");
    exit(2);
  }

  freeaddrinfo(ai); // all done with this

  // listen
  if (listen(listener, 10) == -1) {
    perror("listen");
    exit(3);
  }
```

```
  return listener;
}

/*
* Add new incoming connections to the proper sets
*/
void handle_new_connection(int listener, fd_set *master, int *fdmax)
{
 socklen_t addrlen;
 int newfd; // newly accept()ed socket descriptor
 struct sockaddr_storage remoteaddr; // client address
 char remoteIP[INET6_ADDRSTRLEN];

 addrlen = sizeof remoteaddr;
 newfd = accept(listener,
   (struct sockaddr *)&remoteaddr,
   &addrlen);

 if (newfd == -1) {
   perror("accept");
 } else {
   FD_SET(newfd, master); // add to master set
   if (newfd > *fdmax) { // keep track of the max
      *fdmax = newfd;
   }
   printf("selectserver: new connection from %s on "
      "socket %d\n",
      inet_ntop2(&remoteaddr, remoteIP, sizeof remoteIP),
      newfd);
 }
}

/*
* Broadcast a message to all clients
*/
void broadcast(char *buf, int nbytes, int listener, int s,
        fd_set *master, int fdmax)
{
 for(int j = 0; j <= fdmax; j++) {
   // send to everyone!
   if (FD_ISSET(j, master)) {
      // except the listener and ourselves
      if (j != listener && j != s) {
         if (send(j, buf, nbytes, 0) == -1) {
           perror("send");
         }
      }
   }
 }
}

/*
* Handle client data and hangups
*/
void handle_client_data(int s, int listener, fd_set *master,
              int fdmax)
{
 char buf[256]; // buffer for client data
 int nbytes;

 // handle data from a client
 if ((nbytes = recv(s, buf, sizeof buf, 0)) <= 0) {
   // got error or connection closed by client
   if (nbytes == 0) {
      // connection closed
```

```
      printf("selectserver: socket %d hung up\n", s);
   } else {
      perror("recv");
   }
   close(s); // bye!
   FD_CLR(s, master); // remove from master set
 } else {
   // we got some data from a client
   broadcast(buf, nbytes, listener, s, master, fdmax);
 }
}

/*
* Main
*/
int main(void)
{
 fd_set master; // master file descriptor list
 fd_set read_fds; // temp file descriptor list for select()
 int fdmax; // maximum file descriptor number

 int listener; // listening socket descriptor

 FD_ZERO(&master); // clear the master and temp sets
 FD_ZERO(&read_fds);

 listener = get_listener_socket();

 // add the listener to the master set
 FD_SET(listener, &master);

 // keep track of the biggest file descriptor
 fdmax = listener; // so far, it's this one

 // main loop
 for(;;) {
   read_fds = master; // copy it
   if (select(fdmax+1, &read_fds, NULL, NULL, NULL) == -1) {
      perror("select");
      exit(4);
   }

   // run through the existing connections looking for data
   // to read
   for(int i = 0; i <= fdmax; i++) {
      if (FD_ISSET(i, &read_fds)) { // we got one!!
         if (i == listener)
           handle_new_connection(i, &master, &fdmax);
         else
           handle_client_data(i, listener, &master, fdmax);
      }
   }
 }

 return 0;
}
```

Обратите внимание: в коде есть два набора файловых дескрипторов, `master` и `read_fds`. Первый, `master`, содержит все дескрипторы подключённых сокетов, а также дескриптор сокета, ожидающего новые соединения.

Набор `master` нужен потому, что `select()` изменяет переданный ей набор, оставляя в нём сокеты, готовые к чтению. Поскольку нам нужно отслеживать соединения от одного вызова `select()` до следующего, мы

должны где-то надёжно их хранить. В последний момент я копирую `master` в `read_fds`, после чего вызываю `select()`.

Но разве это не значит, что при каждом новом соединении нужно добавлять его в `master`? Ага! И что каждый раз, когда соединение закрывается, его нужно удалять из `master`? Именно так.

Заметьте, я проверяю готовность прослушивающего сокета к чтению. Когда он готов, это означает наличие ожидающего соединения; я принимаю его через `accept()` и добавляю в `master`. Аналогично, когда соединение клиента готово к чтению и `recv()` возвращает 0, я знаю, что клиент закрыл соединение, и должен удалить его из `master`.

Если же клиентский `recv()` возвращает ненулевое значение, значит получены данные. Я считываю их, затем прохожу по списку `master` и отправляю всем остальным подключённым клиентам.

И это, друзья, был уже не такой простой обзор всемогущей функции `select()`.

Короткое замечание для поклонников Linux: иногда, в редких обстоятельствах, Linux-версия `select()` может сообщить о готовности к чтению, хотя на самом деле её нет! Тогда последующий `read()` заблокируется, хотя `select()` сказала, что этого не произойдёт! Ах ты… Как бы то ни было, решение — установить флаг `O_NONBLOCK` на принимающем сокете, чтобы функция завершалась с `EWOULDBLOCK`; эту ошибку в таком случае можно спокойно игнорировать. Подробнее об установке неблокирующего режима см. справочную страницу `fcntl()`.

И ещё мысль вдогонку: есть другая функция, `poll()`, которая ведёт себя примерно как `select()`, но использует иную систему управления наборами дескрипторов. Обратите на неё внимание!

7.4. Обработка частичной отправки `send()`

Помните, в разделе о `send()` я говорил, что функция может отправить не все байты, о которых вы попросили? Допустим, вы хотите отправить 512 байтов, а она возвращает 412. Куда делись оставшиеся 100?

Они по-прежнему лежат в вашем буфере и ждут отправки. По причинам, находящимся вне вашего контроля, ядро решило не отправлять все данные одним блоком. Теперь, друг мой, именно вам предстоит их дослать.

Для этого можно написать такую функцию:

```
#include <sys/types.h>
#include <sys/socket.h>

int sendall(int s, char *buf, int *len)
{
    int total = 0;        // сколько байтов мы отправили
    int bytesleft = *len; // сколько ещё осталось отправить
    int n;

    while (total < *len) {
        n = send(s, buf+total, bytesleft, 0);
        if (n == -1) { break; }
        total += n;
        bytesleft -= n;
    }

    *len = total; // возвращаем число действительно отправленных байтов
```

```
    return n == -1 ? -1 : 0; // -1 при ошибке, 0 при успехе
}
```

В этом примере `s` — сокет, в который нужно отправить данные, `buf` — буфер с данными, а `len` — указатель на `int`, содержащий число байтов в буфере.

Функция возвращает `-1` при ошибке; значение `errno`, установленное вызовом `send()`, сохраняется. Через `len` также возвращается число фактически отправленных байтов. Если ошибки не было, оно совпадёт с запрошенным. `sendall()` изо всех сил старается отправить данные, но при ошибке сразу сообщает вам об этом.

Для полноты картины вот пример вызова:

```
char buf[10] = "Beej!";
int len;

len = strlen(buf);
if (sendall(s, buf, &len) == -1) {
    perror("sendall");
    printf("We only sent %d bytes because of the error!\n", len);
}
```

Что происходит на принимающей стороне, когда приходит часть пакета? Если пакеты имеют переменную длину, как получатель узнаёт, где один заканчивается и начинается другой? Да, реальные задачи бывают ужасно неудобными. Вероятно, потребуется инкапсуляция — помните раздел об инкапсуляции данных в самом начале? Читайте дальше!

7.5. Сериализация — как упаковать данные

Как вы уже убедились, передавать по сети текст достаточно просто. Но что, если нужно отправить «двоичные» данные, например значения `int` или `float`? Оказывается, есть несколько вариантов.

1. Преобразовать число в текст функцией вроде `sprintf()`, затем отправить текст. Получатель преобразует его обратно в число с помощью функции вроде `strtol()`.
2. Просто передать данные как есть, передав указатель на них в `send()`.
3. Закодировать число в переносимый двоичный формат. Получатель его декодирует.

Предварительный показ! Только сегодня вечером!

Занавес поднимается.

Бидж говорит: «Я предпочитаю третий способ!»

КОНЕЦ.

Прежде чем всерьёз начать этот раздел, замечу: для таких задач существуют библиотеки, а написать собственное переносимое решение без ошибок довольно сложно. Поэтому изучите доступные варианты, прежде чем решать реализовать всё самостоятельно. Я привожу эту информацию для тех, кому интересно узнать, как это работает.

Вообще у всех перечисленных методов есть достоинства и недостатки, но, как я уже сказал, обычно мне больше нравится третий. Сначала обсудим преимущества и недостатки двух других.

Первый способ — кодировать числа как текст перед отправкой — удобен тем, что передаваемые данные легко выводить и читать. Иногда человекочитаемый протокол отлично подходит, когда пропускная способность не является критичным ресурсом, как в Internet Relay Chat (IRC). Но преобразование происходит медленно, а результат почти всегда занимает больше места, чем исходное число!

Второй способ — передать необработанные данные. Это очень просто, но опасно: возьмите указатель на отправляемые данные и вызовите `send`:

```
double d = 3490.15926535;
send(s, &d, sizeof d, 0); /* ОПАСНО: непереносимо! */
```

Получатель принимает их так:

```
double d;
recv(s, &d, sizeof d, 0); /* ОПАСНО: непереносимо! */
```

Быстро и просто — что может не понравиться? Оказывается, разные архитектуры могут использовать разные битовые представления `double` и `int`, а также разный порядок байтов! Этот код совершенно непереносим. Впрочем, возможно, переносимость вам не нужна — тогда способ удобен и быстр.

При упаковке целых типов мы уже видели, как семейство функций `htons()` помогает сохранить переносимость, преобразуя числа в сетевой порядок байтов, и почему так делать правильно. К сожалению, аналогичных функций для `float` нет. Неужели вся надежда потеряна?

Не бойтесь! Вы ведь испугались на секунду? Нет? Даже чуть-чуть? Мы можем упаковать данные — говорят также «маршалировать», «сериализовать» и ещё тысячью разных способов — в известный двоичный формат, который получатель сможет распаковать на другой стороне.

Что значит «известный двоичный формат»? Мы ведь уже видели пример `htons()`: она преобразует, или «кодирует», число из формата узла в сетевой порядок байтов. Для обратного преобразования получатель вызывает `ntohs()`.

Но разве я только что не сказал, что аналогичных функций для нецелых типов нет? Да, сказал. А поскольку в C нет стандартного способа сделать это, мы попали в некоторую передрягу — небольшая игра слов для поклонников Python.

Решение — упаковать данные в известный формат и передать их для декодирования. Например, вот быстрая и грубая упаковка `float`, которую можно значительно улучшить (исходный код):

```
#include <stdint.h>

uint32_t htonf(float f)
{
   uint32_t p;
   uint32_t sign;

   if (f < 0) { sign = 1; f = -f; }
   else { sign = 0; }

  // whole part and sign
  p = ((((uint32_t)f)&0x7fff)<<16) | (sign<<31);

  // fraction
  p |= (uint32_t)(((f - (int)f) * 65536.0f))&0xffff;
```

```
  return p;
}

float ntohf(uint32_t p)
{
  float f = ((p>>16)&0x7fff); // whole part
  f += (p&0xffff) / 65536.0f; // fraction

  if (((p>>31)&0x1) == 0x1) { f = -f; } // sign bit set

  return f;
}
```

Этот код — довольно наивная реализация, сохраняющая `float` в 32-битном числе. Старший бит, 31-й, хранит знак: единица означает отрицательное число. Следующие пятнадцать битов, с 30-го по 16-й, хранят целую часть `float`. Наконец, оставшиеся шестнадцать битов, с 15-го по 0-й, хранят дробную часть.

Использовать её довольно просто:

```
#include <stdio.h>

int main(void)
{
   float f = 3.1415926, f2;
   uint32_t netf;

   netf = htonf(f); // convert to "network" form
   f2 = ntohf(netf); // convert back to test

  printf("Original: %f\n", f); // 3.141593
  printf(" Network: 0x%08X\n", netf); // 0x0003243F
  printf("Unpacked: %f\n", f2); // 3.141586

  return 0;
}
```

Достоинства: код маленький, простой и быстрый. Недостатки: пространство используется неэффективно, а диапазон сильно ограничен — попробуйте сохранить число больше 32767, и оно будет недовольно! Кроме того, в примере видно, что последние несколько десятичных знаков сохраняются неточно.

Что можно сделать вместо этого? Стандарт хранения чисел с плавающей точкой известен как IEEE 754. Большинство компьютеров используют этот формат внутри для вычислений с плавающей точкой, поэтому в этих случаях, строго говоря, преобразование не требуется. Но если важна переносимость исходного кода, на такое предположение нельзя безусловно полагаться.

Или можно? Вероятнее всего, ваши системы используют IEEE 754, как и дополнительный код для целых чисел. Если вы уверены в этом, можете передать данные напрямую, хотя порядок байтов всё равно нужно исправить с помощью `htonl()` или подходящей функции — у `float` тоже есть порядок байтов. Именно так поступают `htons()` и подобные функции на системах с порядком байтов от старшего к младшему, где преобразование не нужно.

Но на случай, если ваша система не использует IEEE 754, вот код, который кодирует `float` и `double` в этом формате. Почти: он не кодирует NaN и бесконечность, хотя его можно для этого изменить.

```
#define pack754_32(f) (pack754((f), 32, 8))
#define pack754_64(f) (pack754((f), 64, 11))
#define unpack754_32(i) (unpack754((i), 32, 8))
```

```
#define unpack754_64(i) (unpack754((i), 64, 11))

uint64_t pack754(long double f, unsigned bits, unsigned expbits)
{
   long double fnorm;
   int shift;
  long long sign, exp, significand;

  // -1 for sign bit
  unsigned significandbits = bits - expbits - 1;

  if (f == 0.0) return 0; // get this special case out of the way

  // check sign and begin normalization
  if (f < 0) { sign = 1; fnorm = -f; }
  else { sign = 0; fnorm = f; }

  // get the normalized form of f and track the exponent
  shift = 0;
  while(fnorm >= 2.0) { fnorm /= 2.0; shift++; }
  while(fnorm < 1.0) { fnorm *= 2.0; shift--; }
  fnorm = fnorm - 1.0;

  // calculate the binary form (non-float) of the significand data
  significand = fnorm * ((1LL<<significandbits) + 0.5f);

  // get the biased exponent
  exp = shift + ((1<<(expbits-1)) - 1); // shift + bias

  // return the final answer
  return (sign<<(bits-1)) | (exp<<(bits-expbits-1)) | significand;
}

long double unpack754(uint64_t i, unsigned bits, unsigned expbits)
{
  long double result;
  long long shift;
  unsigned bias;

  // -1 for sign bit
  unsigned significandbits = bits - expbits - 1;

  if (i == 0) return 0.0;

  // pull the significand
  result = (i&((1LL<<significandbits)-1)); // mask
  result /= (1LL<<significandbits); // convert back to float
  result += 1.0f; // add the one back on

  // deal with the exponent
  bias = (1<<(expbits-1)) - 1;
  shift = ((i>>significandbits)&((1LL<<expbits)-1)) - bias;
  while(shift > 0) { result *= 2.0; shift--; }
  while(shift < 0) { result /= 2.0; shift++; }

  // sign it
  result *= (i>>(bits-1))&1? -1.0: 1.0;

  return result;
}
```

Вверху я добавил удобные макросы для упаковки и распаковки 32-битных, вероятно `float`, и 64-битных, вероятно `double`, чисел. Но `pack754()` можно вызывать и напрямую, задав количество кодируемых битов `bits`, из которых `expbits` отводится под показатель нормализованного числа.

Пример использования:

```
#include <stdio.h>
#include <stdint.h> // defines uintN_t types
#include <inttypes.h> // defines PRIx macros

int main(void)
{
   float f = 3.1415926, f2;
   double d = 3.14159265358979323, d2;
  uint32_t fi;
  uint64_t di;

  fi = pack754_32(f);
  f2 = unpack754_32(fi);

  di = pack754_64(d);
  d2 = unpack754_64(di);

  printf("float before : %.7f\n", f);
  printf("float encoded: 0x%08" PRIx32 "\n", fi);
  printf("float after : %.7f\n\n", f2);

  printf("double before : %.20lf\n", d);
  printf("double encoded: 0x%016" PRIx64 "\n", di);
  printf("double after : %.20lf\n", d2);

  return 0;
}
```

Этот код выводит:

```
float before : 3.1415925
float encoded: 0x40490FDA
float after : 3.1415925
double before : 3.14159265358979311600
double encoded: 0x400921FB54442D18
double after : 3.14159265358979311600
```

Ещё один вопрос: как упаковывать структуры? К сожалению, компилятор волен вставлять в структуру дополняющие байты где угодно, поэтому переносимо отправить её целиком одним блоком нельзя. Вам ещё не надоело слышать «это нельзя», «то нельзя»? Простите! Как сказал мой друг: «Когда что-то идёт не так, я всегда виню Microsoft». Возможно, здесь Microsoft и не виновата, но утверждение моего друга совершенно верно.

Вернёмся к делу: лучший способ отправить структуру — упаковать каждое поле отдельно, а на другой стороне распаковать поля обратно в структуру.

«Это же куча работы», — думаете вы. Да. Можно написать вспомогательную функцию для упаковки данных. Будет весело! Честно!

В книге The Practice of Programming Кернигана и Пайка реализованы похожие на `printf()` функции `pack()` и `unpack()`, которые делают именно это. Я бы дал ссылку на них, но, похоже, их нет среди остальных исходников книги в интернете.

The Practice of Programming — превосходное чтение. Каждый раз, когда я её рекомендую, Зевс спасает котёнка.

Здесь я оставлю ссылку на реализацию Protocol Buffers для C, которой никогда не пользовался, но которая выглядит вполне достойно. Программистам на Python и Perl стоит изучить функции `pack()` и `unpack()` своих языков для решения той же задачи. А в Java есть большой интерфейс `Serializable`, который можно использовать похожим образом.

Если же вы хотите написать собственную утилиту упаковки на C, приём Кернигана и Пайка заключается в использовании списков переменных аргументов для создания похожих на `printf()` функций, формирующих пакеты. Вот версия, которую я придумал на этой основе и которая, надеюсь, даст вам представление о работе такого механизма.

Этот код использует описанные выше функции `pack754()`. Функции `packi*()` работают как знакомое семейство `htons()`, но упаковывают данные в массив `char`, а не в другое целое число.

```
#include <stdio.h>
#include <ctype.h>
#include <stdarg.h>
#include <string.h>

/*
** packi16() -- store a 16-bit int into a char buffer (like htons())
*/
void packi16(unsigned char *buf, unsigned int i)
{
  *buf++ = i>>8; *buf++ = i;
}

/*
** packi32() -- store a 32-bit int into a char buffer (like htonl())
*/
void packi32(unsigned char *buf, unsigned long int i)
{
  *buf++ = i>>24; *buf++ = i>>16;
  *buf++ = i>>8; *buf++ = i;
}

/*
** packi64() -- store a 64-bit int into a char buffer (like htonl())
*/
void packi64(unsigned char *buf, unsigned long long int i)
{
  *buf++ = i>>56; *buf++ = i>>48;
  *buf++ = i>>40; *buf++ = i>>32;
  *buf++ = i>>24; *buf++ = i>>16;
  *buf++ = i>>8; *buf++ = i;
}

/*
** unpacki16() -- unpack a 16-bit int from a char buffer (like
**         ntohs())
*/
int unpacki16(unsigned char *buf)
{
  unsigned int i2 = ((unsigned int)buf[0]<<8) | buf[1];
  int i;

  // change unsigned numbers to signed
  if (i2 <= 0x7fffu) { i = i2; }
  else { i = -1 - (unsigned int)(0xffffu - i2); }

  return i;
}
```

```
/*
** unpacku16() -- unpack a 16-bit unsigned from a char buffer (like
**         ntohs())
*/
unsigned int unpacku16(unsigned char *buf)
{
  return ((unsigned int)buf[0]<<8) | buf[1];
}

/*
** unpacki32() -- unpack a 32-bit int from a char buffer (like
**         ntohl())
*/
long int unpacki32(unsigned char *buf)
{
  unsigned long int i2 = ((unsigned long int)buf[0]<<24) |
                ((unsigned long int)buf[1]<<16) |
                ((unsigned long int)buf[2]<<8) |
                buf[3];
  long int i;

  // change unsigned numbers to signed
  if (i2 <= 0x7fffffffu) { i = i2; }
  else { i = -1 - (long int)(0xffffffffu - i2); }

  return i;
}

/*
** unpacku32() -- unpack a 32-bit unsigned from a char buffer (like
**         ntohl())
*/
unsigned long int unpacku32(unsigned char *buf)
{
  return ((unsigned long int)buf[0]<<24) |
      ((unsigned long int)buf[1]<<16) |
      ((unsigned long int)buf[2]<<8) |
      buf[3];
}

/*
** unpacki64() -- unpack a 64-bit int from a char buffer (like
**         ntohl())
*/
long long int unpacki64(unsigned char *buf)
{
  unsigned long long int i2 =
    ((unsigned long long int)buf[0]<<56) |
    ((unsigned long long int)buf[1]<<48) |
    ((unsigned long long int)buf[2]<<40) |
   ((unsigned long long int)buf[3]<<32) |
   ((unsigned long long int)buf[4]<<24) |
   ((unsigned long long int)buf[5]<<16) |
   ((unsigned long long int)buf[6]<<8) |
   buf[7];
 long long int i;

 // change unsigned numbers to signed
 if (i2 <= 0x7fffffffffffffffu) { i = i2; }
 else { i = -1 -(long long int)(0xffffffffffffffffu - i2); }

 return i;
}

/*
** unpacku64() -- unpack a 64-bit unsigned from a char buffer (like
```

```
**        ntohl())
*/
unsigned long long int unpacku64(unsigned char *buf)
{
 return ((unsigned long long int)buf[0]<<56) |
     ((unsigned long long int)buf[1]<<48) |
     ((unsigned long long int)buf[2]<<40) |
     ((unsigned long long int)buf[3]<<32) |
     ((unsigned long long int)buf[4]<<24) |
     ((unsigned long long int)buf[5]<<16) |
     ((unsigned long long int)buf[6]<<8) |
     buf[7];
}

/*
** pack() -- store data dictated by the format string in the buffer
**
** bits |signed unsigned float string
** -----+----------------------------------
** 8 | c     C
** 16 | h    H      f
** 32 | l    L      d
** 64 | q    Q      g
** - |                   s
**
** (16-bit unsigned length is automatically prepended to strings)
*/

unsigned int pack(unsigned char *buf, char *format, ...)
{
 va_list ap;

 signed char c;    // 8-bit
 unsigned char C;

 int h;            // 16-bit
 unsigned int H;

 long int l;       // 32-bit
 unsigned long int L;

 long long int q;  // 64-bit
 unsigned long long int Q;

 float f;          // floats
 double d;
 long double g;
 unsigned long long int fhold;

 char *s;          // strings
 unsigned int len;

 unsigned int size = 0;

 va_start(ap, format);

 for(; *format != '\0'; format++) {
   switch(*format) {
   case 'c': // 8-bit
      size += 1;
      c = (signed char)va_arg(ap, int); // promoted
      *buf++ = c;
      break;

   case 'C': // 8-bit unsigned
      size += 1;
```

```
      C = (unsigned char)va_arg(ap, unsigned int); // promoted
      *buf++ = C;
      break;

   case 'h': // 16-bit
      size += 2;
      h = va_arg(ap, int);
      packi16(buf, h);
      buf += 2;
      break;

   case 'H': // 16-bit unsigned
      size += 2;
      H = va_arg(ap, unsigned int);
      packi16(buf, H);
      buf += 2;
      break;

   case 'l': // 32-bit
      size += 4;
      l = va_arg(ap, long int);
      packi32(buf, l);
      buf += 4;
      break;

   case 'L': // 32-bit unsigned
      size += 4;
      L = va_arg(ap, unsigned long int);
      packi32(buf, L);
      buf += 4;
      break;

   case 'q': // 64-bit
      size += 8;
      q = va_arg(ap, long long int);
      packi64(buf, q);
      buf += 8;
      break;

   case 'Q': // 64-bit unsigned
      size += 8;
      Q = va_arg(ap, unsigned long long int);
      packi64(buf, Q);
      buf += 8;
      break;

   case 'f': // float-16
      size += 2;
      f = (float)va_arg(ap, double); // promoted
      fhold = pack754_16(f); // convert to IEEE 754
      packi16(buf, fhold);
      buf += 2;
      break;

   case 'd': // float-32
      size += 4;
      d = va_arg(ap, double);
      fhold = pack754_32(d); // convert to IEEE 754
      packi32(buf, fhold);
      buf += 4;
      break;

   case 'g': // float-64
      size += 8;
      g = va_arg(ap, long double);
      fhold = pack754_64(g); // convert to IEEE 754
```

```
      packi64(buf, fhold);
      buf += 8;
      break;

   case 's': // string
      s = va_arg(ap, char*);
      len = strlen(s);
      size += len + 2;
      packi16(buf, len);
      buf += 2;
      memcpy(buf, s, len);
      buf += len;
      break;
   }
 }

 va_end(ap);

 return size;
}

/*
** unpack() -- unpack data dictated by the format string into the
**      buffer
**
** bits |signed unsigned float string
** -----+----------------------------------
** 8 | c     C
** 16 | h    H      f
** 32 | l    L      d
** 64 | q    Q      g
** - |                   s
**
** (string is extracted based on its stored length, but 's' can be
** prepended with a max length)
*/
void unpack(unsigned char *buf, char *format, ...)
{
 va_list ap;

 signed char *c;    // 8-bit
 unsigned char *C;

 int *h;            // 16-bit
 unsigned int *H;

 long int *l;       // 32-bit
 unsigned long int *L;

 long long int *q;  // 64-bit
 unsigned long long int *Q;

 float *f;          // floats
 double *d;
 long double *g;
 unsigned long long int fhold;

 char *s;
 unsigned int len, maxstrlen=0, count;

 va_start(ap, format);

 for(; *format != '\0'; format++) {
   switch(*format) {
   case 'c': // 8-bit
      c = va_arg(ap, signed char*);
```

```
      if (*buf <= 0x7f) { *c = *buf;} // re-sign
      else { *c = -1 - (unsigned char)(0xffu - *buf); }
      buf++;
      break;

   case 'C': // 8-bit unsigned
      C = va_arg(ap, unsigned char*);
      *C = *buf++;
      break;

   case 'h': // 16-bit
      h = va_arg(ap, int*);
      *h = unpacki16(buf);
      buf += 2;
      break;

   case 'H': // 16-bit unsigned
      H = va_arg(ap, unsigned int*);
      *H = unpacku16(buf);
      buf += 2;
      break;

   case 'l': // 32-bit
      l = va_arg(ap, long int*);
      *l = unpacki32(buf);
      buf += 4;
      break;

   case 'L': // 32-bit unsigned
      L = va_arg(ap, unsigned long int*);
      *L = unpacku32(buf);
      buf += 4;
      break;

   case 'q': // 64-bit
      q = va_arg(ap, long long int*);
      *q = unpacki64(buf);
      buf += 8;
      break;

   case 'Q': // 64-bit unsigned
      Q = va_arg(ap, unsigned long long int*);
      *Q = unpacku64(buf);
      buf += 8;
      break;

   case 'f': // float
      f = va_arg(ap, float*);
      fhold = unpacku16(buf);
      *f = unpack754_16(fhold);
      buf += 2;
      break;

   case 'd': // float-32
      d = va_arg(ap, double*);
      fhold = unpacku32(buf);
      *d = unpack754_32(fhold);
      buf += 4;
      break;

   case 'g': // float-64
      g = va_arg(ap, long double*);
      fhold = unpacku64(buf);
      *g = unpack754_64(fhold);
      buf += 8;
      break;
```

```
   case 's': // string
      s = va_arg(ap, char*);
      len = unpacku16(buf);
      buf += 2;
      if (maxstrlen > 0 && len > maxstrlen)
         count = maxstrlen - 1;
      else
         count = len;
      memcpy(s, buf, count);
      s[count] = '\0';
      buf += len;
      break;

   default:
      if (isdigit(*format)) { // track max str len
         maxstrlen = maxstrlen * 10 + (*format-'0');
      }
   }

   if (!isdigit(*format)) maxstrlen = 0;
 }

 va_end(ap);
}
```

А вот [демонстрационная программа](), которая упаковывает данные в `buf`, а затем распаковывает их в переменные. При вызове `unpack()` со строковым аргументом, спецификатор формата `s`, разумно указать перед ним максимальную длину, например `96s`, чтобы избежать переполнения буфера. Осторожно распаковывайте данные, полученные по сети: злоумышленник может отправить специально повреждённые пакеты для атаки на систему!

```
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>

// If you have a C23 compiler
#if __STDC_VERSION__ >= 202311L
#include <stdfloat.h>
#else
// Otherwise let's define our own.
// Varies for different architectures! But you're probably:
typedef float float32_t;
typedef double float64_t;
#endif

int main(void)
{
  uint8_t buf[1024];
  int8_t magic;
  int16_t monkeycount;
  int32_t altitude;
  float32_t absurdityfactor;
  char *s = "Great unmitigated Zot! You've found the Runestaff!";
  char s2[96];
  int16_t packetsize, ps2;

  packetsize = pack(buf, "chhlsf", (int8_t)'B', (int16_t)0,
       (int16_t)37, (int32_t)-5, s, (float32_t)-3490.6677);
  packi16(buf+1, packetsize); // store packet size for kicks

  printf("packet is %" PRId32 " bytes\n", packetsize);
```

```
  unpack(buf, "chhl96sf", &magic, &ps2, &monkeycount, &altitude,
       s2, &absurdityfactor);

  printf("'%c' %" PRId32" %" PRId16 " %" PRId32
       " \"%s\" %f\n", magic, ps2, monkeycount,
       altitude, s2, absurdityfactor);
}
```

Пишете ли вы собственные функции или используете чужие, полезно иметь общий набор процедур упаковки данных, чтобы уменьшить количество ошибок, вместо того чтобы каждый раз упаковывать каждый бит вручную.

Какой формат лучше использовать для упаковки? Отличный вопрос. К счастью, RFC 4506, стандарт внешнего представления данных XDR, уже определяет двоичные форматы множества типов: чисел с плавающей точкой, целых чисел, массивов, необработанных данных и других. Если вы собираетесь реализовать упаковку самостоятельно, советую придерживаться этого стандарта. Но вы не обязаны: полиция пакетов не стоит у вашей двери. По крайней мере, мне так кажется.

В любом случае перед отправкой данные следует каким-либо образом закодировать — это правильный подход!

7.6. Возвращение инкапсуляции данных

Что вообще значит инкапсулировать данные? В простейшем случае это значит добавить заголовок с идентифицирующей информацией, длиной пакета или тем и другим.

Как должен выглядеть заголовок? Это просто двоичные данные, представляющие всё, что вы считаете необходимым для своего проекта.

Ух ты. Очень расплывчато.

Хорошо. Допустим, у вас есть многопользовательский чат, использующий `SOCK_STREAM`. Когда пользователь что-то печатает («говорит»), серверу нужно передать две вещи: что было сказано и кто это сказал.

Пока всё понятно? «В чём проблема?» — спросите вы.

Проблема в том, что сообщения могут иметь разную длину. Пользователь по имени `tom` может сказать `Hi`, а пользователь `Benjamin` — `Hey guys what is up?`.

Вы отправляете всё это клиентам через `send()` по мере поступления. Исходящий поток данных выглядит так:

```
tomHiBenjaminHey guys what is up?
```

И так далее. Как клиенту узнать, где начинается одно сообщение и заканчивается другое? Конечно, можно сделать все сообщения одинаковой длины и просто вызвать реализованную выше `sendall()`. Но это расходует пропускную способность! Мы не хотим отправлять 1024 байта только для того, чтобы `tom` сказал `Hi`.

Поэтому мы инкапсулируем данные, используя небольшой заголовок и структуру пакета. И клиент, и сервер знают, как упаковывать и распаковывать эти данные; эти операции также называют маршалингом и демаршалингом. Сами того не замечая, мы начинаем определять протокол, описывающий общение клиента с сервером!

Допустим, имя пользователя имеет фиксированную длину 8 символов и дополняется символами `\0`. А данные имеют переменную длину, не более 128 символов. Посмотрим на пример структуры пакета для такой ситуации:

1. `len` (1 байт, без знака) — общая длина пакета, считая восьмибайтовое имя и данные чата.
2. `name` (8 байтов) — имя пользователя, при необходимости дополненное нулевыми байтами.
3. `chatdata` (n байтов) — сами данные, не более 128 байтов. Длина пакета вычисляется как длина этих данных плюс 8 — длина поля имени.

Почему я выбрал ограничения в 8 и 128 байтов? Просто взял с потолка, предположив, что их хватит. Возможно, для вас восьми байтов слишком мало и вы захотите поле имени в 30 байтов или другое. Решать вам.

Согласно приведённому определению, первый пакет будет содержать следующие данные в шестнадцатеричном виде и ASCII:

```
0A 74 6F 6D 00 00 00 00 00 48 69
   t  o  m  [дополнение]    H  i
```

Первый байт — длина. Второй пакет аналогичен:

```
18 42 65 6E 6A 61 6D 69 6E 48 65 79 20 67 75 79 73 20 77 ...
   B  e  n  j  a  m  i  n  H  e  y     g  u  y  s     w ...
```

Разумеется, длина хранится в сетевом порядке байтов. Здесь это всего один байт и порядок не имеет значения, но вообще в пакетах лучше хранить все двоичные целые числа в сетевом порядке.

При отправке этих данных для надёжности следует использовать что-то вроде приведённой выше `sendall()`, чтобы все данные были отправлены, даже если потребуется несколько вызовов `send()`.

При получении тоже придётся немного потрудиться. Для надёжности предполагайте, что можете получить только часть пакета: например, от Benjamin одним вызовом `recv()` пришло `18 42 65 6E 6A` и больше ничего. Нужно вызывать `recv()` снова и снова, пока пакет не будет получен целиком.

Но как? Мы знаем, сколько всего байтов нужно получить для полного пакета, поскольку это число указано в его начале. Мы также знаем максимальный размер: 1 + 8 + 128, то есть 137 байтов, потому что сами так определили пакет.

Есть несколько подходов. Поскольку каждый пакет начинается с длины, можно вызвать `recv()` только для её получения. Затем, узнав длину, вызвать её снова, запросив ровно оставшееся количество байтов, при необходимости повторяя вызов до получения полного пакета. Преимущество в том, что нужен буфер лишь на один пакет; недостаток — `recv()` придётся вызвать как минимум дважды.

Другой вариант — вызвать `recv()`, указав максимальное число байтов пакета как объём, который вы готовы принять. Добавьте полученное в конец буфера и проверьте, завершён ли пакет. Конечно, может прийти и часть следующего пакета, поэтому для неё нужно предусмотреть место.

Можно объявить массив размером на два пакета. Это будет рабочий массив, в котором вы собираете пакеты по мере поступления.

Каждый раз после `recv()` добавляйте данные в рабочий буфер и проверяйте полноту пакета: число байтов в буфере должно быть не меньше указанной в заголовке длины плюс один, поскольку сама длина не

включает байт, в котором хранится. Если в буфере меньше одного байта, пакет, очевидно, неполон. Этот случай нужно обработать отдельно: первый байт ещё содержит мусор, и полагаться на него для определения длины нельзя.

Когда пакет готов, можно делать с ним всё необходимое. Используйте его и удалите из рабочего буфера.

Уф! Уже удерживаете всё это в голове? Вот второй удар этой комбинации: один вызов `recv()` мог прочитать данные за концом одного пакета и захватить начало следующего. То есть в рабочем буфере у вас полный пакет и неполная часть следующего! Чёрт возьми. Именно поэтому буфер и был рассчитан на два пакета — на случай такой ситуации.

Поскольку длина первого пакета известна из заголовка и вы следили за числом байтов в рабочем буфере, можно вычитанием определить, сколько байтов принадлежит второму, неполному пакету. Обработав первый пакет, удалите его из рабочего буфера и переместите неполный второй пакет в начало — всё будет готово к следующему `recv()`.

Некоторые читатели заметят, что перемещение неполного второго пакета в начало рабочего буфера занимает время и что можно обойтись без него с помощью кольцевого буфера. К сожалению для остальных, обсуждение кольцевых буферов выходит за рамки этой статьи. Если вам интересно, возьмите книгу по структурам данных и начните с неё.

Я никогда не говорил, что это просто. Ладно, говорил. И это действительно просто: нужна практика, и вскоре всё станет для вас естественным. Клянусь Экскалибуром!

7.7. Широковещательные пакеты — привет, мир!

До сих пор в этом руководстве речь шла об отправке данных от одного узла другому. Но уверяю вас: при наличии соответствующих полномочий можно отправить данные сразу нескольким узлам!

В UDP — только в UDP, не в TCP — при использовании стандартного IPv4 это делается с помощью механизма широковещательной рассылки, или broadcasting. IPv6 её не поддерживает; вместо этого придётся прибегнуть к часто более удобной групповой рассылке, multicasting, которую я, увы, пока не буду обсуждать. Но хватит смотреть на будущее широко раскрытыми глазами — мы застряли в 32-битном настоящем.

Погодите! Нельзя просто так начать рассылать пакеты куда попало: перед отправкой широковещательного пакета в сеть нужно установить параметр сокета `SO_BROADCAST`. Это как пластиковая крышка над кнопкой запуска ракеты! Вот какая мощь у вас в руках!

А если серьёзно, широковещательные пакеты связаны с опасностью: каждая система, получившая такой пакет, должна снять все луковые слои инкапсуляции, прежде чем выяснить порт назначения. Затем она передаёт данные дальше или отбрасывает. В любом случае каждой получившей пакет машине приходится немало потрудиться, а получают его все машины локальной сети — значит, множество компьютеров может выполнить много ненужной работы. Когда впервые вышла игра Doom, на её сетевой код жаловались именно по этой причине.

Как говорится, есть больше одного способа освежевать кошку… минутку. Неужели больше одного? Что это вообще за выражение?[1] Как бы то ни было, широковещательный пакет тоже можно отправить несколькими способами. Итак, по существу: как указать адрес назначения широковещательного сообщения? Есть два распространённых варианта:

1. Отправить данные на широковещательный адрес конкретной подсети. Это номер сети подсети, у которого все биты части адреса, отвечающей за узел, установлены в единицы. Например, дома моя сеть — `192.168.1.0`, маска — `255.255.255.0`, поэтому последний байт адреса — номер узла, а первые три, согласно маске, — номер сети. Значит, широковещательный адрес — `192.168.1.255`. В Unix команда `ifconfig` покажет все эти данные. Если интересно, побитовое выражение для получения адреса: `номер_сети OR (NOT маска_сети)`. Такой широковещательный пакет можно послать и в удалённую сеть, но есть риск, что маршрутизатор назначения его отбросит. Иначе какой-нибудь случайный нарушитель мог бы засыпать локальную сеть широковещательным трафиком.
2. Отправить данные на «глобальный» широковещательный адрес `255.255.255.255`, также известный как `INADDR_BROADCAST`. Многие машины автоматически выполняют побитовое И этого адреса с номером вашей сети, преобразуя его в сетевой широковещательный адрес, но не все: поведение различается. Маршрутизаторы не пересылают такие пакеты за пределы локальной сети.

[1] Для ясности: я люблю кошек. Они прекрасны, и за годы у меня было много любимых кошачьих спутников. Хотя я признаю, что некоторым не нравится это мрачное образное выражение, происхождение которого затерялось во времени, я считаю его уместным в этом месте руководства.

Что произойдёт, если попробовать отправить данные на широковещательный адрес, не установив предварительно параметр `SO_BROADCAST`? Запустим старые добрые `talker` и `listener` и посмотрим:

```
$ talker 192.168.1.2 foo
sent 3 bytes to 192.168.1.2
$ talker 192.168.1.255 foo
sendto: Permission denied
$ talker 255.255.255.255 foo
sendto: Permission denied
```

Да, ему совсем не нравится отсутствие параметра `SO_BROADCAST`. Установите его — и сможете вызывать `sendto()` куда захотите!

По сути, это единственное отличие UDP-приложения, которое может делать широковещательную рассылку, от того, которое не может. Возьмём старый `talker` и добавим участок, устанавливающий `SO_BROADCAST`. Назовём программу broadcaster.c:

```
/*
** broadcaster.c -- a datagram "client" like talker.c, except
**           this one can broadcast
*/

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>

#define SERVERPORT 4950 // the port users will be connecting to

int main(int argc, char *argv[])
{
  int sockfd;
  struct sockaddr_in their_addr; // connector's address info
```

```
  struct hostent *he;
  int numbytes;
  int broadcast = 1;
  //char broadcast = '1'; // if that doesn't work, try this

  if (argc != 3) {
    fprintf(stderr,"usage: broadcaster hostname message\n");
    exit(1);
  }

  if ((he=gethostbyname(argv[1])) == NULL) { // get the host info
    perror("gethostbyname");
    exit(1);
  }

  if ((sockfd = socket(PF_INET, SOCK_DGRAM, 0)) == -1) {
    perror("socket");
    exit(1);
  }

  // this call is what allows broadcast packets to be sent:
  if (setsockopt(sockfd, SOL_SOCKET, SO_BROADCAST, &broadcast,
    sizeof broadcast) == -1) {
    perror("setsockopt (SO_BROADCAST)");
    exit(1);
  }

  their_addr.sin_family = AF_INET; // host byte order
  their_addr.sin_port = htons(SERVERPORT); // network byte order
  their_addr.sin_addr = *((struct in_addr *)he->h_addr);
  memset(their_addr.sin_zero, '\0', sizeof their_addr.sin_zero);

  numbytes = sendto(sockfd, argv[2], strlen(argv[2]), 0,
        (struct sockaddr *)&their_addr, sizeof their_addr);

  if (numbytes == -1) {
    perror("sendto");
    exit(1);
  }

  printf("sent %d bytes to %s\n", numbytes,
    inet_ntoa(their_addr.sin_addr));

  close(sockfd);

  return 0;
}
```

Чем это отличается от «обычной» ситуации с UDP-клиентом и сервером? Ничем! Если не считать того, что теперь клиенту разрешено отправлять широковещательные пакеты. Поэтому запустите старый UDP-`listener` в одном окне, а `broadcaster` — в другом. Теперь должны получиться все отправки, которые выше завершились ошибкой:

```
$ broadcaster 192.168.1.2 foo
sent 3 bytes to 192.168.1.2
$ broadcaster 192.168.1.255 foo
sent 3 bytes to 192.168.1.255
$ broadcaster 255.255.255.255 foo
sent 3 bytes to 255.255.255.255
```

`listener` должен сообщить, что получил пакеты. Если он не отвечает, возможно, он привязан к IPv6-адресу. Попробуйте заменить `AF_INET6` на `AF_INET` в `listener.c`, чтобы принудительно использовать IPv4.

Довольно интересно. А теперь запустите `listener` на другой машине в той же сети, чтобы работали две копии, по одной на каждой машине, и снова запустите `broadcaster` с широковещательным адресом. Эй! Оба `listener` получили пакет, хотя `sendto()` был вызван всего один раз! Здорово!

Если `listener` получает данные, адресованные ему напрямую, но не получает отправленные на широковещательный адрес, возможно, их блокирует межсетевой экран вашей машины. Да, Пэт и Бэппер, спасибо, что раньше меня поняли, почему мой пример не работал. Я обещал упомянуть вас в руководстве — вот вы и здесь. Так-то.

И снова: осторожнее с широковещательными пакетами. Каждая машина локальной сети будет вынуждена обработать пакет, независимо от того, получит ли его через `recvfrom()`, поэтому такие пакеты могут создать существенную нагрузку на всю сеть. Пользоваться ими определённо следует умеренно и по делу.

Глава 8. Частые вопросы

Где взять эти заголовочные файлы?

Если их ещё нет в вашей системе, вероятно, они вам и не нужны. Обратитесь к руководству для вашей платформы. При сборке под Windows достаточно `#include <winsock.h>`.

Что делать, когда bind() сообщает «Address already in use»?

Для слушающего сокета нужно вызвать `setsockopt()` с параметром `SO_REUSEADDR`. Примеры приведены в разделах о `bind()` и `select()`.

Как получить список открытых сокетов?

Используйте `netstat`. Подробности смотрите в странице руководства; полезную информацию можно получить и просто набрав:

```
netstat
```

Осталось только выяснить, какому приложению принадлежит каждый сокет. :-)

Как посмотреть таблицу маршрутизации?

Выполните `route` (в большинстве систем Linux она находится в `/sbin`), `netstat -r` или `ip route`.

Как запустить клиент и сервер, если у меня только один компьютер? Разве для сетевых программ не нужна сеть?

К счастью, почти все машины реализуют петлевое сетевое «устройство» внутри ядра, которое притворяется сетевой картой. В таблице маршрутизации этот интерфейс обозначается `lo`.

Представим, что вы вошли на машину с именем `goat`. Запустите клиент в одном окне, сервер — в другом. Или запустите сервер в фоновом режиме (`server &`), а затем клиент в том же окне. Благодаря петлевому устройству можно выполнить `client goat` или `client localhost` (имя `localhost`, вероятно, определено в `/etc/hosts`), и клиент будет общаться с сервером без сети!

Иными словами, чтобы программы работали на одной машине без сетевого подключения, изменять код не нужно. Ура!

Как узнать, что другая сторона закрыла соединение?

Вызов `recv()` вернёт `0`.

Как написать утилиту ping? Что такое ICMP? Где почитать про сырые сокеты и SOCK_RAW?

Ответы на вопросы о сырых сокетах есть в книгах У. Ричарда Стивенса *UNIX Network Programming*. Загляните также в каталог `ping/` исходного кода к этим книгам.

Как изменить или сократить время ожидания connect()?

Вместо того чтобы повторять ответ У. Ричарда Стивенса, отсылаю вас к файлу `lib/connect_nonb.c` в исходном коде UNIX Network Programming.

Суть такова: создайте дескриптор через `socket()`, установите неблокирующий режим и вызовите `connect()`. Если всё идёт как надо, `connect()` сразу вернёт `-1`, а `errno` будет равен `EINPROGRESS`. Затем вызовите `select()` с нужным временем ожидания, передав дескриптор сокета и в набор чтения, и в набор записи. Если время ожидания не истекло, вызов `connect()` завершился. Теперь с помощью `getsockopt()` и параметра `SO_ERROR` получите результат `connect()`: при отсутствии ошибки он должен быть равен нулю.

Прежде чем передавать данные, вы, вероятно, захотите вернуть сокет в блокирующий режим. Дополнительное преимущество этого подхода — программа может заниматься другими делами во время подключения. Например, можно установить небольшое время ожидания, скажем 500 мс, после каждого истечения обновлять индикатор на экране и снова вызывать `select()`. После, допустим, 20 истечений времени станет ясно, что пора отказаться от подключения. Отличный пример есть в исходном коде Стивенса.

Как собрать программу под Windows?

Сначала удалите Windows и установите Linux или BSD. };-) Шутка: обратитесь к разделу о сборке под Windows во введении.

Как собрать программу под Solaris/SunOS? При компиляции постоянно появляются ошибки компоновщика!

Это происходит потому, что на машинах Sun библиотеки сокетов не подключаются автоматически. Пример есть в разделе о Solaris/SunOS во введении.

Почему select() постоянно прерывается сигналом?

Сигналы обычно заставляют заблокированные системные вызовы возвращать `-1`, устанавливая `errno` в `EINTR`. При установке обработчика с помощью `sigaction()` можно задать флаг `SA_RESTART`, который должен перезапустить системный вызов после прерывания. Естественно, это работает не всегда.

Моё любимое решение использует `goto`. Преподавателей это ужасно раздражает — так что вперёд!

```
select_restart:
if ((err = select(fdmax+1, &readfds, NULL, NULL, NULL)) == -1) {
    if (errno == EINTR) {
        // Нас прервал сигнал, поэтому запускаем снова
        goto select_restart;
```

```
    }
    // Здесь обрабатываем настоящую ошибку:
    perror("select");
}
```

Конечно, здесь можно обойтись без `goto`, используя другие управляющие конструкции. Но, по-моему, с `goto` код получается чище.

Как задать время ожидания для recv()?

Используйте `select()`! Он позволяет задать время ожидания для дескрипторов сокетов, из которых вы собираетесь читать. Можно также объединить эту функциональность в одной функции:

```
#include <unistd.h>
#include <sys/time.h>
#include <sys/types.h>
#include <sys/socket.h>

int recvtimeout(int s, char *buf, int len, int timeout)
{
    fd_set fds;
    int n;
    struct timeval tv;

    // Подготавливаем набор дескрипторов
    FD_ZERO(&fds);
    FD_SET(s, &fds);

    // Подготавливаем время ожидания
    tv.tv_sec = timeout;
    tv.tv_usec = 0;

    // Ждём данные или истечение времени
    n = select(s+1, &fds, NULL, NULL, &tv);
    if (n == 0) return -2; // время истекло!
    if (n == -1) return -1; // ошибка

    // Данные должны быть готовы, выполняем обычный recv()
    return recv(s, buf, len, 0);
}
...

// Пример вызова recvtimeout():
n = recvtimeout(s, buf, sizeof buf, 10); // ожидание 10 секунд

if (n == -1) {
    // произошла ошибка
    perror("recvtimeout");
} else if (n == -2) {
    // истекло время ожидания
} else {
    // получили данные в buf
}
...
```

При истечении времени `recvtimeout()` возвращает `-2`. Почему не `0`? Вспомните: ноль, возвращённый `recv()`, означает, что удалённая сторона закрыла соединение. Это значение уже занято, а `-1` означает ошибку, поэтому для истечения времени я выбрал `-2`.

Как зашифровать или сжать данные перед отправкой через сокет?

Один простой способ шифрования — SSL (secure sockets layer), но это выходит за рамки руководства. Дополнительную информацию см. в проекте OpenSSL.

Если же вы хотите подключить или реализовать собственную систему сжатия или шифрования, представьте, что данные проходят последовательность этапов между двумя сторонами. Каждый этап как-то меняет данные:

1. Сервер читает данные из файла или другого источника.
2. Сервер шифрует или сжимает данные (эту часть добавляете вы).
3. Сервер отправляет обработанные данные через `send()`.

На другой стороне выполняется обратная последовательность:

1. Клиент получает данные через `recv()`.
2. Клиент расшифровывает или распаковывает их (эту часть добавляете вы).
3. Клиент записывает данные в файл или куда-либо ещё.

Если используете и сжатие, и шифрование, сначала сжимайте. :-) Пока клиент правильно обращает операции сервера, данные в конце будут корректными, сколько бы промежуточных этапов вы ни добавили. Чтобы использовать мой код, найдите место между чтением данных и их отправкой по сети через `send()` и вставьте туда код шифрования.

Что за PF_INET мне всё время встречается? Оно связано с AF_INET?

Да, связано. Подробности — в разделе о `socket()`.

Как написать сервер, принимающий от клиента команды оболочки и выполняющий их?

Для простоты предположим, что клиент вызывает `connect()`, `send()` и `close()`, то есть никаких последующих системных вызовов без нового подключения не делает.

Действия клиента:

1. Подключиться к серверу через `connect()`.
2. Отправить `send("/sbin/ls > /tmp/client.out")`.
3. Закрыть соединение через `close()`.

Тем временем сервер обрабатывает данные и исполняет команду:

1. Принять соединение через `accept()`.
2. Получить строку команды через `recv(str)`.
3. Закрыть соединение через `close()`.
4. Выполнить команду через `system(str)`.

Осторожно! Сервер, исполняющий команды клиента, фактически предоставляет удалённый доступ к оболочке: подключившиеся люди смогут выполнять действия от вашей учётной записи. Например, что будет, если клиент отправит `rm -rf ~`? Правильно: всё содержимое вашей учётной записи будет удалено!

Вы можете попытаться разрешить клиенту только несколько заведомо безопасных утилит, например `foobar`:

```
if (!strncmp(str, "foobar", 6)) {
    sprintf(sysstr, "%s > /tmp/server.out", str);
    system(sysstr);
}
```

Но это всё ещё небезопасно: что, если клиент введёт `foobar; rm -rf ~`? Автор рекомендует написать функцию, которая добавляет символ экранирования (\) перед всеми небуквенно-цифровыми символами в аргументах команды, в том числе перед пробелами, если это уместно. Как видите, безопасность становится серьёзной проблемой, когда сервер исполняет то, что присылает клиент.

> Примечание переводчика: экранирование само по себе не гарантирует безопасность для всех оболочек и ситуаций. Предпочтительно вообще не запускать оболочку: проверяйте допустимые команды и аргументы и передавайте их программе через API запуска процесса отдельными аргументами.

Я отправляю много данных, но recv() получает за раз только 536 или 1460 байт. На локальной машине данные приходят целиком. Почему?

Вы упираетесь в MTU — максимальный размер, поддерживаемый физической средой. На локальной машине петлевое устройство без проблем обрабатывает 8 Кбайт и больше. Но Ethernet с заголовком ограничен 1500 байтами, а соединение через модем с MTU 576 — ещё меньшим размером.

Прежде всего убедитесь, что отправлены все данные (см. реализацию `sendall()`). Затем вызывайте `recv()` в цикле, пока не прочитаете всё. О получении целых пакетов данных за несколько вызовов `recv()` см. раздел «Продолжение инкапсуляции данных».

В Windows нет системного вызова fork() и структуры sigaction. Что делать?

Если они где-то и есть, то в библиотеках POSIX, которые могли поставляться с вашим компилятором. У меня нет машины с Windows, поэтому точно ответить не могу, но, насколько помню, у Microsoft есть слой совместимости с POSIX, где может быть `fork()` (а возможно, и `sigaction`). Поищите в справке VC++ слова «fork» или «POSIX».

Если не поможет, откажитесь от `fork()` и `sigaction` и замените их эквивалентом Win32 — `CreateProcess()`. Я не знаю, как пользоваться `CreateProcess()`: у него миллион аргументов, но он должен быть описан в документации VC++.

Я за межсетевым экраном. Как сообщить внешним пользователям свой IP-адрес, чтобы они могли подключиться ко мне?

Назначение межсетевого экрана — не пускать внешние подключения к машинам внутри сети, поэтому такое подключение по сути считается нарушением безопасности. Но не всё потеряно. Часто можно самому выполнить `connect()` через экран, если он использует маскарадинг, NAT или что-то подобное. Спроектируйте программы так, чтобы соединение всегда инициировали вы.

Если это не подходит, попросите системного администратора открыть доступ через экран. Он может перенаправлять соединения к вам через NAT, прокси или другим способом. Помните, что отверстие в межсетевом экране — серьёзное дело. Нужно следить, чтобы злоумышленники не получили доступ к внутренней сети; новичку сделать программу безопасной гораздо сложнее, чем может казаться. Не сердите на меня своего системного администратора. ;-)

Как написать анализатор пакетов? Как включить неразборчивый режим сетевого интерфейса Ethernet?

В неразборчивом режиме (*promiscuous mode*) сетевая карта передаёт операционной системе ВСЕ пакеты, а не только адресованные этой машине. Речь об адресах уровня Ethernet, а не IP; однако Ethernet находится ниже IP, поэтому фактически передаются пакеты со всеми IP-адресами. Подробнее см. раздел о низкоуровневых принципах и теории сетей.

На этом основана работа анализатора пакетов: интерфейс переводится в неразборчивый режим, после чего ОС получает каждый пакет, проходящий по сети. Полученные данные можно читать из сокета подходящего типа. Способ зависит от платформы; поиск по запросу `windows promiscuous ioctl` может помочь. Для Linux также есть подходящее обсуждение на Stack Overflow.

Как задать собственное время ожидания для TCP- или UDP-сокета?

Это зависит от системы. Поищите информацию о `SO_RCVTIMEO` и `SO_SNDTIMEO` для использования с `setsockopt()` и проверьте, поддерживает ли система эти возможности. Страница руководства Linux предлагает в качестве альтернативы `alarm()` или `setitimer()`.

Как узнать, какие порты свободны? Есть ли список официальных номеров портов?

Обычно это не проблема. Если вы пишете веб-сервер, разумно использовать общеизвестный порт 80. Для собственного специализированного сервера выберите случайный порт выше 1023 и попробуйте его.

Если порт уже занят, при вызове `bind()` появится ошибка «Address already in use». Выберите другой порт. Хорошая идея — позволить пользователю задать другой порт в конфигурационном файле или параметром командной строки.

Список официальных номеров портов ведёт IANA. То, что порт выше 1023 есть в списке, не означает, что его нельзя использовать. Например, DOOM от Id Software использует тот же порт, что и «mdqs», что бы это ни было. Важно лишь, чтобы в момент использования порт не был занят другой программой на той же машине.

Глава 9. Страницы руководства

В мире Unix много руководств. Они состоят из небольших разделов, описывающих отдельные доступные вам функции.

Конечно, `manual` было бы слишком долго набирать. Никто в мире Unix, включая меня, не любит столько печатать. Я мог бы ещё долго рассказывать о том, насколько предпочитаю краткость, но буду краток и не стану утомлять вас пространными рассуждениями о том, насколько поразительно кратким я предпочитаю быть практически во всех обстоятельствах без исключения.

[Аплодисменты]

Спасибо. Я веду к тому, что в мире Unix эти страницы называют *man pages*, и здесь для вашего удовольствия я привёл свои сокращённые варианты. Многие из этих функций гораздо универсальнее, чем я показываю, но я опишу лишь то, что относится к программированию интернет-сокетов.

Но подождите! Это ещё не все недостатки моих страниц:

- Они неполные и показывают только основы из руководства.
- В действительности страниц руководства намного больше.
- Они отличаются от страниц в вашей системе.
- Для некоторых функций в вашей системе могут использоваться другие заголовочные файлы.
- Параметры некоторых функций в вашей системе могут отличаться.

Если вам нужна настоящая информация, обратитесь к локальным страницам Unix, набрав `man что-нибудь`, где «что-нибудь» — нечто невероятно интересное для вас, например `accept`. (Уверен, в справке Microsoft Visual Studio есть что-то похожее. Но `man` лучше, потому что на целый байт короче, чем `help`. Unix снова победил!)

Если эти страницы так несовершенны, зачем они в руководстве? Причин несколько, но главные две: а) эти версии предназначены именно для сетевого программирования и усваиваются легче настоящих; б) в них есть примеры!

Кстати о примерах: я обычно не включаю в них всю проверку ошибок, поскольку она сильно увеличивает объём кода. Но вы обязательно должны проверять ошибки практически при каждом системном вызове, если только не уверены на все сто процентов, что он не может завершиться неудачей. Да и тогда, наверное, тоже стоит!

9.1 accept()

Принять входящее соединение на слушающем сокете.

Объявление

```
#include <sys/types.h>
#include <sys/socket.h>
int accept(int s, struct sockaddr *addr, socklen_t *addrlen);
```

Описание

После того как вы потрудились создать сокет `SOCK_STREAM` и подготовить его к входящим соединениям с помощью `listen()`, вызовите `accept()`: он создаст новый дескриптор для дальнейшего обмена с только что подключившимся клиентом. Прежний слушающий сокет остаётся на месте и будет использоваться для следующих вызовов `accept()`.

Параметр	Описание
`s`	Дескриптор слушающего сокета.
`addr`	Заполняется адресом подключающегося узла.
`addrlen`	Заполняется размером структуры, возвращённой в `addr`. Его можно проигнорировать, если вы предполагаете получить `struct sockaddr_in` и знаете это по типу, переданному в `addr`.

Обычно `accept()` блокируется. Можно заранее проверить слушающий дескриптор при помощи `select()`: если он «готов к чтению», ожидает новое соединение, которое можно принять через `accept()`! Ура! Или можно установить на слушающем сокете флаг `O_NONBLOCK` при помощи `fcntl()`: тогда он не блокируется, а возвращает `-1` и устанавливает `errno` в `EWOULDBLOCK`.

Возвращённый `accept()` дескриптор — полноценный сокет, открытый и подключённый к удалённому узлу. Закончив работу с ним, вызовите `close()`.

Возвращаемое значение: дескриптор нового соединения; при ошибке `-1` с соответствующим значением `errno`.

Пример

```
struct sockaddr_storage their_addr;
socklen_t addr_size;
struct addrinfo hints, *res;
int sockfd, new_fd;

// Сначала заполняем адресные структуры при помощи getaddrinfo():
memset(&hints, 0, sizeof hints);
hints.ai_family = AF_UNSPEC; // IPv4 или IPv6
hints.ai_socktype = SOCK_STREAM;
hints.ai_flags = AI_PASSIVE; // подставить мой IP
getaddrinfo(NULL, MYPORT, &hints, &res);

// Создаём сокет, привязываем его и слушаем:
sockfd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
bind(sockfd, res->ai_addr, res->ai_addrlen);
listen(sockfd, BACKLOG);

// Принимаем входящее соединение:
addr_size = sizeof their_addr;
new_fd = accept(sockfd, (struct sockaddr *)&their_addr, &addr_size);
// Можно общаться через дескриптор new_fd!
```

См. также: `socket()`, `getaddrinfo()`, `listen()`, `struct sockaddr_in`.

9.2 bind()

Связать сокет с IP-адресом и номером порта.

Объявление

```
#include <sys/types.h>
#include <sys/socket.h>
int bind(int sockfd, struct sockaddr *my_addr, socklen_t addrlen);
```

Описание

Чтобы удалённая машина подключилась к вашему серверу, ей нужны IP-адрес и номер порта. Именно это позволяет задать `bind()`.

Сначала вызовите `getaddrinfo()`, чтобы заполнить `struct sockaddr` адресом и портом назначения. Затем создайте дескриптор сокета через `socket()` и передайте сокет и адрес в `bind()`. IP-адрес и порт волшебным образом (с помощью настоящего волшебства) привяжутся к сокету!

Если вы не знаете IP-адрес, знаете, что у машины он только один, или вам всё равно, какой адрес использовать, передайте флаг `AI_PASSIVE` в параметре `hints` функции `getaddrinfo()`. Тогда адресная часть `struct sockaddr` заполняется специальным значением, которое указывает `bind()` автоматически выбрать IP-адрес этого узла.

Что-что? Какое специальное значение записывается в IP-адрес `struct sockaddr` для автоматического выбора адреса текущего узла? Я расскажу, но помните: это нужно только при заполнении структуры вручную; иначе используйте результат `getaddrinfo()`, как описано выше. Для IPv4 в поле `sin_addr.s_addr` структуры `sockaddr_in` записывается `INADDR_ANY`. Для IPv6 полю `sin6_addr` структуры `sockaddr_in6` присваивается глобальная переменная `in6addr_any`. А при объявлении новой структуры `in6_addr` можно инициализировать её значением `IN6ADDR_ANY_INIT`.

Наконец, параметр `addrlen` должен быть равен `sizeof my_addr`.

Возвращаемое значение: ноль при успехе или `-1` при ошибке с соответствующим значением `errno`.

Примеры

```
// Современный способ с getaddrinfo()
struct addrinfo hints, *res;
int sockfd;

memset(&hints, 0, sizeof hints);
hints.ai_family = AF_UNSPEC; // IPv4 или IPv6
hints.ai_socktype = SOCK_STREAM;
hints.ai_flags = AI_PASSIVE; // подставить мой IP
getaddrinfo(NULL, "3490", &hints, &res);

// Создаём сокет. В настоящей программе следует пройти связный список
// res и проверить ошибки!
sockfd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
// Привязываем к порту, переданному getaddrinfo():
bind(sockfd, res->ai_addr, res->ai_addrlen);
```

```
// Пример ручного заполнения структуры IPv4
struct sockaddr_in myaddr;
int s;
myaddr.sin_family = AF_INET;
myaddr.sin_port = htons(3490);

// Можно задать IP-адрес:
inet_pton(AF_INET, "63.161.169.137", &(myaddr.sin_addr));
// Или разрешить автоматический выбор:
myaddr.sin_addr.s_addr = INADDR_ANY;
```

```
s = socket(PF_INET, SOCK_STREAM, 0);
bind(s, (struct sockaddr*)&myaddr, sizeof myaddr);
```

См. также: `getaddrinfo()`, `socket()`, `struct sockaddr_in`, `struct in_addr`.

> Примечание переводчика: `INADDR_ANY` означает привязку ко всем доступным локальным адресам, а не выбор одного адреса. В `addrlen` передают размер структуры адреса, на которую указывает параметр, а не размер указателя.

9.3 connect()

Подключить сокет к серверу.

Объявление

```
#include <sys/types.h>
#include <sys/socket.h>
int connect(int sockfd, const struct sockaddr *serv_addr, socklen_t addrlen);
```

Описание

Создав дескриптор через `socket()`, вы можете подключить его к удалённому серверу с помощью удачно названного системного вызова `connect()`. Достаточно передать дескриптор и адрес сервера, с которым вы хотите познакомиться поближе. Ах да, ещё длину адреса, которую обычно передают подобным функциям.

Обычно эта информация получается из `getaddrinfo()`, но при желании можно заполнить собственную структуру `sockaddr`.

Если `bind()` ещё не вызывался для дескриптора, сокет автоматически привязывается к вашему IP-адресу и случайному локальному порту. Для клиента это обычно подходит: локальный порт вам не важен, важен только удалённый, который задаётся в `serv_addr`. Если клиентскому сокету обязательно нужен определённый IP-адрес и порт, можно вызвать `bind()`, но такое требуется редко.

После подключения можно отправлять и получать данные через `send()` и `recv()` сколько душе угодно. Особое замечание: если подключить через `connect()` UDP-сокет `SOCK_DGRAM` к удалённому узлу, можно использовать и `send()` с `recv()`, и `sendto()` с `recvfrom()`.

Возвращаемое значение: ноль при успехе или `-1` при ошибке с соответствующим значением `errno`.

```
// Подключаемся к порту 80 (HTTP) на www.example.com
struct addrinfo hints, *res;
int sockfd;
memset(&hints, 0, sizeof hints);
hints.ai_family = AF_UNSPEC; // IPv4 или IPv6
hints.ai_socktype = SOCK_STREAM;
// Вместо "http" в следующей строке можно было указать "80":
getaddrinfo("www.example.com", "http", &hints, &res);
sockfd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
connect(sockfd, res->ai_addr, res->ai_addrlen);
```

См. также: `socket()`, `bind()`.

9.4 close()

Закрыть дескриптор сокета.

```
#include <unistd.h>
int close(int s);
```

Закончив использовать сокет для своей безумной затеи, когда вы больше не собираетесь ни отправлять, ни получать данные, ни вообще делать что-либо ещё с этим сокетом, вызовите `close()`: он освободится и больше не будет использоваться.

Удалённая сторона может обнаружить это двумя способами. Во-первых, её вызов `recv()` вернёт `0`. Во-вторых, при вызове `send()` она получит сигнал `SIGPIPE`, а `send()` вернёт `-1` и установит `errno` в `EPIPE`.

Пользователи Windows: нужная вам функция называется `closesocket()`, а не `close()`. Если попытаться вызвать `close()` для дескриптора сокета, Windows может рассердиться... А вам не понравится, когда она сердится.

Возвращаемое значение: ноль при успехе или `-1` при ошибке с соответствующим значением `errno`.

```
s = socket(PF_INET, SOCK_DGRAM, 0);
// ... множество всяких действий... *БРРРОННН!* ...
close(s); // ничего сложного.
```

См. также: `socket()`, `shutdown()`.

9.5 getaddrinfo(), freeaddrinfo(), gai_strerror()

Получить сведения об имени узла и/или службе и заполнить результатом `struct sockaddr`.

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
int getaddrinfo(const char *nodename, const char *servname,
                const struct addrinfo *hints, struct addrinfo **res);
void freeaddrinfo(struct addrinfo *ai);
const char *gai_strerror(int ecode);

struct addrinfo {
    int ai_flags;              // AI_PASSIVE, AI_CANONNAME, ...
    int ai_family;             // AF_xxx
    int ai_socktype;           // SOCK_xxx
    int ai_protocol;           // 0 (автоматически), IPPROTO_TCP или IPPROTO_UDP
    socklen_t ai_addrlen;      // длина ai_addr
    char *ai_canonname;        // каноническое имя nodename
    struct sockaddr *ai_addr;  // двоичный адрес
    struct addrinfo *ai_next;  // следующая структура связного списка
};
```

Описание

`getaddrinfo()` — превосходная функция: она возвращает сведения об указанном имени узла, например его IP-адрес, и заполняет для вас `struct sockaddr`, беря на себя неприятные подробности вроде различий IPv4 и IPv6. Она заменяет старые функции `gethostbyname()` и `getservbyname()`. В описании ниже много

информации, которая может поначалу испугать, но пользоваться функцией довольно просто. Возможно, сначала стоит посмотреть примеры.

Интересующее имя передаётся в `nodename`. Это может быть имя узла, например `www.example.com`, или адрес IPv4 либо IPv6, переданный строкой. При использовании флага `AI_PASSIVE` здесь можно передать `NULL`.

Параметр `servname` по сути задаёт номер порта: числом в строке, например `"80"`, или именем службы, например `"http"`, `"tftp"`, `"smtp"` или `"pop"`. Общеизвестные имена служб приведены в списке портов IANA или в файле `/etc/services`.

Последний входной параметр — `hints`. Именно здесь вы определяете поведение `getaddrinfo()`. Перед использованием обнулите всю структуру с помощью `memset()`. Рассмотрим необходимые поля.

Полю `ai_flags` можно задать различные значения. Несколько флагов объединяют побитовым ИЛИ (|). Полный список есть в странице руководства вашей системы. Флаг `AI_CANONNAME` заполняет `ai_canonname` результата каноническим, то есть настоящим, именем узла. `AI_PASSIVE` заполняет IP-адрес результата значением `INADDR_ANY` для IPv4 или `in6addr_any` для IPv6; последующий вызов `bind()` автоматически использует IP-адрес текущего узла. Это удобно при настройке сервера, когда не хочется жёстко задавать адрес. При использовании `AI_PASSIVE` можно передать `NULL` в `nodename`, поскольку адрес будет задан позднее через `bind()`.

Продолжая с входными параметрами: обычно `ai_family` устанавливают в `AF_UNSPEC`, чтобы искать и IPv4, и IPv6. Можно ограничиться одним семейством с помощью `AF_INET` или `AF_INET6`. Затем задайте `ai_socktype` равным `SOCK_STREAM` или `SOCK_DGRAM` в зависимости от нужного типа сокета. Наконец, оставьте `ai_protocol` равным нулю для автоматического выбора протокола.

Теперь можно вызвать `getaddrinfo()`! Вот тут и начинается самое интересное. `res` будет указывать на связный список структур `addrinfo`; его можно обойти, получив все адреса, соответствующие параметрам `hints`. Некоторые адреса могут не работать по тем или иным причинам, поэтому пример в руководстве Linux перебирает список, вызывая `socket()` и `connect()` либо `bind()` при настройке сервера с `AI_PASSIVE`, пока не добьётся успеха.

Закончив работу со списком, вызовите `freeaddrinfo()` для освобождения памяти. Иначе произойдёт утечка, и Некоторые Люди расстроятся.

Возвращаемое значение: ноль при успехе, ненулевое значение при ошибке. В последнем случае `gai_strerror()` возвращает представление кода ошибки в виде строки.

Пример: клиент, подключающийся потоковым сокетом к порту 80 на www.example.com по IPv4 или IPv6

```
int sockfd;
struct addrinfo hints, *servinfo, *p;
int rv;

memset(&hints, 0, sizeof hints);
hints.ai_family = AF_UNSPEC; // AF_INET6 принудительно задаёт IPv6
hints.ai_socktype = SOCK_STREAM;
rv = getaddrinfo("www.example.com", "http", &hints, &servinfo);
if (rv != 0) {
    fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(rv));
    exit(1);
}
```

```
// Перебираем результаты до первого успешного подключения
for (p = servinfo; p != NULL; p = p->ai_next) {
    if ((sockfd = socket(p->ai_family, p->ai_socktype,
                         p->ai_protocol)) == -1) {
        perror("socket");
        continue;
    }
    if (connect(sockfd, p->ai_addr, p->ai_addrlen) == -1) {
        perror("connect");
        close(sockfd);
        continue;
    }
    break; // если мы здесь, подключение успешно
}
if (p == NULL) {
    fprintf(stderr, "failed to connect\n");
    exit(2);
}
freeaddrinfo(servinfo); // структура больше не нужна
```

Пример: сервер, ожидающий соединения на порту 3490 по IPv4 или IPv6

```
int sockfd;
struct addrinfo hints, *servinfo, *p;
int rv;

memset(&hints, 0, sizeof hints);
hints.ai_family = AF_UNSPEC; // AF_INET6 принудительно задаёт IPv6
hints.ai_socktype = SOCK_STREAM;
hints.ai_flags = AI_PASSIVE; // использовать мой IP-адрес
if ((rv = getaddrinfo(NULL, "3490", &hints, &servinfo)) != 0) {
    fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(rv));
    exit(1);
}

// Перебираем результаты до первой успешной привязки
for (p = servinfo; p != NULL; p = p->ai_next) {
    if ((sockfd = socket(p->ai_family, p->ai_socktype,
                         p->ai_protocol)) == -1) {
        perror("socket");
        continue;
    }
    if (bind(sockfd, p->ai_addr, p->ai_addrlen) == -1) {
        close(sockfd);
        perror("bind");
        continue;
    }
    break; // если мы здесь, привязка успешна
}
if (p == NULL) {
    fprintf(stderr, "failed to bind socket\n");
    exit(2);
}
freeaddrinfo(servinfo); // структура больше не нужна
```

См. также: `gethostbyname()`, `getnameinfo()`.

9.6 gethostname()

Получить имя системы.

```
#include <sys/unistd.h>
int gethostname(char *name, size_t len);
```

У вашей системы есть имя. Как и у всех остальных. Эта тема относится скорее к Unix, чем к сети, но всё равно бывает полезной. Например, можно получить имя узла и вызвать `gethostbyname()`, чтобы узнать его IP-адрес.

Параметр `name` должен указывать на буфер для имени, а `len` — содержать его размер в байтах. `gethostname()` не будет записывать за пределы буфера: функция может вернуть ошибку или просто остановить запись. Если в буфере достаточно места, строка завершается нулевым символом `NUL`.

Возвращаемое значение: ноль при успехе или `-1` при ошибке с соответствующим значением `errno`.

```
char hostname[128];
gethostname(hostname, sizeof hostname);
printf("My hostname: %s\n", hostname);
```

См. также: `gethostbyname()`.

9.7 gethostbyname(), gethostbyaddr()

Получить IP-адрес по имени узла или наоборот.

```
#include <sys/socket.h>
#include <netdb.h>
struct hostent *gethostbyname(const char *name); // УСТАРЕЛО!
struct hostent *gethostbyaddr(const char *addr, int len, int type);
```

Обратите внимание: эти функции заменены на `getaddrinfo()` и `getnameinfo()`! В частности, `gethostbyname()` плохо работает с IPv6.

Они преобразуют имена узлов в IP-адреса и обратно. Например, по имени `www.example.com` можно получить адрес с помощью `gethostbyname()` и сохранить его в `struct in_addr`. И наоборот, имея `struct in_addr` или `struct in6_addr`, можно получить имя при помощи `gethostbyaddr()`. Последняя совместима с IPv6, но лучше применять более новую и блестящую `getnameinfo()`.

Если IP-адрес, имя которого нужно узнать, задан строкой с числами и точками, лучше использовать `getaddrinfo()` с флагом `AI_CANONNAME`.

`gethostbyname()` принимает строку вроде `www.yahoo.com` и возвращает `struct hostent` с массой сведений, включая IP-адрес: официальное имя, список псевдонимов, тип и длину адресов, список адресов. Это универсальная структура, которую легко приспособить к нашим задачам, когда разберётесь как.

`gethostbyaddr()` принимает `struct in_addr` или `struct in6_addr` и возвращает соответствующее имя, если оно существует: это своего рода обратная операция. Хотя параметр `addr` имеет тип `char *`, на самом деле следует передать указатель на `struct in_addr`; `len` должен быть равен `sizeof(struct in_addr)`, а `type` — `AF_INET`.

Как устроена возвращаемая `struct hostent`? Она содержит следующие поля:

Поле	Описание
`char *h_name`	Настоящее каноническое имя узла.
`char **h_aliases`	Список псевдонимов, доступный как массив; последний элемент — `NULL`.
`int h_addrtype`	Тип адреса результата, для наших целей обычно `AF_INET`.
`int length`	Длина адреса в байтах: для IPv4 — 4.
`char **h_addr_list`	Список IP-адресов узла. Несмотря на тип `char **`, это фактически массив указателей `struct in_addr *`; последний элемент — `NULL`.
`h_addr`	Распространённый псевдоним `h_addr_list[0]`. Используйте его, если подходит любой IP-адрес узла; адресов может быть несколько.

Возвращаемое значение: указатель на результирующую структуру `hostent` при успехе или `NULL` при ошибке. Вместо обычных `perror()` и подобных средств ошибки этих функций отражаются в переменной `h_errno`; их можно вывести с помощью `herror()` или `hstrerror()`. Они работают аналогично привычным `errno`, `perror()` и `strerror()`.

```
// УСТАРЕВШИЙ СПОСОБ ПОЛУЧЕНИЯ ИМЕНИ УЗЛА
// Вместо него используйте getaddrinfo()!
#include <stdio.h>
#include <errno.h>
#include <netdb.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

int main(int argc, char *argv[])
{
    int i;
    struct hostent *he;
    struct in_addr **addr_list;
    if (argc != 2) {
        fprintf(stderr, "usage: ghbn hostname\n");
        return 1;
    }
    if ((he = gethostbyname(argv[1])) == NULL) {
        herror("gethostbyname");
        return 2;
    }
    printf("Official name is: %s\n", he->h_name);
    printf("    IP addresses: ");
    addr_list = (struct in_addr **)he->h_addr_list;
    for (i = 0; addr_list[i] != NULL; i++) {
        printf("%s ", inet_ntoa(*addr_list[i]));
    }
    printf("\n");
    return 0;
}
```

```
// ЭТОТ СПОСОБ УСТАРЕЛ: используйте getnameinfo()!
struct hostent *he;
struct in_addr ipv4addr;
struct in6_addr ipv6addr;
inet_pton(AF_INET, "192.0.2.34", &ipv4addr);
he = gethostbyaddr(&ipv4addr, sizeof ipv4addr, AF_INET);
```

```
printf("Host name: %s\n", he->h_name);
inet_pton(AF_INET6, "2001:db8:63b3:1::beef", &ipv6addr);
he = gethostbyaddr(&ipv6addr, sizeof ipv6addr, AF_INET6);
printf("Host name: %s\n", he->h_name);
```

См. также: `getaddrinfo()`, `getnameinfo()`, `gethostname()`, `errno`, `perror()`, `strerror()`, `struct in_addr`.

> Примечание переводчика: стандартное имя поля длины в `struct hostent` — `h_length`. При использовании IPv6 передают адрес и размер `struct in6_addr` и семейство `AF_INET6`.

9.8 getnameinfo()

Найти имя узла и службы для указанной структуры `sockaddr`.

```
#include <sys/socket.h>
#include <netdb.h>
int getnameinfo(const struct sockaddr *sa, socklen_t salen,
                char *host, size_t hostlen,
                char *serv, size_t servlen, int flags);
```

Эта функция обратна `getaddrinfo()`: она получает уже заполненную `struct sockaddr` и ищет соответствующие имя узла и службы. Она заменяет старые функции `gethostbyaddr()` и `getservbyport()`.

В `sa` передаётся указатель на `struct sockaddr` (обычно это приведённый к этому типу указатель на `struct sockaddr_in` или `struct sockaddr_in6`), а в `salen` — размер структуры. Результаты записываются в буферы `host` и `serv`, максимальные размеры которых указываются в `hostlen` и `servlen`.

Можно передавать несколько флагов; приведу пару полезных. `NI_NOFQDN` оставляет в `host` только имя узла без полного доменного имени. `NI_NAMEREQD` заставляет функцию завершиться ошибкой, если имя не найдено через DNS. Без этого флага вместо ненайденного имени в `host` будет помещён IP-адрес в виде строки. Как всегда, за подробностями обращайтесь к локальным страницам руководства.

Возвращаемое значение: ноль при успехе, ненулевое значение при ошибке. Последнее можно передать в `gai_strerror()`, чтобы получить понятное описание. Подробнее см. `getaddrinfo()`.

```
struct sockaddr_in6 sa; // при желании можно использовать IPv4
char host[1024];
char service[20];
// Предположим, sa заполнена правильными данными об узле и порте...
getnameinfo(&sa, sizeof sa, host, sizeof host, service,
            sizeof service, 0);
printf("   host: %s\n", host);       // например, "www.example.com"
printf("service: %s\n", service);   // например, "http"
```

См. также: `getaddrinfo()`, `gethostbyaddr()`.

9.9 getpeername()

Получить адресные сведения об удалённой стороне соединения.

```
#include <sys/socket.h>
int getpeername(int s, struct sockaddr *addr, socklen_t *len);
```

Приняв удалённое соединение через `accept()` или подключившись к серверу через `connect()`, вы получаете так называемого партнёра (*peer*). Это просто компьютер, с которым вы соединены, определяемый IP-адресом и портом. `getpeername()` возвращает `struct sockaddr_in` со сведениями об этой машине.

Почему используется слово «имя»? Есть множество видов сокетов, не только интернет-сокеты из нашего руководства, и «имя» — удобный общий термин для всех случаев. В нашем случае «имя» партнёра — IP-адрес и порт. Хотя функция возвращает размер полученного адреса в `len`, перед вызовом в `len` необходимо записать размер `addr`.

Возвращаемое значение: ноль при успехе или `-1` при ошибке, с соответствующим значением `errno`.

```
// Предположим, s — подключённый сокет
socklen_t len;
struct sockaddr_storage addr;
char ipstr[INET6_ADDRSTRLEN];
int port;
len = sizeof addr;
getpeername(s, (struct sockaddr*)&addr, &len);

// Обрабатываем и IPv4, и IPv6:
if (addr.ss_family == AF_INET) {
    struct sockaddr_in *s = (struct sockaddr_in *)&addr;
    port = ntohs(s->sin_port);
    inet_ntop(AF_INET, &s->sin_addr, ipstr, sizeof ipstr);
} else { // AF_INET6
    struct sockaddr_in6 *s = (struct sockaddr_in6 *)&addr;
    port = ntohs(s->sin6_port);
    inet_ntop(AF_INET6, &s->sin6_addr, ipstr, sizeof ipstr);
}
printf("Peer IP address: %s\n", ipstr);
printf("Peer port      : %d\n", port);
```

См. также: `gethostname()`, `gethostbyname()`, `gethostbyaddr()`.

9.10 errno

Содержит код ошибки последнего системного вызова.

```
#include <errno.h>
int errno;
```

Эта переменная хранит информацию об ошибках многих системных вызовов. Как вы помните, такие функции, как `socket()` и `listen()`, при ошибке возвращают `-1` и записывают в `errno` точный код, сообщающий, что именно произошло.

Заголовок `errno.h` перечисляет символические константы ошибок, например `EADDRINUSE`, `EPIPE`, `ECONNREFUSED` и другие. На страницах руководства вашей системы указано, какие коды могут возникнуть; во время работы программы вы можете по-разному обрабатывать разные ошибки. Чаще вызывают `perror()` или `strerror()`, чтобы получить понятное человеку описание.

Для любителей многопоточности: в большинстве систем `errno` определена безопасным для потоков образом. На самом деле это не глобальная переменная, хотя в однопоточной программе ведёт себя так же.

Значение: код последней произошедшей ошибки, который может обозначать успех, если последнее действие завершилось успешно.

```
s = socket(PF_INET, SOCK_STREAM, 0);
if (s == -1) {
    perror("socket"); // или используйте strerror()
}
tryagain:
if (select(n, &readfds, NULL, NULL) == -1) {
    // произошла ошибка!!
    // если нас только прервали, перезапускаем select():
    if (errno == EINTR) goto tryagain; // AAAA! goto!!!
    // иначе ошибка серьёзнее:
    perror("select");
    exit(1);
}
```

См. также: `perror()`, `strerror()`.

> Примечание переводчика: успешный вызов не обязан обнулять `errno`, поэтому проверяйте его только тогда, когда результат функции указывает на ошибку. В приведённом автором вызове `select()` отсутствует пятый аргумент `timeout`; добавьте `NULL`, если нужно ждать без ограничения времени. При повторе `select()` также необходимо заново подготовить наборы дескрипторов и тайм-аут.

9.11 fcntl()

Управлять дескрипторами сокетов.

```
#include <sys/unistd.h>
#include <sys/fcntl.h>
int fcntl(int s, int cmd, long arg);
```

Эта функция обычно используется для блокировки файлов и других файловых операций, но у неё также есть несколько связанных с сокетами возможностей. Параметр `s` — нужный дескриптор сокета, `cmd` следует установить в `F_SETFL`, а в `arg` передать одну из следующих команд. Как я говорил, `fcntl()` умеет больше, но здесь я сосредоточен на сокетах.

Значение	Описание
`O_NONBLOCK`	Установить неблокирующий режим сокета. Подробнее см. раздел о блокировке.
`O_ASYNC`	Включить асинхронный ввод-вывод. Когда данные готовы к получению через `recv()`, возникает сигнал `SIGIO`. Это встречается редко, выходит за рамки руководства и, кажется, доступно только в некоторых системах.

Возвращаемое значение: ноль при успехе или `-1` при ошибке с соответствующим значением `errno`. У других вариантов использования `fcntl()` могут быть другие возвращаемые значения, но здесь они не рассмотрены, поскольку не относятся к сокетам. Подробнее см. локальную страницу `fcntl()`.

```
int s = socket(PF_INET, SOCK_STREAM, 0);
fcntl(s, F_SETFL, O_NONBLOCK); // неблокирующий режим
fcntl(s, F_SETFL, O_ASYNC);    // асинхронный ввод-вывод
```

См. также: раздел о блокировке, `send()`.

> Примечание переводчика: `F_SETFL` задаёт флаги состояния файла, поэтому для сохранения уже установленных флагов обычно сначала получают их через `F_GETFL`, а затем добавляют нужные побитовым ИЛИ. Два последовательных вызова выше могут перезаписать ранее заданные флаги.

9.12 htons(), htonl(), ntohs(), ntohl()

Преобразовать многобайтовые целые числа между порядком байтов машины и сетевым порядком.

```
#include <netinet/in.h>
uint32_t htonl(uint32_t hostlong);
uint16_t htons(uint16_t hostshort);
uint32_t ntohl(uint32_t netlong);
uint16_t ntohs(uint16_t netshort);
```

Чтобы вам жизнь мёдом не казалась, разные компьютеры по-разному располагают байты многобайтовых целых чисел, то есть чисел больше `char`. Поэтому, отправив двухбайтовый `short int` с Intel на Mac (я имею в виду времена до перехода Mac на Intel), можно обнаружить, что число 1 на одном компьютере выглядит как 256 на другом, и наоборот.

Чтобы обойти проблему, все откладывают разногласия, соглашаются, что Motorola и IBM были правы, а Intel всё сделал странно, и перед отправкой преобразуют байты к порядку *big-endian*. Поскольку Intel использует *little-endian*, политически корректнее называть выбранный порядок «сетевым порядком байтов». Эти функции преобразуют родной порядок в сетевой и обратно.

На Intel они переставляют байты; на PowerPC ничего не делают, потому что там порядок уже сетевой. Тем не менее всегда используйте их в коде: кто-то может собрать его на Intel и захочет, чтобы он работал правильно. Типы здесь — 32-битные числа (4 байта, вероятно `int`) и 16-битные (2 байта, скорее всего `short`).

В разных системах есть и 64-битные варианты. Посмотрите функцию `htobe64()` и родственные ей из `<endian.h>`, если этот заголовок у вас есть (на macOS его, видимо, нет). Во встроенных функциях GCC для перестановки байтов есть даже 128-битные варианты. Можно написать свою, но выполнять перестановку только на машине с little-endian!

Чтобы выбрать функцию, сначала определите направление: от порядка машины (*host*, первая буква `h`) или от сетевого (*network*, первая буква `n`). В середине всегда `to`, а следующая буква обозначает порядок назначения. Последняя буква — размер: `s` для short, `l` для long.

Функция	Направление
`htons()`	host → network, short
`htonl()`	host → network, long
`ntohs()`	network → host, short
`ntohl()`	network → host, long

Каждая функция возвращает преобразованное значение.

```
uint32_t some_long = 10;
uint16_t some_short = 20;
uint32_t network_byte_order;
// Преобразуем и отправляем
network_byte_order = htonl(some_long);
send(s, &network_byte_order, sizeof(uint32_t), 0);
some_short == ntohs(htons(some_short)); // выражение истинно
```

9.13 inet_ntoa(), inet_aton(), inet_addr()

Преобразовать IP-адрес между строкой с числами и точками и структурой `in_addr`.

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
// ВСЕ ЭТИ ФУНКЦИИ УСТАРЕЛИ!
// Вместо них используйте inet_pton() или inet_ntop()!
char *inet_ntoa(struct in_addr in);
int inet_aton(const char *cp, struct in_addr *inp);
in_addr_t inet_addr(const char *cp);
```

Эти функции устарели, поскольку не поддерживают IPv6! Вместо них используйте `inet_ntop()` или `inet_pton()`. Они включены сюда, потому что по-прежнему встречаются в реальных программах.

Они преобразуют `struct in_addr` (скорее всего, часть `struct sockaddr_in`) в строку с числами и точками, например `192.168.5.10`, и обратно. Если IP-адрес передан через командную строку, это простейший способ получить `struct in_addr` для `connect()` или другой операции. Если нужна большая мощь, попробуйте DNS-функции вроде `gethostbyname()` или устройте государственный переворот в своей стране.

`inet_ntoa()` преобразует сетевой адрес в `struct in_addr` в строку. Буква `n` означает *network*, а `a` — *ASCII* по историческим причинам: «network to ASCII». В библиотеке C есть аналогичная функция `atoi()`, преобразующая строку ASCII в целое число.

`inet_aton()` выполняет обратную операцию, преобразуя строку в `in_addr_t` — тип поля `s_addr` структуры `in_addr`. Наконец, более старая `inet_addr()` делает примерно то же, что `inet_aton()`. Теоретически она устарела, но встречается часто, и полиция не придёт вас арестовывать, если вы ею воспользуетесь.

Возвращаемое значение: `inet_aton()` возвращает ненулевое значение для допустимого адреса и ноль для недопустимого. `inet_ntoa()` возвращает строку в статическом буфере, который перезаписывается при каждом вызове. `inet_addr()` возвращает адрес как `in_addr_t` или `-1` при ошибке. Такой же результат получается при преобразовании допустимого IP-адреса `255.255.255.255`, поэтому `inet_aton()` лучше.

```
struct sockaddr_in antelope;
char *some_addr;
inet_aton("10.0.0.1", &antelope.sin_addr); // сохраняем IP в antelope
some_addr = inet_ntoa(antelope.sin_addr);    // получаем IP обратно
printf("%s\n", some_addr);                  // выводит "10.0.0.1"
// Этот вызов аналогичен inet_aton() выше:
antelope.sin_addr.s_addr = inet_addr("10.0.0.1");
```

См. также: `inet_ntop()`, `inet_pton()`, `gethostbyname()`, `gethostbyaddr()`.

9.14 inet_ntop(), inet_pton()

Преобразовать IP-адрес в удобочитаемую форму и обратно.

```
#include <arpa/inet.h>
const char *inet_ntop(int af, const void *src, char *dst, socklen_t size);
int inet_pton(int af, const char *src, void *dst);
```

Эти функции преобразуют удобочитаемые IP-адреса в двоичное представление для использования в системных вызовах и других функциях. `n` означает *network*, а `p` — *presentation*, или текстовое представление. Можно запомнить его как *printable*: `ntop` — «network to printable».

Иногда вместо набора двоичных чисел хочется увидеть IP-адрес в красивой печатной форме, например `192.0.2.180` или `2001:db8:8714:3a90::12`. Для этого есть `inet_ntop()`.

Параметр `af` задаёт семейство адресов — `AF_INET` или `AF_INET6`. `src` должен указывать на `struct in_addr` либо `struct in6_addr` с адресом, преобразуемым в строку. `dst` — указатель на строку назначения, `size` — её максимальная длина. Сколько места выделить? К счастью, для максимальной длины адресов есть макросы `INET_ADDRSTRLEN` и `INET6_ADDRSTRLEN`.

В других случаях у вас есть строка с читаемым IP-адресом, которую нужно упаковать в `struct sockaddr_in` или `struct sockaddr_in6`. Здесь нужна обратная функция `inet_pton()`. Она также принимает семейство в `af`; `src` указывает на строку адреса, а `dst` — на место для результата, обычно `struct in_addr` или `struct in6_addr`.

Эти функции не обращаются к DNS; для этого нужна `getaddrinfo()`.

Возвращаемое значение: `inet_ntop()` возвращает `dst` при успехе и `NULL` при ошибке, устанавливая `errno`. `inet_pton()` возвращает 1 при успехе, -1 при ошибке с установкой `errno` или 0, если входная строка не является допустимым IP-адресом.

```
// Демонстрация inet_ntop() и inet_pton() для IPv4
struct sockaddr_in sa;
char str[INET_ADDRSTRLEN];
inet_pton(AF_INET, "192.0.2.33", &(sa.sin_addr));
inet_ntop(AF_INET, &(sa.sin_addr), str, INET_ADDRSTRLEN);
printf("%s\n", str); // "192.0.2.33"
```

```
// Для IPv6 практически то же, только с кучей шестёрок
struct sockaddr_in6 sa;
char str[INET6_ADDRSTRLEN];
inet_pton(AF_INET6, "2001:db8:8714:3a90::12", &(sa.sin6_addr));
inet_ntop(AF_INET6, &(sa.sin6_addr), str, INET6_ADDRSTRLEN);
printf("%s\n", str); // "2001:db8:8714:3a90::12"
```

```
// Вспомогательная функция: преобразовать sockaddr IPv4 или IPv6 в строку
char *get_ip_str(const struct sockaddr *sa, char *s, size_t maxlen)
{
    switch (sa->sa_family) {
    case AF_INET:
        inet_ntop(AF_INET, &(((struct sockaddr_in *)sa)->sin_addr), s, maxlen);
        break;
    case AF_INET6:
        inet_ntop(AF_INET6, &(((struct sockaddr_in6 *)sa)->sin6_addr), s, maxlen);
        break;
```

```
    default:
        strncpy(s, "Unknown AF", maxlen);
        return NULL;
    }
    return s;
}
```

См. также: `getaddrinfo()`.

9.15 listen()

Перевести сокет в режим ожидания входящих соединений.

```
#include <sys/socket.h>
int listen(int s, int backlog);
```

Вы можете взять дескриптор, созданный `socket()`, и указать ему слушать входящие соединения. Именно это, ребята, и отличает серверы от клиентов.

Параметр `backlog` может иметь несколько разные значения в зависимости от системы, но приблизительно он задаёт количество ожидающих соединений, после которого ядро начинает отклонять новые. Поэтому принимайте новые соединения через `accept()` достаточно быстро, чтобы очередь не заполнилась. Попробуйте установить около 10; если при большой нагрузке клиенты начнут получать `Connection refused`, увеличьте значение.

Перед `listen()` сервер должен вызвать `bind()`, чтобы привязаться к определённому номеру порта. Именно к этому порту на IP-адресе сервера будут подключаться клиенты.

Возвращаемое значение: ноль при успехе или `-1` при ошибке, с соответствующим значением `errno`.

```
struct addrinfo hints, *res;
int sockfd;
memset(&hints, 0, sizeof hints);
hints.ai_family = AF_UNSPEC; // IPv4 или IPv6
hints.ai_socktype = SOCK_STREAM;
hints.ai_flags = AI_PASSIVE; // подставить мой IP
getaddrinfo(NULL, "3490", &hints, &res);
sockfd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
bind(sockfd, res->ai_addr, res->ai_addrlen);
listen(sockfd, 10); // настраиваем серверный сокет
// Затем где-то здесь будет цикл accept()
```

См. также: `accept()`, `bind()`, `socket()`.

9.16 perror(), strerror()

Вывести ошибку в понятном человеку виде.

```
#include <stdio.h>
#include <string.h> // для strerror()
void perror(const char *s);
char *strerror(int errnum);
```

Многие функции возвращают `-1` при ошибке и устанавливают `errno` в некоторое число. Было бы здорово легко напечатать это число в понятной форме. К счастью, `perror()` именно это и делает. Если перед сообщением нужно вывести дополнительное описание, передайте его в `s`; можно оставить `s` равным `NULL`, тогда ничего дополнительного не выводится. Вкратце, функция преобразует значения вроде `ECONNRESET` в понятный текст вроде «Connection reset by peer.».

`strerror()` очень похожа на `perror()`, но возвращает указатель на строку сообщения для указанного значения; обычно ей передают `errno`.

Возвращаемое значение: `strerror()` возвращает указатель на строку сообщения об ошибке.

```
int s;
s = socket(PF_INET, SOCK_STREAM, 0);
if (s == -1) {
    perror("socket error"); // выводит префикс и сообщение об ошибке
}
if (listen(s, 10) == -1) {
    printf("an error: %s\n", strerror(errno));
}
```

См. также: `errno`.

9.17 poll()

Одновременно проверить события на нескольких сокетах.

```
#include <sys/poll.h>
int poll(struct pollfd *ufds, unsigned int nfds, int timeout);
```

Эта функция очень похожа на `select()`: обе следят за наборами дескрипторов и событиями, такими как готовность входящих данных для `recv()`, готовность сокета к `send()`, внеполосные данные, ошибки и тому подобное.

В `ufds` передаётся массив из `nfds` структур `pollfd`, а также время ожидания в миллисекундах (1000 миллисекунд в секунде). Отрицательное время означает бесконечное ожидание. Если до истечения времени ни на одном сокете не произошло событий, `poll()` вернётся.

Каждый элемент массива представляет один дескриптор:

```
struct pollfd {
    int fd;        // дескриптор сокета
    short events;  // битовая маска интересующих событий
    short revents; // после возврата — маска произошедших событий
};
```

До вызова запишите дескриптор в `fd`. Если он отрицателен, структура игнорируется, а её `revents` устанавливается в ноль. Поле `events` составляется объединением следующих макросов побитовым ИЛИ:

Макрос	Назначение
`POLLIN`	Сообщить, когда данные готовы к получению через `recv()`.
`POLLOUT`	Сообщить, когда можно отправлять данные через `send()` без блокировки.

Макрос	Назначение
`POLLPRI`	Сообщить, когда готовы внеполосные данные.

После возврата `poll()` поле `revents` содержит побитовое ИЛИ произошедших событий. Дополнительно возможны следующие значения:

Макрос	Значение
`POLLERR`	На сокете произошла ошибка.
`POLLHUP`	Удалённая сторона разорвала соединение.
`POLLNVAL`	С дескриптором `fd` что-то не так; возможно, он не инициализирован.

Возвращаемое значение: количество элементов массива `ufds`, на которых произошли события; при истечении времени оно может быть равно нулю. При ошибке возвращается `-1` и устанавливается `errno`.

```
int s1, s2;
int rv;
char buf1[256], buf2[256];
struct pollfd ufds[2];
s1 = socket(PF_INET, SOCK_STREAM, 0);
s2 = socket(PF_INET, SOCK_STREAM, 0);
// Предположим, оба сокета подключены к серверу:
// connect(s1, ...)...
// connect(s2, ...)...

// На первом проверяем обычные и внеполосные данные,
// на втором — только обычные.
ufds[0].fd = s1;
ufds[0].events = POLLIN | POLLPRI;
ufds[1].fd = s2;
ufds[1].events = POLLIN;
rv = poll(ufds, 2, 3500); // ожидание 3,5 секунды
if (rv == -1) {
    perror("poll");
} else if (rv == 0) {
    printf("Timeout occurred! No data after 3.5 seconds.\n");
} else {
    if (ufds[0].revents & POLLIN) {
        recv(s1, buf1, sizeof buf1, 0);
    }
    if (ufds[0].revents & POLLPRI) {
        recv(s1, buf1, sizeof buf1, MSG_OOB);
    }
    if (ufds[1].revents & POLLIN) {
        recv(s1, buf2, sizeof buf2, 0);
    }
}
```

См. также: `select()`.

> Примечание переводчика: в последнем вызове `recv()` в примере автора, вероятно, опечатка: при проверке `ufds[1]` следует читать из `s2`, а не `s1`.

9.18 recv(), recvfrom()

Получить данные из сокета.

```
#include <sys/types.h>
#include <sys/socket.h>
ssize_t recv(int s, void *buf, size_t len, int flags);
ssize_t recvfrom(int s, void *buf, size_t len, int flags,
                 struct sockaddr *from, socklen_t *fromlen);
```

Создав и подключив сокет, вы можете читать данные удалённой стороны через `recv()` для TCP-сокетов `SOCK_STREAM` и `recvfrom()` для UDP-сокетов `SOCK_DGRAM`.

Обе функции принимают дескриптор `s`, указатель на буфер `buf`, его размер в байтах `len` и набор управляющих флагов. Кроме того, `recvfrom()` принимает указатель `from` на `struct sockaddr`, куда записывается адрес отправителя, и заполняет `fromlen` размером адреса. Перед вызовом необходимо инициализировать `fromlen` размером буфера `from`.

Какие замечательные флаги можно передать? Вот некоторые из них; сведения о поддержке в вашей системе ищите в местных страницах руководства. Объединяйте флаги побитовым ИЛИ или задайте `flags` равным нулю для обычного `recv()`.

Флаг	Описание
`MSG_OOB`	Получить внеполосные данные, отправленные с флагом `MSG_OOB` в `send()`. Получателю может быть послан сигнал `SIGURG`; в его обработчике можно вызвать `recv()` с этим флагом.
`MSG_PEEK`	Выполнить `recv()` «понарошку»: посмотреть, что ожидает в буфере до настоящего вызова без `MSG_PEEK`. Это предварительный просмотр следующего чтения.
`MSG_WAITALL`	Попросить `recv()` не возвращаться, пока не получены все `len` байт. В исключительных обстоятельствах — сигнал, ошибка, закрытие соединения удалённой стороной и так далее — ваше желание будет проигнорировано. Не сердитесь.

Обычно `recv()` блокируется до появления данных. Чтобы этого избежать, переведите сокет в неблокирующий режим или сначала проверьте наличие входящих данных через `select()` либо `poll()`.

Возвращаемое значение: фактическое число полученных байт (оно может быть меньше `len`) либо `-1` при ошибке с соответствующим `errno`. Если удалённая сторона закрыла соединение, `recv()` вернёт 0. Это обычный способ обнаружить закрытие соединения. Нормальность — хорошо, бунтарь!

```
// Потоковые сокеты и recv()
struct addrinfo hints, *res;
int sockfd;
char buf[512];
int byte_count;
memset(&hints, 0, sizeof hints);
hints.ai_family = AF_UNSPEC;
hints.ai_socktype = SOCK_STREAM;
getaddrinfo("www.example.com", "3490", &hints, &res);
sockfd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
connect(sockfd, res->ai_addr, res->ai_addrlen);
// Подключились и теперь можем получить данные!
byte_count = recv(sockfd, buf, sizeof buf, 0);
printf("recv()'d %d bytes of data in buf\n", byte_count);
```

```
// Дейтаграммные сокеты и recvfrom()
struct addrinfo hints, *res;
int sockfd;
int byte_count;
socklen_t fromlen;
struct sockaddr_storage addr;
char buf[512];
char ipstr[INET6_ADDRSTRLEN];
memset(&hints, 0, sizeof hints);
hints.ai_family = AF_UNSPEC;
hints.ai_socktype = SOCK_DGRAM;
hints.ai_flags = AI_PASSIVE;
getaddrinfo(NULL, "4950", &hints, &res);
sockfd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
bind(sockfd, res->ai_addr, res->ai_addrlen);
// accept() не нужен — сразу recvfrom():
fromlen = sizeof addr;
byte_count = recvfrom(sockfd, buf, sizeof buf, 0, &addr, &fromlen);
printf("recv()'d %d bytes of data in buf\n", byte_count);
printf("from IP address %s\n",
       inet_ntop(addr.ss_family,
                 addr.ss_family == AF_INET ?
                 ((struct sockaddr_in *)&addr)->sin_addr :
                 ((struct sockaddr_in6 *)&addr)->sin6_addr,
                 ipstr, sizeof ipstr);
```

См. также: `send()`, `sendto()`, `select()`, `poll()`, раздел о блокировке.

> Примечание переводчика: в последнем примере оригинала есть ошибки типов и пропущена скобка. Для `recvfrom()` нужно привести `&addr` к `struct sockaddr *`. В `inet_ntop()` нужно передать указатель на соответствующее поле адреса, а не само поле, и закрыть скобки обоих вызовов. Нулевой результат для дейтаграммного сокета также может означать получение пустой дейтаграммы.

9.19 select()

Проверить готовность дескрипторов сокетов к чтению и записи.

```
#include <sys/select.h>
int select(int n, fd_set *readfds, fd_set *writefds,
           fd_set *exceptfds, struct timeval *timeout);
FD_SET(int fd, fd_set *set);
FD_CLR(int fd, fd_set *set);
FD_ISSET(int fd, fd_set *set);
FD_ZERO(fd_set *set);
```

`select()` позволяет одновременно проверить несколько сокетов: ожидают ли данные для `recv()`, можно ли отправлять данные через `send()` без блокировки и не возникло ли исключительное состояние.

Наборы дескрипторов заполняются макросами вроде `FD_SET()`. Готовый набор передаётся как `readfds`, если вас интересует готовность к получению, как `writefds` — если готовность к отправке, и/или как `exceptfds` — для проверки исключений (ошибок). Любые из этих параметров могут быть `NULL`, если события соответствующего вида вас не интересуют.

После возврата `select()` изменяет наборы, показывая готовность к чтению или записи и наличие исключений. Первый параметр `n` — наибольший числовой дескриптор сокета плюс один.

Последний параметр, `struct timeval *timeout`, задаёт время проверки. Функция вернётся по истечении времени либо при событии, что наступит раньше. В структуре `tv_sec` задаёт секунды, а `tv_usec` — добавляемые к ним микросекунды (1 000 000 в секунде).

Макрос	Действие
`FD_SET(fd, set)`	Добавить дескриптор в набор.
`FD_CLR(fd, set)`	Удалить дескриптор из набора.
`FD_ISSET(fd, set)`	Вернуть истину, если дескриптор входит в набор.
`FD_ZERO(set)`	Очистить набор.

Пользователям Linux: `select()` может сообщить о готовности к чтению, когда чтение фактически не готово, и следующий `read()` заблокируется. Можно обойти эту ошибку, установив на принимающем сокете `O_NONBLOCK` и игнорируя возникшую ошибку `EWOULDBLOCK`. Подробнее см. страницу `fcntl()`.

Возвращаемое значение: количество готовых дескрипторов при успехе, 0 при истечении времени или -1 при ошибке с соответствующим `errno`. Наборы изменяются и показывают готовые сокеты.

```
int s1, s2, n;
fd_set readfds;
struct timeval tv;
char buf1[256], buf2[256];
// Предположим, оба сокета уже созданы и подключены к серверу.
// s1 = socket(...); s2 = socket(...);
// connect(s1, ...)...
// connect(s2, ...)...
FD_ZERO(&readfds);
FD_SET(s1, &readfds);
FD_SET(s2, &readfds);
// Мы получили s2 вторым и считаем его большим:
n = s2 + 1;
// Ждём данные на любом из сокетов не более 10,5 секунды.
tv.tv_sec = 10;
tv.tv_usec = 500000;
rv = select(n, &readfds, NULL, NULL, &tv);
if (rv == -1) {
    perror("select");
} else if (rv == 0) {
    printf("Timeout occurred! No data after 10.5 seconds.\n");
} else {
    if (FD_ISSET(s1, &readfds)) {
        recv(s1, buf1, sizeof buf1, 0);
    }
    if (FD_ISSET(s2, &readfds)) {
        recv(s2, buf2, sizeof buf2, 0);
    }
}
```

См. также: `poll()`.

> Примечание переводчика: пример автора предполагает, что `s2` больше `s1`, но в общем случае нужно вычислить максимум. В нём также не объявлена переменная `rv`.

9.20 setsockopt(), getsockopt()

Установить или получить параметры сокета.

```
#include <sys/types.h>
#include <sys/socket.h>
int getsockopt(int s, int level, int optname, void *optval, socklen_t *optlen);
int setsockopt(int s, int level, int optname, const void *optval, socklen_t optlen);
```

Сокеты довольно гибко настраиваются. Настолько, что я даже не буду описывать здесь всё; к тому же возможности, вероятно, зависят от системы. Поговорим об основах.

Эти функции получают и устанавливают параметры сокета. В Linux сведения содержатся в странице `socket` раздела 7: наберите `man 7 socket`. Параметр `s` задаёт сокет, `level` — `SOL_SOCKET`, а `optname` — нужный параметр. Полный список см. в руководстве; вот некоторые интересные:

Параметр	Описание
`SO_BINDTODEVICE`	Привязать сокет к символьному имени устройства, например `eth0`, вместо привязки к IP-адресу через `bind()`. Имена устройств можно посмотреть командой Unix `ifconfig`.
`SO_REUSEADDR`	Разрешить другим сокетам привязываться к порту, если на нём уже нет активного слушающего сокета. Это помогает обойти ошибку «Address already in use» при перезапуске сервера после сбоя.
`SO_BROADCAST`	Разрешить UDP-сокетам `SOCK_DGRAM` отправлять и получать широковещательные пакеты. Для TCP-сокетов не делает ничего — НИЧЕГО!! Ха-ха-ха!

`optval` обычно указывает на `int` со значением параметра. Для логических значений ноль означает ложь, ненулевое — истину. И это абсолютный факт, если только в вашей системе не иначе. Если значение не требуется, `optval` может быть `NULL`.

`optlen` задаёт длину `optval`, часто `sizeof(int)`, но она зависит от параметра. В `getsockopt()` это указатель на `socklen_t`: он указывает максимальный размер записываемого объекта, защищая от переполнения буфера. Функция меняет значение на число фактически записанных байтов.

Предупреждение: в некоторых системах, особенно Sun и Windows, параметр может иметь тип `char`, а не `int`, и устанавливаться, например, в символьное значение `'1'` вместо целого 1. За подробностями обратитесь к `man setsockopt` и `man 7 socket`!

Возвращаемое значение: ноль при успехе или -1 при ошибке с соответствующим `errno`.

```
int optval;
int optlen;
char *optval2;
optval = 1;
setsockopt(s1, SOL_SOCKET, SO_REUSEADDR, &optval, sizeof optval);
optval2 = "eth1"; // длина 4 байта
setsockopt(s2, SOL_SOCKET, SO_BINDTODEVICE, optval2, 4);
getsockopt(s3, SOL_SOCKET, SO_BROADCAST, &optval, &optlen);
if (optval != 0) {
    print("SO_BROADCAST enabled on s3!\n");
}
```

См. также: `fcntl()`.

> Примечание переводчика: перед `getsockopt()` следует инициализировать `optlen` размером `optval` и использовать тип `socklen_t`; в примере `print` также, вероятно, опечатка вместо `printf`. Поведение `SO_REUSEADDR` зависит от платформы.

9.21 send(), sendto()

Отправить данные через сокет.

```
#include <sys/types.h>
#include <sys/socket.h>
ssize_t send(int s, const void *buf, size_t len, int flags);
ssize_t sendto(int s, const void *buf, size_t len, int flags,
               const struct sockaddr *to, socklen_t tolen);
```

Эти функции отправляют данные через сокет. Обычно `send()` применяют для подключённых TCP-сокетов `SOCK_STREAM`, а `sendto()` — для неподключённых UDP-сокетов `SOCK_DGRAM`. В последнем случае адрес назначения нужно задавать при каждой отправке, для чего и служат последние параметры `sendto()`.

В обеих функциях `s` — сокет, `buf` — указатель на отправляемые данные, `len` — число байт, а `flags` задаёт дополнительные условия отправки. Для обычных данных используйте ноль. Ниже некоторые распространённые флаги; подробности см. в локальной странице `send()`.

Флаг	Описание
`MSG_OOB`	Отправить внеполосные данные. TCP поддерживает этот способ обозначить данные как более приоритетные: получатель получает сигнал `SIGURG` и может прочитать их, не получив сначала все обычные данные из очереди.
`MSG_DONTROUTE`	Не отправлять данные через маршрутизатор, оставить их локальными.
`MSG_DONTWAIT`	Если `send()` заблокировался бы из-за загруженного исходящего канала, вернуть `EAGAIN`: неблокирующий режим только для этого вызова. См. раздел о блокировке.
`MSG_NOSIGNAL`	Не генерировать `SIGPIPE`, если удалённый узел больше не принимает данные.

Возвращаемое значение: фактическое число отправленных байт или -1 при ошибке с соответствующим `errno`. Отправленных байт может оказаться меньше запрошенного количества! См. вспомогательную функцию в разделе о частичных `send()`. Если сокет закрыт одной из сторон, процесс, вызывающий `send()`, получает `SIGPIPE`, если не задан `MSG_NOSIGNAL`.

```
int spatula_count = 3490;
char *secret_message = "The Cheese is in The Toaster";
int stream_socket, dgram_socket;
struct sockaddr_in dest;
int temp;
// TCP: предполагаем, что сокет создан и подключён.
// stream_socket = socket(...
// connect(stream_socket, ...
temp = htonl(spatula_count);
send(stream_socket, &temp, sizeof temp, 0);
send(stream_socket, secret_message, strlen(secret_message)+1, MSG_OOB);
// UDP: предполагаем, что dest содержит адрес и сокет создан.
// getaddrinfo(...
// dest = ... // адрес назначения
// dgram_socket = socket(...
```

```
sendto(dgram_socket, secret_message, strlen(secret_message)+1, 0,
       (struct sockaddr*)&dest, sizeof dest);
```

См. также: `recv()`, `recvfrom()`.

9.22 shutdown()

Запретить дальнейшую отправку и приём данных через сокет.

```
#include <sys/socket.h>
int shutdown(int s, int how);
```

Всё! С меня хватит! Больше никаких `send()` на этом сокете, но `recv()` я всё ещё хочу использовать! Или наоборот! Как это сделать?

`close()` закрывает обе стороны сокета — чтение и запись — и освобождает дескриптор. Если нужно закрыть только одну сторону, используйте `shutdown()`. Параметр `s` задаёт сокет, а `how` — действие: `SHUT_RD` запрещает дальнейшие `recv()`, `SHUT_WR` — `send()`, а `SHUT_RDWR` — и то и другое.

`shutdown()` не освобождает дескриптор, поэтому в конце всё равно нужно вызвать `close()`, даже если обе стороны сокета отключены. Этот системный вызов используется редко.

Возвращаемое значение: ноль при успехе или -1 при ошибке с соответствующим `errno`.

```
int s = socket(PF_INET, SOCK_STREAM, 0);
// ... здесь несколько send() и другие действия ...
shutdown(s, SHUT_WR); // теперь запрещаем дальнейшую отправку
```

См. также: `close()`.

9.23 socket()

Создать дескриптор сокета.

```
#include <sys/types.h>
#include <sys/socket.h>
int socket(int domain, int type, int protocol);
```

Функция возвращает новый дескриптор, с которым можно проделывать всякие сокетные вещи. Обычно это первый вызов в грандиозном процессе написания сетевой программы; результат используется в последующих вызовах `listen()`, `bind()`, `accept()` и многих других функций.

Обычно значения параметров получают через `getaddrinfo()`, как в примере ниже. Но при желании вы можете заполнить их вручную.

Параметр	Описание
`domain`	Определяет нужный вид сокета. Вариантов, поверьте, множество, но в нашем руководстве это `PF_INET` для IPv4 или `PF_INET6` для IPv6.
`type`	Вероятно, вы выберете `SOCK_STREAM` для надёжных TCP-сокетов (`send()`, `recv()`) или `SOCK_DGRAM` для быстрых, но ненадёжных UDP-сокетов (`sendto()`, `recvfrom()`). Ещё один интересный тип — `SOCK_RAW` для

Параметр	Описание
	ручного создания пакетов. Довольно круто.
`protocol`	Определяет протокол для типа сокета. Например, как уже говорилось, `SOCK_STREAM` использует TCP. К счастью, для `SOCK_STREAM` и `SOCK_DGRAM` достаточно задать 0 — подходящий протокол выберется автоматически. В остальных случаях можно найти номер протокола через `getprotobyname()`.

Возвращаемое значение: новый дескриптор для последующих вызовов или -1 при ошибке с соответствующим `errno`.

```
struct addrinfo hints, *res;
int sockfd;
// Сначала заполняем адресные структуры с помощью getaddrinfo():
memset(&hints, 0, sizeof hints);
hints.ai_family = AF_UNSPEC; // AF_INET, AF_INET6 или AF_UNSPEC
hints.ai_socktype = SOCK_STREAM; // SOCK_STREAM или SOCK_DGRAM
getaddrinfo("www.example.com", "3490", &hints, &res);
// Создаём сокет по информации, полученной от getaddrinfo():
sockfd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
```

См. также: `accept()`, `bind()`, `getaddrinfo()`, `listen()`.

9.24 struct sockaddr и её друзья

Структуры для работы с интернет-адресами.

```
#include <netinet/in.h>
// Указатели на структуры адресов сокетов перед различными
// функциями и системными вызовами часто приводят к этому типу:
struct sockaddr {
    unsigned short sa_family; // семейство адресов, AF_xxx
    char sa_data[14];         // 14 байт адреса протокола
};

// Сокеты IPv4 AF_INET:
struct sockaddr_in {
    short sin_family;         // например, AF_INET, AF_INET6
    unsigned short sin_port;  // например, htons(3490)
    struct in_addr sin_addr;  // см. struct in_addr ниже
    char sin_zero[8];         // при желании обнулите
};
struct in_addr {
    unsigned long s_addr;     // заполните через inet_pton()
};

// Сокеты IPv6 AF_INET6:
struct sockaddr_in6 {
    u_int16_t sin6_family;    // семейство адресов, AF_INET6
    u_int16_t sin6_port;      // номер порта, сетевой порядок
    u_int32_t sin6_flowinfo;  // сведения о потоке IPv6
    struct in6_addr sin6_addr;// адрес IPv6
    u_int32_t sin6_scope_id;  // идентификатор области
};
struct in6_addr {
    unsigned char s6_addr[16]; // заполните через inet_pton()
};

// Общая структура хранения адреса сокета; достаточно велика
// для struct sockaddr_in или struct sockaddr_in6:
```

```
struct sockaddr_storage {
    sa_family_t ss_family;    // семейство адресов
    // Далее заполнение, зависит от реализации; игнорируйте:
    char __ss_pad1[_SS_PAD1SIZE];
    int64_t __ss_align;
    char __ss_pad2[_SS_PAD2SIZE];
};
```

Это основные структуры всех системных вызовов и функций, работающих с интернет-адресами. Часто вы будете заполнять их при помощи `getaddrinfo()` и читать по мере необходимости.

В памяти `struct sockaddr_in` и `struct sockaddr_in6` начинаются так же, как `struct sockaddr`, и можно свободно приводить указатель одного типа к другому без последствий, кроме возможного конца Вселенной.

Шучу насчёт конца Вселенной... Если она всё-таки закончится, когда вы приведёте `struct sockaddr_in *` к `struct sockaddr *`, обещаю: это чистое совпадение, и волноваться совершенно не о чем.

Поэтому помните: когда функция принимает `struct sockaddr *`, можно просто и безопасно привести к этому типу свой `struct sockaddr_in *`, `struct sockaddr_in6 *` или `struct sockaddr_storage *`.

`struct sockaddr_in` используется для IPv4-адресов, например `192.0.2.10`. Она содержит семейство адресов `AF_INET`, порт `sin_port` и IPv4-адрес `sin_addr`.

Есть ещё поле `sin_zero`. Одни утверждают, что его нужно обнулять; другие не утверждают ничего (документация Linux его даже не упоминает). Судя по всему, обнуление не обязательно, но при желании можно сделать это с помощью `memset()`.

`struct in_addr` в разных системах устроена по-разному: иногда это странное объединение с кучей `#define` и прочими нелепостями. Используйте только поле `s_addr`, поскольку многие системы реализуют только его. `struct sockaddr_in6` и `struct in6_addr` похожи, но предназначены для IPv6.

`struct sockaddr_storage` можно передавать в `accept()` или `recvfrom()` при написании кода, независимого от версии IP, когда неизвестно, будет ли новый адрес IPv4 или IPv6. В отличие от исходной маленькой `struct sockaddr`, она достаточно велика для обоих типов.

```
// IPv4:
struct sockaddr_in ip4addr;
int s;
ip4addr.sin_family = AF_INET;
ip4addr.sin_port = htons(3490);
inet_pton(AF_INET, "10.0.0.1", &ip4addr.sin_addr);
s = socket(PF_INET, SOCK_STREAM, 0);
bind(s, (struct sockaddr*)&ip4addr, sizeof ip4addr);
```

```
// IPv6:
struct sockaddr_in6 ip6addr;
int s;
ip6addr.sin6_family = AF_INET6;
ip6addr.sin6_port = htons(4950);
inet_pton(AF_INET6, "2001:db8:8714:3a90::12", &ip6addr.sin6_addr);
s = socket(PF_INET6, SOCK_STREAM, 0);
bind(s, (struct sockaddr*)&ip6addr, sizeof ip6addr);
```

См. также: `accept()`, `bind()`, `connect()`, `inet_aton()`, `inet_ntoa()`.

> Примечание переводчика: приведённые объявления структур иллюстрируют принцип и могут отличаться от определений вашей платформы; используйте системные заголовки. Перед заполнением структур целесообразно обнулить их и проверять результаты вызовов.

Глава 10. Дополнительные источники

Вы добрались досюда и теперь требуете ещё! Где узнать больше обо всём этом?

10.1 Книги

Если вам нужны настоящие бумажные книги старой школы, которые можно подержать в руках, попробуйте следующие отличные издания. Ссылки перенаправляют на партнёрские страницы популярного книжного магазина и приносят мне приятное вознаграждение. Если вы просто хотите проявить щедрость, можете отправить пожертвование через PayPal на `beej@beej.us`. :-)

- W. Richard Stevens, *Unix Network Programming*, тома 1–2. Addison-Wesley Professional и Prentice Hall. ISBN: 978-0131411555, 978-0130810816.
- Douglas E. Comer, *Internetworking with TCP/IP*, том I. Pearson. ISBN 978-0136085300.
- W. Richard Stevens и Gary R. Wright, *TCP/IP Illustrated*, тома 1–3. Addison Wesley. ISBN: 978-0201633467, 978-0201633542, 978-0201634952; комплект из трёх томов, 978-0201776317.
- Craig Hunt, *TCP/IP Network Administration*. O'Reilly & Associates, Inc. ISBN 978-0596002978.
- W. Richard Stevens, *Advanced Programming in the UNIX Environment*. Addison Wesley. ISBN 978-0321637734.

10.2 Интернет-источники

В интернете:

- BSD Sockets: A Quick And Dirty Primer — также о системном программировании Unix.
- The Unix Socket FAQ.
- TCP/IP FAQ.
- The Winsock FAQ.

Несколько статей Википедии по теме:

- Сокеты Беркли.
- Internet Protocol (IP).
- Transmission Control Protocol (TCP).
- User Datagram Protocol (UDP).
- Клиент — сервер.
- Сериализация — упаковка и распаковка данных.

10.3 RFC

RFC — самая суть! Эти документы описывают присвоенные номера, программные интерфейсы и протоколы, применяемые в интернете. Для вашего удовольствия я привёл ссылки на некоторые из них. Берите ведро попкорна и включайте голову:

- RFC 1 — первый RFC: он даёт представление об «интернете» в момент его зарождения и о том, как его проектировали с нуля. Очевидно, документ полностью устарел!
- RFC 768 — User Datagram Protocol (UDP).
- RFC 791 — Internet Protocol (IP).
- RFC 793 — Transmission Control Protocol (TCP).
- RFC 854 — протокол Telnet.
- RFC 959 — File Transfer Protocol (FTP).
- RFC 1350 — Trivial File Transfer Protocol (TFTP).
- RFC 1459 — Internet Relay Chat (IRC).
- RFC 1918 — распределение адресов для частных сетей.
- RFC 2131 — Dynamic Host Configuration Protocol (DHCP).
- RFC 9110 — Hypertext Transfer Protocol (HTTP).
- RFC 2821 — Simple Mail Transfer Protocol (SMTP).
- RFC 3330 — IPv4-адреса специального назначения.
- RFC 3493 — базовые расширения интерфейса сокетов для IPv6.
- RFC 3542 — расширенный программный интерфейс сокетов для IPv6.
- RFC 3849 — префикс IPv6-адресов, зарезервированный для документации.
- RFC 3920 — Extensible Messaging and Presence Protocol (XMPP).
- RFC 3977 — Network News Transfer Protocol (NNTP).
- RFC 4193 — уникальные локальные одноадресные IPv6-адреса.
- RFC 4506 — стандарт внешнего представления данных (XDR).

У IETF есть удобный инструмент поиска и просмотра RFC.

Указатель

Порядок терминов и номера страниц относятся к английскому оригиналу версии 3.3.3.

- 10.x.x.x, 16
- 192.168.x.x, 16
- 255.255.255.255, 73, 100
- accept() функция, 26, 82
- Адрес уже используется, 25, 76
- AF_INET макрос, 13, 23, 79
- AF_INET6 макрос, 13
- Bapper, 75
- bind() функция, 23, 25, 76, 84
- неявный, 26
- Бла-бла-бла, 8
- Блокировка, 28, 42
- Широковещательная рассылка, 72
- BSD, 2
- Порядок байтов, 11, 14, 58, 98
- Клиент
- дейтаграммный, 39–40
- потоковый, 35–37
- Клиент и сервер, 32–41
- close() функция, 30, 87
- closesocket() функция, 3, 30, 87
- Компиляторы
- GCC, 1
- Сжатие, 78
- connect() функция, 6, 23, 25, 85
- для дейтаграммных сокетов, 30, 40, 86
- В соединении отказано, 37
- CreateProcess() функция, 4, 80
- CreateThread() функция, 4
- CSocket класс, 3
- Cygwin, 2
- Инкапсуляция данных, 8, 57
- концевик, 8
- заголовок, 8
- Дейтаграммные сокеты, 6–7

- DHCP, 119
- Ослы, 57
- EAGAIN макрос, 42, 112
- Письма Биджу, 4
- Шифрование, 78
- EPIPE макрос, 87
- errno переменная, 96, 103
- Ethernet, 8
- EWOULDBLOCK макрос, 42
- Excalibur, 72
- F_SETFL макрос, 97
- fcntl() функция, 42, 83, 97
- FD_CLR() макрос, 50, 109
- FD_ISSET() макрос, 50, 109
- FD_SET() макрос, 50, 109
- FD_ZERO() макрос, 50, 109
- Дескриптор файла, 6
- Межсетевой экран, 16, 75, 80
- открытие проходов, 80
- fork() функция, 3, 32, 80
- freeaddrinfo() функция, 87
- FTP, 119
- gai_strerror() функция, 87
- getaddrinfo() функция, 13, 17, 19, 31, 87
- gethostbyaddr() функция, 31, 91
- gethostbyname() функция, 91, 91
- gethostname() функция, 31, 91
- getnameinfo() функция, 17, 31, 94
- getpeername() функция, 30, 95
- getprotobyname() функция, 115
- getsockopt() функция, 110
- gettimeofday() функция, 52
- Коза, 76
- goto оператор, 77
- Заголовочные файлы, 76
- herror() функция, 92
- hstrerror() функция, 92
- htonl() функция, 12, 98
- htons() функция, 12, 14, 58, 98

- HTTP протокол, 7, 119
- ICMP, 76
- IEEE-754, 60
- illumos, 2
- INADDR_BROADCAST макрос, 73
- inet_addr() функция, 15, 99
- inet_aton() функция, 15, 99
- inet_ntoa() функция, 16, 99
- inet_ntop() функция, 15, 31, 100
- inet_pton() функция, 15, 100
- ioctl() функция, 80
- IP, 7, 8, 119
- IP-адрес, 9, 15, 24, 29, 31
- ip route команда, 76
- IPv4, 9
- IPv6, 9, 14, 16, 17
- IRC, 58, 119
- ISO/OSI, 8
- Многоуровневая сетевая модель, 8
- Linux, 2
- listen() функция, 23, 26, 102
- очередь ожидания, 26
- с select(), 52
- localhost, 76
- Устройство обратной связи, 76
- Страницы руководства man, 82
- Зеркала руководства, 4
- MSG_DONTROUTE макрос, 112
- MSG_DONTWAIT макрос, 112
- MSG_NOSIGNAL макрос, 112
- MSG_OOB макрос, 107, 112
- MSG_PEEK макрос, 107
- MSG_WAITALL макрос, 107
- MTU, 79
- NAT, 16
- netstat команда, 76
- NNTP, 120
- Неблокирующие сокеты, 42, 112
- ntohl() функция, 12, 98

- ntohs() функция, 12, 98
- O_ASYNC макрос, 97
- O_NONBLOCK макрос, 56, 83, 97, 109
- OpenSSL, 78
- Внеполосные данные, 107, 112
- Анализатор пакетов, 80
- Pat, 75
- perror() функция, 96, 103
- PF_INET макрос, 79, 114
- ping команда, 76
- poll(), 43–50
- poll() функция, 42, 56, 104
- Порт, 23, 24, 29
- Частная сеть, 16
- Неразборчивый режим, 80
- Сырые сокеты, 6, 76
- read() функция, 6
- recv() функция, 6, 28, 106
- тайм-аут, 77
- recvfrom() функция, 29, 106
- recvтайм-аут() функция, 78
- Источники
- книги, 118
- RFCs, 119–120
- веб-источники, 118–119
- RFCs, 119–120
- route команда, 76
- SA_RESTART макрос, 77
- Безопасность, 79
- select() функция, 3, 50–56, 76, 77, 108
- with listen(), 52
- send() функция, 6, 8, 28, 112
- sendall() функция, 57, 71
- sendto() функция, 8, 112
- Сериализация, 57–70
- Сервер
- дейтаграммный, 37–39
- потоковый, 32–35
- setsockopt() функция, 25, 72, 76, 80, 110

- SHUT_RD макрос, 113
- SHUT_RDWR макрос, 113
- SHUT_WR макрос, 113
- shutdown() функция, 30, 113
- sigaction() функция, 35, 77
- SIGIO сигнал, 97
- SIGPIPE макрос, 87, 112
- SIGURG макрос, 107, 112
- SMTP, 119
- SO_BINDTODEVICE макрос, 111
- SO_BROADCAST макрос, 72, 111
- SO_RCVTIMEO макрос, 80
- SO_REUSEADDR макрос, 25, 76, 111
- SO_SNDTIMEO макрос, 80
- SOCK_DGRAM макрос, 6, 7, 29, 106, 114
- SOCK_RAW макрос, 76, 114
- SOCK_STREAM макрос, 6, 106, 114
- Дескриптор сокета, 6, 12
- socket() функция, 6, 22, 114
- SOL_SOCKET макрос, 110
- Solaris, 1, 111
- SSL, 78
- Потоковые сокеты, 6
- strerror() функция, 96, 103
- struct addrinfo тип, 12
- struct hostent тип, 92
- struct in6_addr тип, 115
- struct in_addr тип, 115
- struct pollfd тип, 43, 104
- struct sockaddr тип, 13, 29, 106, 115
- struct sockaddr_in тип, 115
- struct sockaddr_in6 тип, 115
- struct sockaddr_storage тип, 115
- struct timeval тип, 50–52, 109
- SunOS, 1, 111
- TCP, 7, 119
- telnet, 6, 119
- TFTP, 8, 119
- Тайм-аут

- настройка, 80
- Перевод руководства, 4
- TRON, 25
- UDP, 7, 8, 72, 119
- Vint Cerf, 9
- Windows, 2, 30, 76, 87, 111
- Подсистема Windows для Linux, 2
- Winsock, 2, 30
- write() функция, 6
- WSACleanup() функция, 3
- WSAStartup() функция, 3
- WSL, 2
- XDR, 70, 120
- XMPP, 120
- Процесс-зомби, 35